



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

Erkennung und Filterung von Artefakten in Musikaufnahmen

Bachelorarbeit

von

Leif Benedikt Rehtanz

Matrikelnummer: 583463

Fachbereich 4 – Informatik, Kommunikation und Wirtschaft –
der Hochschule für Technik und Wirtschaft Berlin

zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

im Studiengang

Internationaler Studiengang Medieninformatik

Tag der Abgabe: 04.03.2025

Erstgutachten: Prof. Dr. Klaus Jung

Zweitgutachten: Prof. Dr. Carsten Busch

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Ziele der Arbeit	2
2	Grundlagen	3
2.1	Grundlagen der digitalen Signalverarbeitung	3
2.2	JUCE und die Programmierung mit digitalem Audio	4
2.3	Echtzeit- versus Offline-Verarbeitung	4
2.4	Fourier-Transformation	5
2.5	Was sind Artefakte in Musikaufnahmen?	6
2.5.1	Charakteristik von Sibilanten	7
2.6	Grundlagenbegriffe der Audiotbearbeitung	8
2.7	Aktuelle Verfahren zur Artefakt- und Sibilantenreduktion	9
2.7.1	De-Esser-Plugins und frequenzbasierte Kompression	9
2.7.2	Manuelles Ausschneiden und Bearbeiten	11
2.7.3	Spektrale Techniken mit Tools wie iZotope RX	11
2.8	Ausblick: Problemstellung und Zielsetzung	12
3	Analyse und Konzeption	13
3.1	Anforderungsdefinitionen und Zielkriterien	13
3.2	Techniken zur Sibilantenerkennung und Trennung	14
3.2.1	Sample-basierter Schwellenwert-Ansatz	15
3.2.2	RMS-basierter Schwellenwert-Ansatz	15
3.2.3	Spektrale Subtraktion via Fast-Fourier-Transformation	17
3.3	Auswahl der Methoden und Parameter	18
3.4	Teststrategie und Evaluationskriterien	19
3.4.1	Testdaten	19
3.4.2	Bewertung der Audioqualität	20
3.4.3	Bewertung der Erkennungsgenauigkeit	20
3.4.4	Definierte Testziele (Kriterien 1–4)	21
4	Implementierung	22
4.1	Implementierung mit JUCE und Aufbau der Applikation	22
4.1.1	Audio-Processing-Schritte und JUCE-Klassen	22
4.1.2	Architektur	24
4.2	GUI und Parameter	26

4.3	Sample-based Algorithmus	27
4.4	RMS-based Algorithmus	28
4.5	FFT-based Algorithmus	30
5	Tests und Validierung	33
5.1	Testumgebung	33
5.2	Parameter	34
5.3	Testergebnisse	35
5.3.1	Phase 1: Baseline Tests	35
5.3.2	Phase 2: Test der Randfälle & Akustikgitarre	40
6	Fazit	42
6.1	Bewertung der Algorithmen	42
6.2	Einschätzung der Problemlösung	43
6.3	Ausblick	44
A	Anhang	46
A.1	Testbeobachtungen	46
A.2	Quellcode der Applikation	51
A.2.1	Main.cpp	51
A.2.2	MainComponent.h	52
A.2.3	MainComponent.cpp	55
A.2.4	BufferAudioSource.h	67
A.2.5	BufferAudioSource.cpp	68
A.2.6	AudioProcessorManager.h	70
A.2.7	AudioProcessorManager.cpp	72
A.2.8	SampleControl.h	84
A.2.9	RmsControl.h	86
A.2.10	FftControl.h	88
A.2.11	PositionOverlay.h	91
A.2.12	PositionOverlay.cpp	92
A.2.13	WaveformDisplay	93
A.2.14	WaveformDisplay.cpp	95
	Abkürzungsverzeichnis	99
	Abbildungsverzeichnis	100
	Tabellenverzeichnis	101
	Literaturverzeichnis	102
	Eidesstattliche Versicherung	106

Kurzfassung

In der Musikproduktion können Artefakte wie Sibilanten in Gesangsaufnahmen trotz moderner Aufnahmeverfahren nicht vollständig vermieden werden, da sie schon im Quellsignal auftreten. Das Ziel der vorliegenden Arbeit ist es, ein Verfahren zur präzisen automatisierten Erkennung und Trennung der artefaktbehafteten Signalanteile vom tonalen Restsignal zu entwickeln. Konträr zu aktuell verbreiteten Methoden soll dieser Ansatz die unabhängige Kontrolle über beide getrennten Signalanteile erlauben und so eine höhere Klangqualität ermöglichen.

Zur Lösung dieser Forschungsfrage kommen verschiedene Techniken aus der digitalen Signalverarbeitung zum Einsatz. Im Mittelpunkt davon steht der Vergleich dreier Ansätze - einem Sample-basierten Frequenzschwellenwert-Ansatz, einem RMS-basierten Frequenzschwellenwert-Ansatz und einem FFT-basierten spektralen Ansatz. Für die Implementierung der drei Algorithmen wurde unter Verwendung des JUCE-Frameworks eine Standalone-Applikation in C++ entwickelt. In ihr befindet sich ein modularer Signalfluss, der es erlaubt, die Algorithmen im Signalweg gegeneinander auszutauschen und so ohne äußeren Einfluss gegeneinander testbar zu machen. Die Evaluation der Algorithmen wird anhand von Tests mit professionellen Gesangsaufnahmen durchgeführt und mittels objektiver Kriterien wie z. B. Spektrogramm- und Nulltests sowie subjektiver Hörtests bewertet.

Die Ergebnisse zeigen die Einschränkungen reiner Frequenzschwellenwert-basierter Verfahren und die Vorteile der Betrachtung von Frequenzverhältnissen mittels FFT. Mit dem FFT-basierten Algorithmus und der Erkennung der Sibilanten durch die Analyse der Frequenzverhältnisse kann eine sehr robuste Erkennung und Trennung der Sibilanten erreicht werden, welche sich auch auf Signale wie Saitenquietschen in Akustikgitarrenaufnahmen ausweiten lässt.

Abstract

In music production, artifacts such as sibilants in vocal recordings can degrade the sound quality and disrupt the listening experience. Despite modern recording techniques, these artifacts cannot be completely avoided, as they are already present in the source signal. The goal of this thesis is to develop a procedure that allows for the precise automated detection and filtering of these artifacts from the rest of the tonal signal. Compared to the methods currently used in the market, this approach should allow independent control over both separated signal components and, therefore, allow a higher sound quality.

Various digital signal processing techniques are used in the pursuit of solving this research question. The focus is on the comparison of three algorithmic approaches - a sample-based frequency threshold approach, an RMS-based frequency threshold approach, and an FFT-based spectral approach. These three algorithms were implemented in a standalone application which was developed in C++ using the JUCE framework. It features a modular

signal flow that allows the switching of the algorithms within the signal chain, which enables the ability to test them without external influences. Finally, the algorithms are evaluated based on tests with professional vocal recordings and assessed using objective criteria such as visual tests using a spectrogram and null tests, as well as subjective listening tests.

The results reveal the limitations of purely frequency-threshold-based methods and show the advantages of algorithms using frequency ratios via FFT in the detection process. The FFT-based algorithm, with its detection of sibilants through the analysis of frequency ratios, shows that this approach leads to a very robust detection and separation of sibilants, which can also be extended to signals such as finger noise when sliding over strings in acoustic guitar recordings.

Kapitel 1: Einleitung

Die Produktion von Musikaufnahmen ist ein vielschichtiger Prozess, bei dem viele Faktoren die Klangqualität beeinflussen. Neben vermeidbaren technischen Problemen wie der Qualität des Aufnahmewegs, dem Raumklang oder der Mikrofonposition treten in vielen natürlichen Quellen unerwünschte Störgeräusche auf. Diese Arten von Störgeräuschen sind schon im natürlichen Signal vorhanden und lassen sich durch die technische Optimierung der Aufnahme nicht verhindern [1]. Insbesondere handelt es sich hierbei um Sibilanten (Zischlaute) in Gesangsaufnahmen oder Saitenquietschen bei Gitarrenaufnahmen [2, 3]. Diese können bei der Aufnahme nur sehr schwer vermieden werden, da verschiedene Sänger natürlicherweise Sibilanten unterschiedlich stark betonen oder Gitarristen beim Wechsel bestimmter Akkorde ein Rutschen über die Saiten nicht unterbinden können. Charakteristisch für diese störenden Klanganteile sind im Frequenzspektrum vornehmlich hochfrequente Anteile, welche über dem natürlichen Klang des Restsignals herausstechen und zu einem subjektiv negativen Höreindruck führen [1]. Eine zeitlich undifferenzierte Bearbeitung des Frequenzgangs der Aufnahme, z. B. durch einen Equalizer, behebt diesen Kontrast zwischen tonalem Signal und Störgeräusch nicht [1]. Je präsenter diese Artefakte in Aufnahmen vorhanden sind, desto schwerer wird der nachfolgende Bearbeitungs- und Veredelungsprozess, da deren Präsenz z. B. durch starkes Komprimieren eines Signals noch weiter hervorgehoben wird [3]. Somit müssen alternative Verfahren gefunden werden, um diese Artefakte zu unterdrücken.

1.1 Problemstellung

Um den klanglichen Anspruch einer professionellen Audioproduktion zu erfüllen, werden verschiedene Methoden genutzt, um diese Störgeräusche zu minimieren [1]. Die gängigsten digitalen Verfahren sind hierbei an analogen Ansätzen orientiert und mit mangelnder Kontrolle und Präzision oder viel manuellem Arbeitsaufwand verbunden [1]. So leidet die klassische frequenzbasierte Kompression, wie sie in De-Essern der meisten modernen DAWs (Digital Audio Workstations) zu finden ist, unter mangelnder Präzision und häufig unnatürlichen Klangergebnissen. Das manuelle Ausschneiden der Störgeräusche anhand der Wellenformanzeige (Waveform) ist dagegen mit hohem Arbeitsaufwand verbunden [3]. Auch spezialisierte Software wie iZotope RX, dem Industriestandard für Audioaufbereitung, bietet wenig Vorteile zur Lösung dieser Probleme [4]. Das Problem dieser Ansätze ist in automatisierten Verfahren die fehlende Präzision und Kontrolle und in manuellen Ansätzen,

auch unter Einsatz spezialisierter Tools, der hohe Arbeitsaufwand, welcher benötigt wird, um ein präzises Ergebnis zu erreichen.

1.2 Ziele der Arbeit

Ziel dieser Arbeit ist es, eine automatisierte Methode zu entwickeln, die die Störgeräusche präzise erkennt und diese dann, wie in der manuellen Bearbeitung, vom restlichen Material trennt. Hierzu können die Vorteile der digitalen Signalverarbeitung genutzt werden. Digitale Filterung, Sample-genaue Bearbeitung und Methoden wie die Fourier-Transformation erlauben eine detaillierte Untersuchung und Bearbeitung von Frequenz- und Amplitudenverläufen. Durch die automatisierte Erkennung, Trennung und gezielte Bearbeitung des störenden Anteils (z. B. durch die Bearbeitung des Frequenzgangs oder der Amplitude) soll eine hohe Kontrolle über das Endergebnis ermöglicht werden – ohne Beeinträchtigung des restlichen Signals. Hierdurch soll eine höhere Klangqualität sowie ein effizienterer und flexiblerer Bearbeitungsprozess erzielt werden. Um die beste Lösung dafür zu finden, sollen aus gängigen Komponenten des *DSP* (Digital Signal Processing) verschiedene Erkennungs- und Filtermethoden implementiert und mit dem Fokus auf Präzision und Klangästhetik miteinander verglichen werden. Insbesondere werden hier frequenzschwellenbasierte Ansätze mit spektralen Verfahren gegeneinander getestet, um die jeweiligen Vor- und Nachteile für die Eignung in professionellen Produktionen zu erschließen. Hieraus werden Schlüsse für die Verbesserung der bestehenden Praxis im Umgang mit Störgeräuschen in Audioaufnahmen gezogen.

Da Sibilanten das prominenteste Anwendungsgebiet derartiger Störgeräusche sind und die meisten industrieüblichen Verfahren hierfür speziell entwickelt werden, bietet die Beschränkung auf Sibilanten einen guten Fokus der Arbeit. Sibilanten sind in so gut wie jeder modernen Pop-Produktion ein störendes Problem. Außerdem eignen sie sich ideal als Untersuchungsobjekt, um verschiedene Erkennungs- und Filterungsverfahren zu testen, da sie meist klar herausstechen und definierbar sind. Ähnlich wie andere für Sibilanten entwickelte Verfahren können die in dieser Arbeit entwickelten Algorithmen auch für andere Audioquellen im Nachgang getestet und verwendet werden [3].

In der folgenden Arbeit werden zunächst in Kapitel 2 die theoretischen Grundlagen der Programmierung mit digitalem Audio und der Signalverarbeitung vorgestellt, die Sibilanten als Untersuchungsobjekt charakterisiert und aktuelle Verfahren miteinander verglichen. Abschließend wird die genaue Problemstellung und Zielsetzung definiert. Aufbauend auf diesen Grundlagen werden im darauffolgenden Kapitel 3 die Konzepte für die Implementierung und eine Teststrategie mit relevanten Evaluationskriterien entwickelt. Darauf folgend werden, wie in Kapitel 4 beschrieben, die entwickelten Algorithmen mithilfe des JUCE-Frameworks in einer Beispielapplikation implementiert und sowohl die Algorithmen als auch die Applikationsarchitektur erklärt. Zuletzt werden in Kapitel 5 Tests mit professionellen Audiobeispielen durchgeführt und ausgewertet, um anschließend in Kapitel 6 die Arbeitsweise der Algorithmen abschließend zu bewerten.

Kapitel 2: Grundlagen

Im folgenden Kapitel werden die Grundlagen zum Verständnis der technischen Ansätze und Problemlösungen der Arbeit vorgestellt. Vorangehend werden wichtige Grundlagen von digitalem Audio beschrieben, der Artefaktbegriff für den Rahmen dieser Arbeit definiert und Grundlagenbegriffe der Musikproduktion für das Verständnis der Applikationskomponenten eingeführt. Darauf folgend werden aktuelle Verfahren zur Artefakt- und Sibilantenreduktion vorgestellt und daraus die Problemstellung und Zielsetzung der Arbeit entwickelt.

2.1 Grundlagen der digitalen Signalverarbeitung

Das DSP ist der zentrale Grundstein moderner Audioproduktionen [5]. Es wird durch die Wandlung analoger Audiosignale in die digitale Domäne ermöglicht. Durch die Abtastung und Quantisierung kontinuierlicher analoger Signale werden diskrete Amplitudenwerte abgespeichert, mit denen nun komplexe Berechnungen und Algorithmen möglich sind [6]. Diese diskreten Werte sind von den zwei Faktoren *Sample Rate* (Abtastrate) und Bittiefe der gespeicherten Werte abhängig [5, Kapitel 1]. Die nötige Abtastrate für die Wiedergabe des menschlichen Hörspektrums von 20 Hz bis 20 kHz hängt hierbei vom Nyquist-Theorem ab, nach dem die Abtastrate mindestens doppelt so hoch wie die höchste wiederzugebende Frequenz sein muss [7].

Diese Signale enthalten, wie in der analogen Domäne auch, die komplette Bandbreite an Frequenzen. Zur Darstellung und visuellen Analyse dieser komplexen Signale kann die *Waveform* (Wellenform) herangezogen werden, die das Signal als Amplitude über die Zeit darstellt. Die meisten DAWs nutzen diese Ansicht zur Darstellung ihrer Spuren. Eine weitere gängige Darstellungsmethode ist das Spektrogramm, das Frequenzen über die Zeit abbildet und die Amplitude der enthaltenen Frequenzen üblicherweise durch Färbung und Helligkeit visualisiert.

Besonders das Verständnis der Samplewerte und der darin enthaltenen Signalinformationen sowie die gängigen Visualisierungsmethoden sind für das Verständnis der Algorithmen und der Darstellung der Ergebnisse von besonderer Wichtigkeit.

2.2 JUCE und die Programmierung mit digitalem Audio

JUCE ist ein plattformübergreifendes C++-Framework für die Entwicklung von Desktop- und mobilen Applikationen. Es wurde speziell für die Programmierung von Audioanwendungen entwickelt und stellt alle nötigen Basisfunktionen für die Erstellung von plattformübergreifender Audiosoftware in Form von Standalone-Applikationen oder Audio-Plugins zur Verfügung [8, 9]. Für den vorliegenden Anwendungsfall sind diesbezüglich folgende Kernkomponenten relevant:

- GUI-Erstellung mit Klassen und Funktionen für die flexible Gestaltung von Layouts mit Buttons oder Komponenten wie einer Waveformvisualisierung durch **AudioThumbnail**.
- Datei- und OS-Handling durch Klassen wie den **AudioFormatReader**, welcher das Einlesen und Schreiben von Audiodateien (WAV, MP3, etc.) ermöglicht.
- DSP (Signalverarbeitung) durch Klassen wie **AudioBuffer**, **LinkwitzRileyFilter** und **FFT** (Fast-Fourier-Transformation).

Da JUCE sehr weit verbreitet ist und auch von vielen großen und bekannten Audiosoftware-Herstellern genutzt wird, bieten die existierenden Ressourcen und Anwendungsbeispiele eine hervorragende Grundlage für die Entwicklung einer eigenen Applikation, in der sich auch die Praxistauglichkeit der Algorithmen sehr gut testen und vergleichen lässt. Aus diesen Gründen wird JUCE als Basis dieser Arbeit und zur Implementierung der Applikation einschließlich der Algorithmen verwendet.

2.3 Echtzeit- versus Offline-Verarbeitung

Für die Entwicklung einer Audioapplikation sind die zwei grundsätzlichen Ansätze der Echtzeit- und Offline-Verarbeitung für die Arbeit mit Audiodaten zu betrachten.

Audio-Plugins sind Programme, die in den Signalweg einzelner Spuren oder Gruppen zur Bearbeitung des Signals in einer DAW genutzt werden. Sie bekommen abhängig von der Position in der Signalkette Audiodaten von der DAW geliefert, verarbeiten diese und geben sie wieder an die DAW zur Weiterverarbeitung und Summierung zurück. In Audio-Plugins werden Audiodaten üblicherweise in Echtzeit verarbeitet [10]. Dies bedeutet, dass immer nur Samples in Größe der in der Host-Anwendung (hier DAW) eingestellten *Buffer Size* (Puffer-Größe) dem Plugin zur Verfügung stehen. DSP-Berechnungen können also immer nur auf die Samples in Größe der Buffer Size an der aktuellen Abspielposition erfolgen. Dies erlaubt das variable Abspielen von bearbeitetem Audio, ohne dass die Quelldatei verändert werden muss, und bietet die flexible Grundlage der Arbeitsweise moderner DAWs. Für die Programmierung bedeutet dies Folgendes:

Die Samples des ausgelesenen Audiofiles werden in einen *Buffer* (Pufferspeicher) geladen, bis die Buffer Size erreicht ist, und nach dem FIFO-Prinzip Sample für Sample weiter befüllt, während die ältesten Samples wieder aus dem Buffer entfernt werden. Mit den aktuell im Buffer liegenden Samples kann dann gerechnet werden. Dieses Grundprinzip kann zwar in einem gewissen Rahmen durch *Circular Buffers* für Reverb- oder Delay-Plugins erweitert werden, dennoch kann es nicht komplett umgangen werden [5, S. 384–386]. Dies verhindert zum Beispiel die Zugriffsmöglichkeit auf eine andere Stelle im Signal, sollte im aktuellen Buffer ein besonderer Fall eintreten. Auch Berechnungen, die sich auf das ganze Signal im Verhältnis beziehen, werden dadurch unterbunden.

Um dieses Problem zu lösen, steht dem die Offline-Verarbeitung gegenüber. Bei offlinefähigen Prozessen kann das gesamte Audiosignal verarbeitet werden, ohne an die Buffer Size oder Abspielposition gebunden zu sein. Dies ermöglicht unter anderem das mehrmalige Durchlaufen des kompletten Signals und die unabhängige Bearbeitung einzelner Stellen im Verhältnis zum Gesamtsignal. Der Nachteil einer Offline-Bearbeitung ist allerdings, dass das Signal gänzlich verarbeitet und zwischengespeichert werden muss, um das Ergebnis hör- und abspielbar zu machen. Neben Standalone-Applikationen bietet der ARA (Audio Random Access) Standard diese Offline-Bearbeitungsmöglichkeiten seit einigen Jahren auch für die Verwendung in DAWs an. Hier erlaubt die ARA-Schnittstelle dem Plugin, auf das sich auf der Spur befindende Audiofile zuzugreifen und es Plugin-intern gänzlich zu verarbeiten [11].

Im Rahmen der Arbeit wird nur die Offline-Verarbeitung in einer Standalone-Applikation verwendet.

2.4 Fourier-Transformation

Durch die Fourier-Transformation lassen sich komplexe periodische Funktionen, wie ein Abschnitt eines Audiosignals, in eine Summe von Sinus- und Kosinusfunktionen zerlegen. Die ursprüngliche Funktion wird somit als eine Überlagerung von harmonischen Schwingungen beschrieben. Dieser Ansatz ermöglicht die Analyse und Synthese komplexer Audiosignale durch einfache periodische Funktionen [5, S. 564]. Der große Vorteil von digitalem Audio für die Fourier-Transformation ist die Speicherung des Signals als diskrete Werte, wodurch die Berechnung der Signalanteile deutlich vereinfacht wird. Anstatt kontinuierliche Integrale zu verwenden, arbeitet die *DFT* (Diskrete-Fourier-Transformation) mit Summen. Dadurch wird die Umsetzung und Berechnung in der digitalen Domäne stark erleichtert und ermöglicht eine effiziente Analyse der im Signal vorhandenen Frequenzinformationen [5, S. 566]. Im Folgenden ist die Summenformel der DFT gezeigt [5, S. 567]:

$$X[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1. \quad (2.1)$$

Die *FFT* (Fast-Fourier-Transformation) ist eine optimierte Version der DFT. Im Gegensatz zur DFT basiert die FFT auf einer rekursiven Zerlegung der Werte, wodurch die Effizienz von $O(n^2)$ auf $O(n \log n)$ deutlich verbessert wird [5, Kapitel 20]. Für ein N -Punkt-Signal $x[n]$ wird die DFT in gerade und ungerade Indizes aufgeteilt und die DFT für die kleineren Teilfolgen berechnet [12]:

$$E[k] = \sum_{n=0}^{\frac{N}{2}-1} x[2n] e^{-i \frac{2\pi}{N/2} kn}, \quad (2.2)$$

$$O[k] = \sum_{n=0}^{\frac{N}{2}-1} x[2n+1] e^{-i \frac{2\pi}{N/2} kn}. \quad (2.3)$$

Daraus ergibt sich unter Hinzuziehen des Faktors W_N^k , wobei

$$W_N = e^{-i \frac{2\pi}{N}}, \quad (2.4)$$

die primäre N te Einheitswurzel darstellt, für $k = 0, 1, \dots, \frac{N}{2} - 1$ wieder die DFT [12]:

$$X[k] = E[k] + W_N^k O[k], \quad (2.5)$$

$$X\left[k + \frac{N}{2}\right] = E[k] - W_N^k O[k]. \quad (2.6)$$

Diese Variante der Fourier-Transformation ist die gängigste für die Programmierung von Audio-Applikationen und wird demnach auch in dieser Arbeit verwendet [5, Kapitel 20]. Sie findet unter anderem auch Verwendung in Bereichen wie Audiostretching oder FFT-basierter spektraler Kompression [13, 14].

Für den vorliegenden Anwendungsfall bedeutet die Fourier-Transformation einen möglichen Zugriff auf detaillierte Frequenzinformationen und deren Bearbeitung in einem vorliegenden Audiosignal, was spektrale Bearbeitungen ermöglicht, die über die klassische Filterung hinausgehen.

2.5 Was sind Artefakte in Musikaufnahmen?

Bei der Aufnahme und Digitalisierung von Audiosignalen gibt es etliche Komponenten, die die Klangqualität beeinträchtigen können. Hierzu zählt alles von Rauschen der Komponenten im Signalweg und Quantisierungsartefakten der Analog-Digital-Wandlung bis hin zu ungewollten Raumresonanzen und Quellenartefakten wie Saitenquietschen oder Sibilanten [1, 15, 2].

Durch den Fortschritt der letzten Jahrzehnte sind technische Probleme wie Rauschen und Analog-Digital-Wandlung in Studiosettings keine relevanten Probleme mehr [15]. Auch ungewollte Raumresonanzen lassen sich durch die Optimierung der Aufnahmeumgebung und Mikrofonposition sehr gut verhindern. Artefakte, die heute allerdings immer noch bestehen bleiben und sich nicht durch Optimierung des Aufnahmewegs vermeiden lassen, sind Sibilanten oder Saitenquietschen als natürliche Nebenprodukte der Quelle [2, 1].

Wie in Abbildung 2.1 zu sehen ist, sind diese Nebenprodukte meist hochfrequente, transiente und nicht-tonale Anteile des Signals. Sie sind oft von kurzer Dauer und weisen eine hohe Amplitude im Vergleich zum restlichen Signal in diesem Frequenzbereich auf. Wie schon in der Einleitung beschrieben, fokussiert sich diese Arbeit dazu spezifisch auf Sibilanten, da diese besonders klar kategorisierbar und definierbar sind.

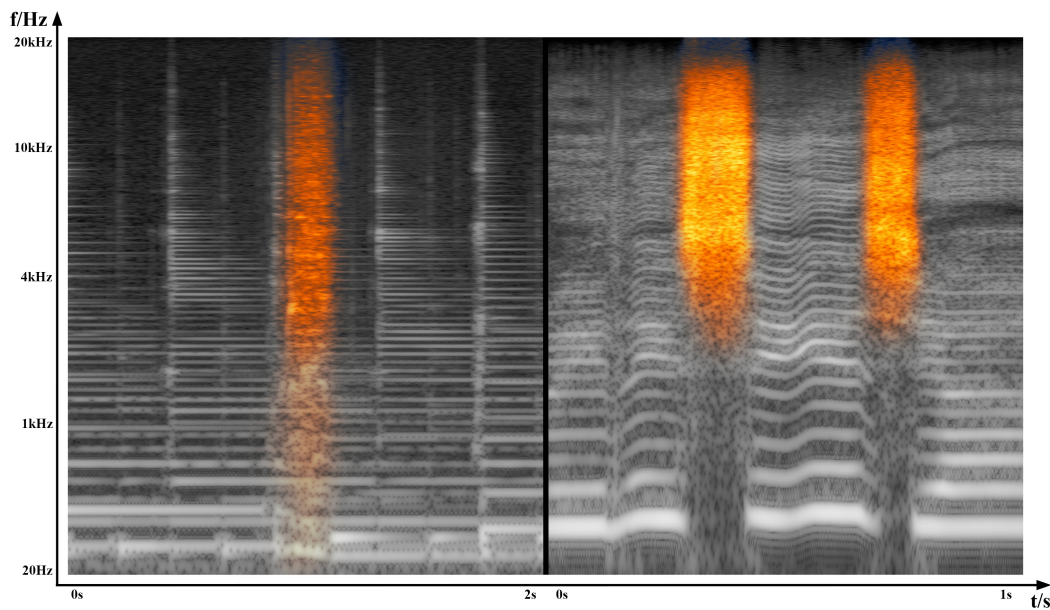


Abbildung 2.1: Spektrogramm in iZotope RX - links rot markiert Saitenquietschen, rechts rot markiert Sibilanten. Quelle: Eigene Auswertung.

2.5.1 Charakteristik von Sibilanten

Sibilanten bzw. Zischlaute sind beim Sprechen oder Singen entstehende Laute aus der Familie der Frikative [16]. Sie unterscheiden sich von anderen Konsonanten durch ihren charakteristisch besonders hochfrequenten, rauschähnlichen Anteil häufig zwischen 4 kHz und 10 kHz [3]. Wie in der Abbildung 2.2 gut zu erkennen ist, unterscheiden sie sich demnach sehr von tonalem Material, welches aus einem Grundton und verschiedenen Obertonresonanzen besteht. Diese Obertonresonanzen sind im Spektrogramm der Abbildung 2.2 an den gebündelten Frequenzamplituden gut zu erkennen, wobei im Sibilanten keine differenzierten Frequenzbetonungen zu erkennen sind. Durch diesen hochfrequenten rauschähnlichen

Pegelausschlag können Sibilanten im Höreindruck häufig als scharf oder unangenehm wahrgenommen werden [3].

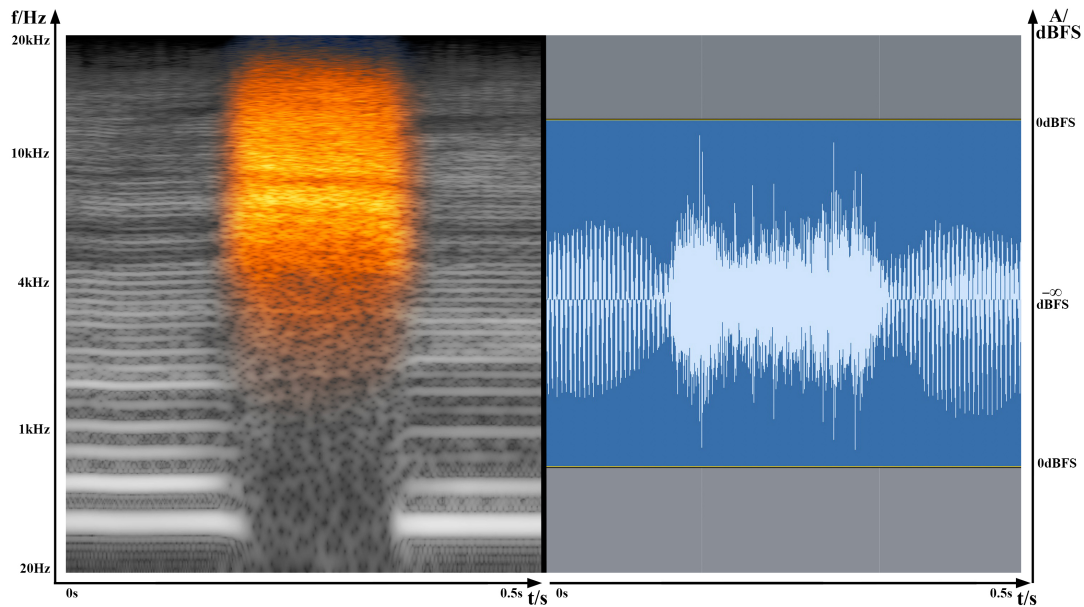


Abbildung 2.2: Links Spektrogramm mit Sibilant in rot, rechts Waveform des selben Sibilanten hochfrequent in der Mitte. Quelle: Eigene Auswertung.

Die typischsten Sibilanten sind Laute wie *s*, *sch* oder *z* und bilden für diese Arbeit die *Level-1-Sibilanten-Kategorie*. Laute wie *t*, *k* oder *f* sind zwar im klassischen Sinne keine Sibilanten, bilden aber hier die *Level-2-Kategorie*, da sie einen sehr sibilant-ähnlichen Frequenzgang besitzen. Sie sind in Gesangsaufnahmen meist nicht so problematisch wie Level-1-Sibilanten, dienen aber durchaus als gutes Kriterium, um die Präzision der Erkennung eines Algorithmus einzuschätzen, und sollten in einem besonders präzisen Algorithmus ebenfalls erkannt werden.

Für die spätere Arbeit besonders relevant ist hier der hochfrequente Pegelausschlag der Sibilanten und die daraus resultierende Frequenzverteilung im Gegensatz zum tonalen Signalanteil.

2.6 Grundlagenbegriffe der Audibearbeitung

Einige Begriffe aus den Grundlagen der Arbeit mit Audiosignalen sind für das Verständnis der Zielsetzung, Algorithmen und Auswertung dieser Arbeit essenziell. Dynamik und Frequenzgang sind die Grundbausteine im Umgang mit Audiosignalen und Veredelung. Für die Kontrolle dieser Parameter sind in der Audioproduktion die gängigsten Komponenten der Kompressor und der Equalizer.

Ein Kompressor reagiert auf die aktuelle Signalstärke über einen *Threshold* (Amplitudenschwelle). Übersteigt das Signal diesen Threshold, wird die Amplitude dieses Signalanteils in einem Verhältnis (typisch 2:1 bis 8:1) abgesenkt und somit der Dynamikbereich *komprimiert*.

Ein *EQ* (Equalizer) erlaubt dagegen, den Frequenzgang eines Signals zu verändern. Mittels meist mehrerer Filterbänder kann das Signal an verschiedenen Frequenzstellen statisch angehoben oder abgesenkt werden. Die wichtigsten Parameter sind hierbei die Breite des beeinflussten Frequenzbands und die Stärke der Anhebung bzw. Absenkung.

Auch unter die Kategorie der Equalizer fallen Filter wie Hoch- und Tiefpassfilter. Diese senken das bearbeitete Signal ab einer definierten Grenzfrequenz in einem Verhältnis (z. B. 12 dB pro Oktave) kontinuierlich bis auf Null ab. Dies wird häufig zur Eliminierung unerwünschter Frequenzanteile verwendet oder zur Frequenztrennung von Signalen in komplexeren Audioprozessoren [17, 18]. Wichtig ist hierbei zu beachten, dass es verschiedene Ansätze für den Aufbau von Filtern gibt, die die Phase des bearbeiteten Signals unterschiedlich beeinflussen. Als Nebeneffekt drehen viele Filter die Phase des Signals um die Grenzfrequenz, was zu Phasenauslöschungen führen kann.

Als Phasenauslöschungen bzw. *Comb-Filter-Effekte* (Kammfiltereffekte) bezeichnet man das Phänomen, wenn ein gleiches oder ähnliches Audiosignal kombiniert wird und sich die Signalstärke des Gesamtsignals bzw. verschiedener Frequenzbereiche des Signals durch die Kombination von gegensätzlichen Positiv-Negativ-Schwingungen auslöscht [19]. Dies führt zu Pegelveränderungen und hörbaren Artefakten. Wenn zwei gleiche Signale, bei denen, wie in Abbildung 2.3 zu erkennen ist, die Phase eines der Signale um 180 Grad gedreht wurde, rekombiniert werden, summieren sich die Signale auf Null. Dies nennt man auch den Nulltest, welcher häufig dazu verwendet wird, Signalveränderungen durch Audibearbeitungen zu erkennen [20].

2.7 Aktuelle Verfahren zur Artefakt- und Sibilantenreduktion

In der Audioproduktion gibt es zahlreiche etablierte Verfahren zur Reduktion von Sibilanten. Diese Methoden variieren in ihrer Komplexität und Effektivität und lassen sich grob in drei Kategorien einteilen: frequenzbasierte Kompression, manuelles Bearbeiten und spektrale Bearbeitung durch spezialisierte Bearbeitungsprogramme.

2.7.1 De-Esser-Plugins und frequenzbasierte Kompression

De-Esser-Plugins gehören zu den am häufigsten verwendeten Werkzeugen zur Reduktion von Sibilanten. Sie arbeiten auf der Basis frequenzabhängiger Kompression, bei der spezifische hochfrequente Bereiche gezielt abgesenkt werden, sobald diese einen bestimmten

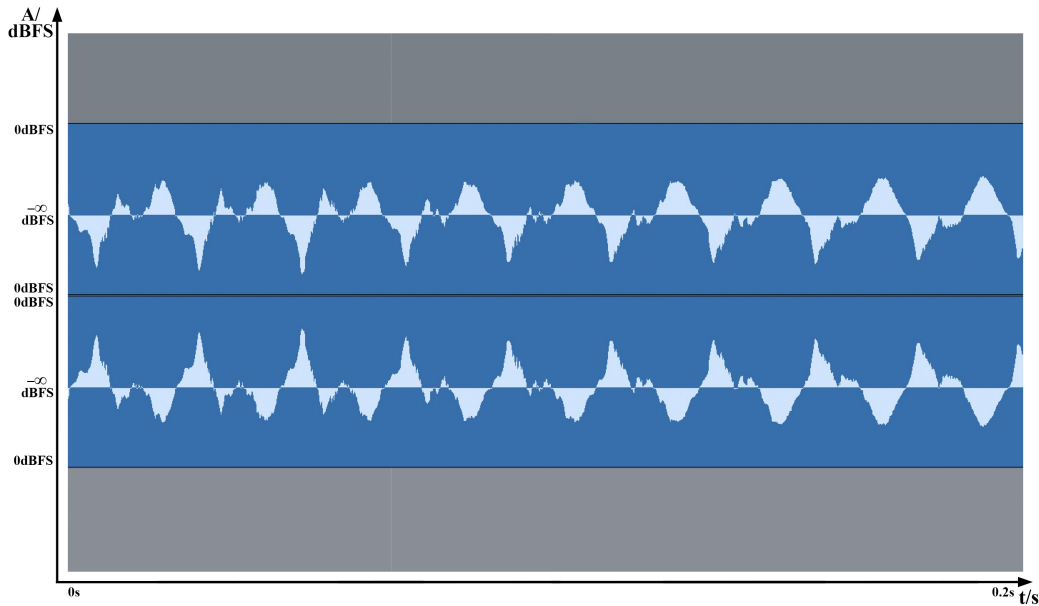


Abbildung 2.3: Waveform Abbildung von einem regulären und einem phasengedrehten Signal.
Quelle: Eigene Auswertung.

Threshold überschreiten [3]. Dieser Threshold betrachtet statt des gesamten Signals nur ein hochpassgefiltertes Triggersignal, welches auf die hochfrequenten Sibilanten anspringt. Anschließend wird in den meisten De-Essern wahlweise das gesamte Signal oder nur der hochfrequente Anteil komprimiert, um die Amplitude der Sibilanten abzusenken und den Höreindruck dadurch zu verbessern. Zu dieser Kategorie gehören die meisten kommerziellen Plugins wie der „FabFilter Pro-DS“ und der „Waves DeEsser“, wobei simplere Versionen dieser Plugins schon in den meisten DAWs als Standard mitgeliefert werden [21, 22]. Auch wenn diese Methode relativ unkompliziert und effizient ist, hat sie folgende Einschränkungen:

- Es werden häufig nicht nur die Sibilanten komprimiert, sondern auch Teile des restlichen Signals, was den Frequenzgang beeinflussen kann.
- Die Komprimierung verändert die Hüllkurve (Amplitudenverlauf) und häufig auch den Frequenzgang der Sibilanten, was schnell dazu führt, dass Sänger klingen, als würden sie lispeln.
- Eine vollständige Trennung der Sibilanten vom tonalen Signal ist mit dieser Methode nicht möglich.

Zusätzlich zu dieser einfachen Art des De-Essers gibt es ebenfalls Plugins, die einen ähnlichen Ansatz verfolgen, aber ihre Erkennung über neuronale Netze lösen. Hierzu gehört vor allem der „smart:deess“ von Sonible [23]. Die Sibilanten werden hier, statt über einen einfachen Frequenz-Threshold, per Machine Learning erkannt und dann abgesenkt. Dabei gibt es auch die Möglichkeit, die erkannten Laute an ein ideales Frequenzprofil anzupassen. Ein als

„S“ erkannter Sibilant wird somit an den idealen Frequenzgang dieses gelernten Lautes angepasst. Dies löst die meisten der Klangprobleme der klassischen De-Esser-Plugins, erlaubt allerdings trotzdem keine komplette Trennung von sibilantem und tonalem Signalanteil [23].

2.7.2 Manuelles Ausschneiden und Bearbeiten

Eine weitere Methode ist das manuelle Bearbeiten, bei dem anhand der Waveform der Spur in der DAW die Sibilanten erkannt und ausgeschnitten werden. Durch diese Trennung können beide Signalanteile unabhängig bearbeitet werden. Die Sibilanten können in ihrer Amplitude abgesenkt oder auf einer separaten Spur durch Kompression und Equalizing in Dynamik und Frequenzgang angepasst werden [3]. Die Vorteile liegen hier in der Präzision der Bearbeitung. Allerdings gibt es auch erhebliche Nachteile:

- Der Bearbeitungsprozess ist sehr zeit- und arbeitsaufwändig, besonders bei langen oder komplexen Aufnahmen.
- Wenn nicht sorgfältig gearbeitet wird, kann das manuelle Ausschneiden zu unnatürlichen Klangübergängen führen.
- Es wird immer das gesamte Frequenzspektrum ausgeschnitten und nicht nur der Bereich der Sibilanten [3].

2.7.3 Spektrale Techniken mit Tools wie iZotope RX

Tools wie iZotope RX bieten besondere Flexibilität und spezialisierte Funktionen für die spektrale Bearbeitung [4]. iZotope RX ist der Industriestandard für Audionachbearbeitung mit besonderem Schwerpunkt auf Audioreparatur und -restauration. Es stellt umfassende Funktionen und Werkzeuge zur Entfernung von Rauschen, Klicks, Knacken und anderen Störgeräuschen zur Verfügung [24]. Mit den enthaltenen Funktionalitäten gibt es zur Bearbeitung von Sibilanten zum einen die Möglichkeit, mit den manuellen Tools über die Gesangsspur zu gehen, die störenden Sibilanten anhand des Spektrogramms zu identifizieren und an den erkannten Stellen die Sibilanten-Frequenzbereiche eigenhändig im Pegel zu reduzieren. Dieser Prozess ist noch präziser als das zuvor beschriebene manuelle Ausschneiden. Des Weiteren gibt es einen eingebauten De-Esser, der spektral arbeiten kann und somit ein deutlich besseres Klangergebnis als klassische De-Esser liefert. Anstatt nur über einen Frequenz-Threshold kann hier das Spektrum des Signals analysiert und die Sibilanten anhand des spektralen Profils automatisch erkannt werden. Eine wichtige Anmerkung ist, dass iZotope RX hauptsächlich als Offline-Standalone-Applikation für den vollen Funktionsumfang genutzt wird und nur eingeschränkte Versionen der Tools als Echtzeit-Plugins zur Verfügung stehen.

Sowohl der manuelle Ansatz als auch der De-Esser in iZotope RX sind jedoch nicht ohne Einschränkungen:

- Die Sibilanten können nur abgesenkt und nicht vom tonalen Signal getrennt frei bearbeitet werden.
- Die automatische Erkennung des De-Essers arbeitet nicht immer zuverlässig und in der manuellen Bearbeitung sind die fließenden Übergangsbereiche schwer klar abzugrenzen.
- Der manuelle Prozess mit iZotope RX ist noch arbeitsaufwändiger als das reguläre manuelle Ausschneiden.

2.8 Ausblick: Problemstellung und Zielsetzung

Nach Sichtung aktueller Methoden ergibt sich eine Lücke für eine automatisierte Methode zur Trennung von Sibilanten und Restsignal. Zur reinen Klangverbesserung sind die existierenden Methoden zwar ausreichend, allerdings auch durch unsaubere Erkennung von Sibilanten und fälschlich bearbeitetem tonalen Material klanglich nicht ideal. Um die höchste Qualität und Kontrolle zu erreichen, sind zeitaufwändige manuelle Eingriffe nötig. Der Fokus dieser Arbeit liegt demnach auf der Entwicklung eines Verfahrens, welches die vollständige Isolierung der Sibilanten ermöglicht und somit, wie beim manuellen Ausschneiden, die volle Kontrolle über die Bearbeitung des Sibilanten-Anteils liefert. Ein solcher Ansatz bietet die folgenden Vorteile:

- Die unabhängige Bearbeitung der Teilsignale ermöglicht mehr Kontrolle über das Klangergebnis. Beispielsweise können im tonalen Signal die hohen Frequenzen angehoben werden, ohne die Sibilanten dabei mit zu verstärken. Auch können die Signale voneinander getrennt komprimiert werden, welches Vorteile für die klangliche Kontrolle und das Ergebnis hat.
- Durch die Automatisierung mit einem Algorithmus wird der manuelle Bearbeitungsaufwand für den Nutzer stark reduziert. Somit bietet dieser Ansatz eine praktikable Alternative und Verbesserung zu klassischen De-Essern und ähnlichen automatisierten Verfahren.
- Durch die Trennung des Sibilanten-Signalanteils wird das tonale Restsignal durch die Bearbeitungen nicht beeinträchtigt.

Die wesentlichen Grundlagen zur Ausarbeitung der Problemlösung sind nun gelegt, um im nächsten Kapitel auf die Konzeption der Algorithmen einzugehen.

Kapitel 3: Analyse und Konzeption

Um für die Problemstellung eine Lösung zu finden, werden im folgenden Kapitel Anforderungen an die Algorithmen definiert, daraus Ansätze zur Sibilantenerkennung und Filterung entwickelt und zuletzt eine Teststrategie mit Evaluationskriterien aufgestellt. Die konzeptionierten Erkennungs- und Filterungsansätze umfassen zwei Frequenzschwellenwert-basierte Ansätze und einen spektralen Ansatz mittels FFT.

3.1 Anforderungsdefinitionen und Zielkriterien

Es ergeben sich aus den genannten Grundlagen und Zielen zwei zentrale Anforderungen für die Arbeit:

1. **Die Erkennung von Sibilanten:** Das Ziel ist es diese hochfrequenten Artefakte in Gesangsaufnahmen so präzise wie möglich zu identifizieren. Dabei darf kein oder nur ein minimaler Anteil des restlichen tonalen Signals fälschlicherweise als Sibilant eingestuft werden.
2. **Die Trennung von Sibilanten:** Im Anschluss an die Erkennung sollen die Sibilanten vom verbleibenden Signal getrennt werden. Das Ziel hierbei ist, durch die Möglichkeit, beide Signalanteile getrennt voneinander bearbeiten zu können, die meiste Kontrolle und das beste klangliche Ergebnis zu erreichen (z. B. Sibilanten in ihrer Amplitude zu senken und mit einem EQ zu bearbeiten, ohne das übrige Material zu beeinflussen). Wird keine Bearbeitung der Signalanteile vorgenommen, soll das Signal wieder so rekombinierbar sein, ohne dass es sich vom Ausgangssignal unterscheidet (d. h. ohne erkennbare Klangverluste oder Artefakte).

Aus diesen Anforderungen werden nun in dieser Arbeit drei Algorithmen entwickelt und in einer Beispielanwendung implementiert. Um dies zielführend umzusetzen, werden technische Anforderungen aufgestellt und Konzepte für die Umsetzung entwickelt. In der späteren Implementierung werden diese Konzepte dann unter Beachtung der technischen Anforderungen in einer eigens dafür entwickelten Applikation umgesetzt. Um das Gelingen der Umsetzung der Anforderungen beurteilen zu können, wird eine Teststrategie und damit einhergehende Evaluationskriterien definiert, womit die Algorithmen anhand von praxistauglichen Beispielen final getestet werden.

Die technischen Anforderungen an die Beispielapplikation lauten wie folgt:

- **Teilung des Signals in tonale und sibilante Komponenten:** Das ursprüngliche Audiosignal soll nach der Bearbeitung in zwei separaten Buffern bzw. Audiofiles vorliegen - jeweils ein File bzw. Buffer für den tonalen und den sibilanten Anteil. Die getrennten Signale sollen so unabhängig voneinander weiterverarbeitet, rekombiniert und analysiert werden können.
- **Modularität der Algorithmen im Signalweg:** Um eine Vergleichbarkeit der Algorithmen zu gewährleisten, soll der Signalweg für das Einlesen, Verarbeiten, Speichern und Ausgeben der Daten in jedem Algorithmus gleich sein. Um dies sicherzustellen, wird nur der Algorithmus selbst ausgetauscht und der restliche Signalweg bleibt gleich. Jeder Algorithmus bekommt die gleichen Daten und schreibt an die gleiche Stelle als Ausgabe.
- **Offline-Processing:** Da das Finden der idealen Lösung zur automatisierten Erkennung und Filterung im Vordergrund steht, wird ein Offline-Processing-Ansatz in einer Standalone-Applikation gewählt, um mehr Möglichkeiten in der Berechnung zu bieten. Die Restriktionen einer Echtzeitanwendung entfallen für diese Implementierung und erlauben deshalb die Effizienz für ein besseres Ergebnis in den Hintergrund zu stellen. Eine mögliche Echtzeit-Integration steht nicht im Fokus der Arbeit.
- **Vermeidung von Processing-Artefakten:** Die Filterung darf keine hörbaren Klicks, Pops oder Verfärbungen im Signal erzeugen. Besonders im Hinblick auf die Klangqualität ist essenziell, dass nach der Verarbeitung keine störenden Nebenprodukte zu hören sind. Das bedeutet insbesondere:
 - Keine Knackgeräusche an Übergängen zwischen Sibilant und restlichem Signal.
 - Minimale Phasendrehungen oder Comb-Filter-Effekte durch die Filterung, insbesondere durch die Rekombination von zwei in Frequenzbereiche geteilten Signalen.
 - Idealerweise Sample-genaue Rekonstruierbarkeit des Originalsignals, um Qualitätsverluste zu vermeiden.

3.2 Techniken zur Sibilantenerkennung und Trennung

Aus diesen Anforderungen sind im Rahmen dieser Arbeit drei Methoden zur Sibilantenerkennung und Filterung entwickelt worden. Diese Ansätze wurden unter Analyse gängiger Methoden des DSP entwickelt und nutzen verschiedene Erkennungs- und Filterungsmethoden zur Umsetzung. Die drei resultierenden Verfahren basieren auf verschiedenen Vorgehensweisen mit steigender Komplexität. Es handelt sich um zwei schwellenwertbasierte Ansätze und einen spektralen Ansatz.

Die Gemeinsamkeit dieser drei entwickelten Ansätze ist, dass sie hochfrequente Signalanteile (typischerweise oberhalb von ca. 4–5 kHz) genauer analysieren und über Schwellenwerte oder

Frequenzverteilung entscheiden, ob ein Sibilant aktiv ist oder nicht. Die Komplexität hierbei reicht von der simpelst möglichen Betrachtung einzelner Samples, über die Betrachtung von Sample-Gruppen mittels *RMS* (Root Mean Square) bis hin zu spektraler Analyse durch FFT.

3.2.1 Sample-basierter Schwellenwert-Ansatz

Der erste Algorithmus dient unter anderem dazu, das grundsätzliche Prinzip der Erkennung und Filterung an einem möglichst simplen Ansatz zu testen und zu veranschaulichen. Da der Ausschlag der störenden Sibilanten in den hohen Frequenzen größer ist als der des tonalen Signals, kommt hier ein ähnlicher Ansatz wie in klassischen De-Essern zur Erkennung der Sibilanten zum Einsatz. Es wird ein vom Originalsignal unabhängiges Triggersignal generiert, hochpassgefiltert und dann ein Threshold (Schwellenwert) angewendet. Jedes Sample im Triggersignal, dessen Amplitude über dem Threshold liegt, wird demnach als Sibilant eingestuft. Um nicht nur einzelne Samples zu isolieren, wird eine Gruppe nachfolgender Samples (z. B. 100–200 Samples) ebenfalls als Sibilant markiert. Um nun eine Signaltrennung zu erreichen, werden die als Sibilant erkannten Samples getrennt vom Originalsignal abgespeichert und aus dem Ursprungssignal entfernt.

Auch wenn die Betrachtung einzelner Samples durch den Einschluss von Folgesamples sowohl simpel als auch vielversprechend klingt, führt ein solcher Ansatz zu keiner robusten Aussage über die wahre Signalstärke. Beispielsweise kann der kurzfristige Ausschlag einer bestimmten Frequenz die Bedingung auch dann auslösen, wenn das Signal im restlichen relevanten Bereich nicht stark ausgeprägt ist. Auch die Position des korrekten Startsamples des Sibilanten kann nicht richtig gefunden werden, da nur der Punkt des höchsten Ausschlags beachtet wird und die Durchgänge durch Null gänzlich missachtet werden.

3.2.2 RMS-basierter Schwellenwert-Ansatz

Um eine robustere Erkennung zu erreichen, soll der zweite Algorithmus mit einem größeren Betrachtungsfenster statt nur mit einzelnen Samples arbeiten. Hierfür wird aus einer Gruppe an Samples der RMS-Wert berechnet. RMS-Werte sind in der Audioproduktion zur Messung des Effektivwerts der Signalstärke weit verbreitet [25]. Um diesen aus einer Gruppe an Samples zu berechnen, wird die Wurzel des Mittelwerts der quadrierten Einzelwerte gebildet. Dieser Wert wird aus einem gleitenden Zeitfenster (z. B. 10 ms) über die Audio-daten gebildet und in *dB* (Dezibel) umgerechnet. Wie im samplebasierten Algorithmus erfolgt diese Berechnung auf einem hochpassgefilterten Signal, um den Frequenzbereich zu untersuchen, in dem die Sibilanten über dem tonalen Material herausstechen.

Um die Gesamtheit der Sibilanten einzuschließen, wird für die Erkennung ein zweiter Hysteresis-Threshold implementiert, der die Sibilantenbedingung erweitert. Als Hysteresis bezeichnet man den Zustand eines Systems, das nicht nur von aktuellen Bedingungen, sondern auch von vorhergehenden Ereignissen abhängt. Diese Logik findet im Audiobereich

häufig in Noise-Gates Anwendung, bei denen unterschiedliche Thresholds zum Öffnen und Schließen festgelegt werden [26, 27]. Für den vorliegenden Anwendungsfall wird diese Logik ähnlich verwendet. Das Ende eines Sibilanten tritt demnach erst ein, wenn das Signal nicht nur unter den ersten, sondern auch unter den zweiten, niedrigeren Threshold fällt.

Nach dieser robusteren Erkennung soll auch die Trennlogik verbessert werden. Anstatt einfach nur ganze Samples bzw. Samplefenster voneinander zu trennen, soll der Sibilant auch auf Frequenzebene vom Restsignal separiert werden, um gegebenenfalls unterliegendes tonales Signal nicht mit zu bearbeiten. In Abbildung 3.1 sieht man, dass sich an den Rändern in einem reinen Gesangssignal sibilantes und tonales Signal überlappen können. Bei fehlender Frequenzdifferenzierung würde das tonale Signal mit bearbeitet werden, wenn der Sibilant direkt am Anfang über den kompletten Frequenzbereich abgeschnitten wird.

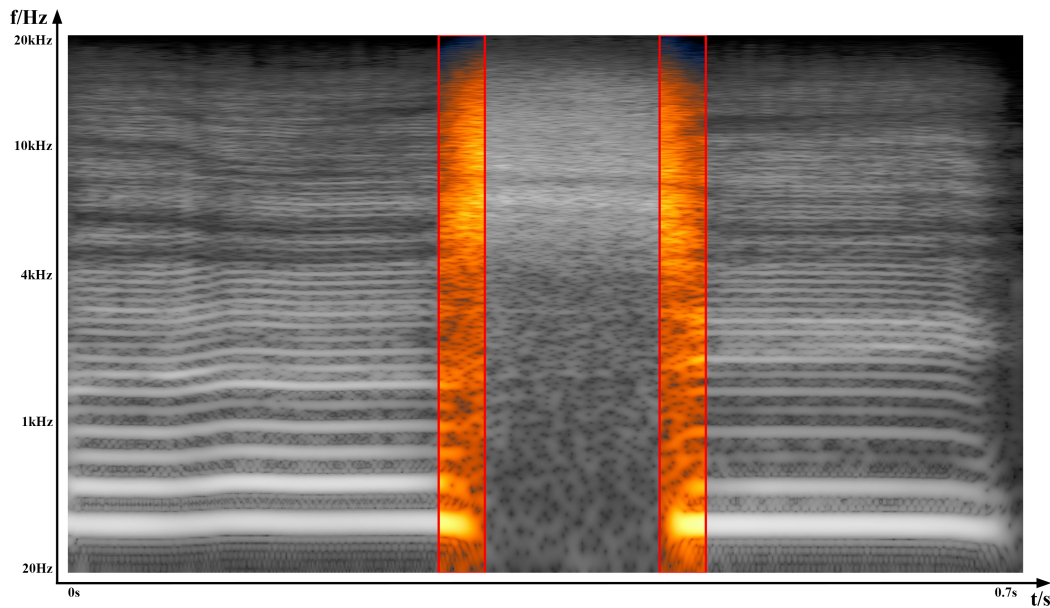


Abbildung 3.1: Spektrogramm-Darstellung Überlappung (in rot) von Sibilant und tonalem Material. Quelle: Eigene Auswertung.

Hierfür wird vor der Erkennung, wie in Abbildung 3.2 visualisiert, das originale Signal durch komplementäre Hoch- und Tiefpassfilter in zwei Frequenzbereiche aufgeteilt. Das tiefpassgefilterte Signal bleibt gänzlich unbearbeitet und wird am Ende wieder zum tonalen Signal hinzuaddiert. Die Trennung von tonalem und sibilantem Material findet nur auf dem hochpassgefilterten Signal statt. Die Trennung beginnt am Startsample des als Sibilant erkannten RMS-Fensters. Um harte Sampleübergänge an den Grenzen der Sibilanten zu vermeiden, wird ein *Crossfade* (Überblendung) implementiert. In einem Crossfade wird die Amplitude zweier Signale über einen kurzen Zeitabschnitt gegeneinander ein- und ausgeblendet. Dies verhindert ungewollte Klick-Artefakte, wenn die Audiosignale nicht exakt am Nulldurchgang abgeschnitten wurden. Diese Überblendung wird nach Beginn des Sibilanten und vor dessen Ende angewendet und sorgt so für artefaktfreie Übergänge.

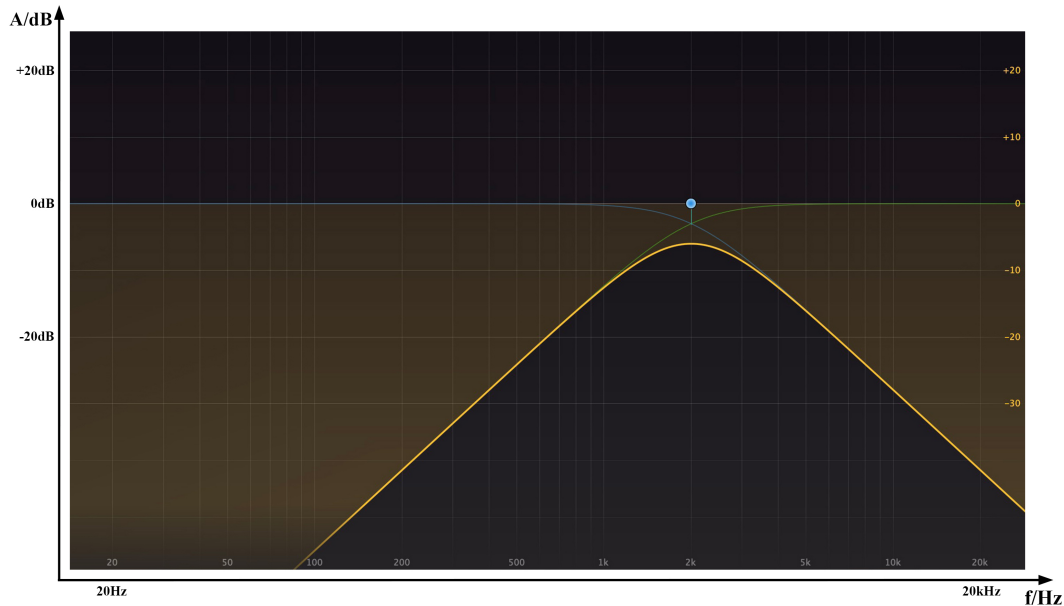


Abbildung 3.2: Komplementäre Hoch- und Tiefpassfilter in einem Equalizer. Quelle: Eigene Auswertung.

3.2.3 Spektrale Subtraktion via Fast-Fourier-Transformation

Als Alternative zu einem reinen Frequenzthreshold-basierten Verfahren und um eine detailliertere Betrachtung der Frequenzen des Audiosignals zu ermöglichen, wird für den dritten Ansatz eine spektrale Methode mittels FFT genutzt.

Wie in Abbildung 3.3 zu erkennen, verändert sich die Gewichtung der Frequenzbereiche zwischen tonalem und sibilantem Material deutlich. Das Verhältnis von hohen zu tiefen Frequenzen ist in den Sibilanten sehr hoch, wohingegen die tonalen Bereiche eher einen ausgeglichenen Frequenzgang mit einer Betonung der tiefen Frequenzen aufweisen. Da sich, wie zuvor festgestellt, einige Sibilanten mit tonalem Material überlappen, ist es allerdings nicht immer sinnvoll, nur das Verhältnis von hoch- zu tieffrequentem Signal zu vergleichen, da dies die Gewichtung der Sibilanten zur Gesamtsignalstärke außer Betracht lässt und an den überlappenden Stellen zu Fehldetektionen führen kann. Da die Betrachtung des hochfrequenten Signals im Vergleich zum Gesamtsignal die überlappenden Grundtöne weniger gewichtet, wird dieser Ansatz zur Erkennung gewählt.

Umgesetzt wird dies mithilfe der spektralen Analyse durch FFT. Ähnlich zum RMS-basierten Algorithmus wird hierfür ein Fenster (Gruppe) an Samples (z. B. 1024 Samples) mit der FFT in die Frequenzdomäne überführt. Diese Transformation gibt die Stärke der im Signal enthaltenen Frequenzen dieses kurzen Zeitbereichs als konkrete Werte zurück. Diese werden in spezifischen Frequenzbereichen zugehörigen Speicherabschnitten (Frequenz-Bins) abgelegt. Mit diesen Werten kann nun präzise das Verhältnis von hohen Frequenzen (z. B.

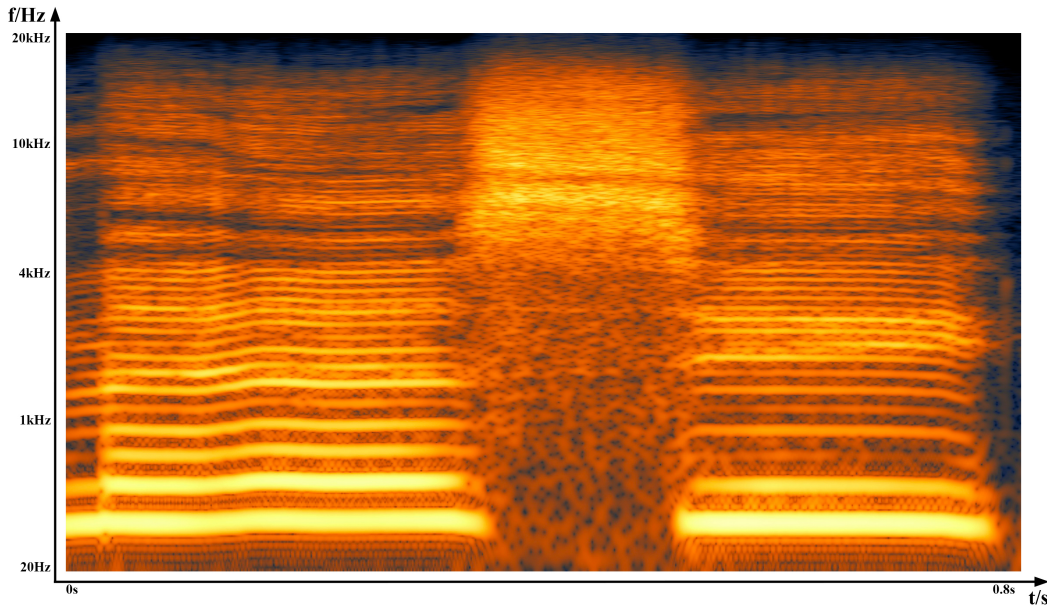


Abbildung 3.3: Sibilant (mittig) und tonales Material (links und rechts) im Spektrogramm.
Quelle: Eigene Auswertung.

ab 4 kHz) zu allen im Signal enthaltenen Frequenzen verglichen werden. Liegt beispielsweise mehr als 40% der gesamten spektralen Energie in den hohen Frequenzen, wird dieses Fenster als Sibilant erkannt.

Der Vorteil dieser spektral differenzierten Daten ist, dass auch die Frequenztrennung von tonalem und sibilantem Material in der Frequenzdomäne möglich wird. Hierzu werden die Frequenz-Bins oberhalb der Grenzfrequenz separat abgespeichert und in den restlichen Daten auf Null gesetzt. Diese getrennten Frequenzdaten können nun unabhängig voneinander ohne weitere Filterung rücktransformiert werden, um separate Signale für tonale und sibilante Anteile zu erhalten.

3.3 Auswahl der Methoden und Parameter

Die drei erläuterten Verfahren sollen in dieser Arbeit nun als Algorithmen umgesetzt und anschließend getestet werden. Hierbei stellen sich einige Fragen an die Umsetzung und Parameter der Ansätze, um das gewünschte Ergebnis zu erzielen. Um die Algorithmen ideal vergleichbar zu machen, müssen folgende Parameter korrekt eingestellt und getestet werden:

- **Grenzfrequenz (Cutoff) der Filter:** Welche Frequenz für die Trennung des sibilanten und tonalen Bereichs ist sinnvoll? Gibt es Unterschiede zwischen Frauen- und Männerstimmen?

- **Threshold:** Wie reagiert die Erkennung des sample- bzw. RMS-basierten Algorithmus auf unterschiedliche Schwellenwerte?
- **Transformationsparameter der FFT:** Beeinflusst die Auswahl der Fenstergröße in der FFT die Frequenz- und Zeitauflösung maßgeblich?
- **Hysteresis- und Crossfade-Längen:** Wie stark verbessern die Sample- bzw. Threshold-Hysteresis der filterbasierten Algorithmen das Ergebnis?

3.4 Teststrategie und Evaluationskriterien

Für den Test und die Bewertung der zu implementierenden Algorithmen werden folgende Testaufnahmen, Auswertungsmethoden und Kriterien verwendet.

3.4.1 Testdaten

Für die Beschaffung der Testdaten mit voller Kontrolle über den Signalweg werden im Tonstudio eigene Gesangsspuren aufgenommen und bei der Entwicklung sowie den Tests der Algorithmen verwendet. Der Anspruch an diese Aufnahmen liegt in der Übertragbarkeit der Testergebnisse auf den Standard moderner professioneller Audioproduktionen. Kriterien hierfür sind:

- **Hochqualitativer Signalweg:** Die Wahl eines der Industriestandards entsprechenden hochqualitativen Signalwegs. Dieser besteht für die verwendeten Aufnahmen aus:
 - Einer Aufnahmeumgebung ohne übermäßigen natürlichen Raumklang
 - Einem Großmembran-Kondensatormikrofon (gängigste Wahl für Gesangsaufnahmen)
 - Einem Audiointerface mit neutralen Vorverstärkern (Preamps) und rauscharmer Analog-Digital-Wandlung
 - Minimaler EQ und Kompression auf dem Aufnahmeweg (2–3 dB Gain Reduction, Hochpassfilter bei 100 Hz und High-Shelf mit 2–3 dB Höhenanhebung)
- **Professionelle Sänger:** Die Aufnahme von professionellen Sängern ohne bewusste Über- oder Unterbetonung der Sibilanten.

3.4.2 Bewertung der Audioqualität

Zur Beurteilung und Auswertung der Algorithmen werden verschiedene Tests durchgeführt, um sie miteinander zu vergleichen. Dazu kommen folgende Methoden zur Auswertung der später genannten Kriterien zum Einsatz:

- **Subjektive Hörtests:** Durch AB-Vergleiche wird festgestellt, inwiefern sich die Audioqualität und der Höreindruck verändert haben.
- **Spektrogramm- und Waveform-Betrachtung:** Als visuelle Hilfe zur Erkennung von entstandenen Artefakten oder Rückständen sowie zur Beurteilung der Effektivität der Trennung wird die Betrachtung der gefilterten Audiosignale in einem iZotope RX Spektrogramm verwendet.
- **Phasenumkehrtest (Nulltest):** Zur Beurteilung der Veränderung zweier Signale zueinander wird die Phase eines der Signale invertiert, bevor beide Signale summiert werden. In den Rückständen lassen sich Phasendrehungen und Bearbeitungsreste erkennen.

Als Artefakt gilt in der Bewertung unter anderem:

- **Klicks und Pops:** kurze, impulsartige Störgeräusche, die häufig an Übergängen der getrennten Signale auftauchen können. Diese lassen sich im Spektrogramm und meist auch in der Waveform gut erkennen.
- **Phasendrehung oder Comb-Filtering:** häufig hörbar als hohler oder verwaschener Klang bzw. als Verschiebungen im Frequenzgang. Dies tritt entweder an den Übergängen der Sibilantstellen, bei der Rückkombination von gefilterten Frequenzbereichen oder im gesamten Signal (erkennbar durch den Nulltest) auf. Häufig ist dies die Auswirkung von nichtlinearer Filterung oder inkorrektur Anwendung der FFT-Transformation.
- **Verzerrungen:** hörbar durch Veränderung des Klangbildes und erkennbar im Nulltest.

3.4.3 Bewertung der Erkennungsgenauigkeit

Für die Beurteilung der Erkennungsgenauigkeit werden die getrennten Signale in der DAW mit einem manuell ausgeschnittenen Signal verglichen, um Fehler in der Erkennung zu beobachten. Zur Präzision der Erkennung werden diese Signale ebenfalls im Spektrogramm gegenübergestellt, um Rückstände und Fehler zu erkennen.

3.4.4 Definierte Testziele (Kriterien 1–4)

Mit den genannten Methoden sollen die Algorithmen auf die in der Tabelle 3.1 definierten 4 Testkriterien untersucht werden.

Tabelle 3.1: Zusammenfassung der Testkriterien 1–4 für die Bewertung.

Kriterium	Messmethode	Ziel/Erwartung
1 Hörbare Reduktion von Sibilanten	Subjektiver Hörtest, Spektrogrammvergleich	Weniger hervortretende Zischlaute, ohne das restliche Signal dumpf zu machen
2 Präzise Erkennung der Sibilanten	Overlay-Ansicht in der Testapplikation, Vergleich mit manuell ausgeschnittenen Sibilanten	Möglichst alle Sibilanten erkennen, wenige Fehlerkennungen im tonalen Material
3 Saubere Trennung (Artefaktfreiheit)	Visuelle Inspektion von Waveform/Spektrogramm, Erkennung von Klicks/Pops	Keine hörbaren Übergangsklicks oder Lücken, möglichst keine Überreste im Sibilanten- oder Restbuffer
4 Minimale Beeinflussung des Restsignals	Phasenumkehrtest (Nulltest), Subjektiver Hörtest	Restsignal bleibt unverfälscht. Keine signifikanten Rückstände im Nulltest oder Comb-Filtering bei Signal Addition.

Unter Beachtung der Anforderungen für die Algorithmen und die Applikation werden die zwei Frequenzschwellen-basierten Ansätze sowie der spektrale Ansatz mittels FFT im nächsten Kapitel implementiert. Im darauf folgenden Kapitel 5 werden diese Algorithmen dann Anhand der entwickelten Teststrategie unter Hinzunahme der Kriterien getestet und in Kapitel 6 bewertet.

Kapitel 4: Implementierung

Im Folgenden wird zuerst der Aufbau der Applikation, in der die Algorithmen implementiert sind, unter der Verwendung von JUCE erläutert. Nachdem anschließend auf das GUI eingegangen worden ist, werden die einzelnen Algorithmen im Detail mit den bei der Umsetzung entstandenen Problemen erklärt.

4.1 Implementierung mit JUCE und Aufbau der Applikation

Um die geplanten Ansätze nun effektiv umzusetzen, ist das Framework **JUCE** eine essenzielle Unterstützung. Wie schon vorab beschrieben, bietet es alle grundlegenden Bausteine an Audio- und GUI-Klassen sowie Funktionen für die Entwicklung des Testprogramms. Entwickelt wurde der Programmcode in *Xcode* unter *macOS* mit Hilfe der JUCE-spezifischen *Projucer*-Anwendung für die Konfiguration des Frameworks [28].

4.1.1 Audio-Processing-Schritte und JUCE-Klassen

Wichtig für das Verständnis der Umsetzung der Algorithmen im Programmcode ist der Linkwitz-Riley-Filter und die JUCE FFT-Klasse.

Linkwitz-Riley-Filter: Linkwitz-Riley-Filter finden im DSP besonders häufig Anwendung, wenn es um die Frequenztrennung von Audiosignalen geht, die ohne Pegelabweichungen oder Phasenauslöschungen wieder kombiniert werden sollen. Häufige Anwendungsbereiche sind somit Multiband-Kompressoren und ähnliche Verfahren. Die Charakteristik ergibt sich aus zwei in Reihe geschalteten Butterworth-Filtern gleicher Ordnung, wodurch eine steilere Absenkung und eine gleichmäßige Phasenlage erreicht werden. Es ist sehr relevant zu beachten, dass Linkwitz-Riley-Filter in DSP-Anwendungen sowohl Phasendrehungen als auch Latenz im gefilterten Signal auslösen. Um Phasenauslöschungen und Sample-genaue Rückkombinierung zu gewährleisten, müssen demnach auch Signalanteile, in denen der komplette Frequenzanteil verbleibt, mit einem als Allpass konfigurierten Linkwitz-Riley-Filter bearbeitet werden. Dieser nimmt keine Frequenzanteile weg, stellt aber sicher, dass die Phasenkohärenz und Latenz mit dem gefilterten Signal übereinstimmen [29].

JUCE stellt für die Filterung mit einem Linkwitz-Riley-Filter eine eigene Klasse zur Verfügung, welche auch in der Implementierung genutzt wird und die Verwendung des Linkwitz-Riley-Filters vereinfacht.

JUCE FFT: Die in JUCE enthaltene DSP-Klasse für die FFT bietet eine effiziente Implementierung des Algorithmus, die hier für spektrale Arbeit genutzt werden kann. Mit `performRealOnlyForwardTransform` kann das Signal in den Frequenzbereich transformiert werden, während `performRealOnlyInverseTransform` die Rücktransformation in den Zeitbereich übernimmt [30].

Um die FFT korrekt anzuwenden, wird das Signal in Abschnitte der Größe `FFTSize` Samples unterteilt. Die Betrachtung einzelner Samples kann keine Information über den Frequenzanteil des Signals liefern, weshalb immer eine Gruppe von mehreren Samples betrachtet werden muss. Hier geht unausweichlich etwas Zeitpräzision verloren, weshalb die Größe des Fensters bewusst gewählt werden muss, um den besten Kompromiss zwischen Frequenz- und Zeitauflösung zu bieten. Um Fehlerkennungen von enthaltenen Frequenzen (Spektral Leakage) zu reduzieren, muss auf jedes Fenster vor der Transformation eine Faktorkurvenfunktion (Fenster-Funktion) angewendet werden, welche die Amplitude der Samples im Fenster gewichtet. Entschieden wird sich hier für das Hann-Fenster, eine der gängigsten Funktionen für diesen Einsatzzweck. Das Hann-Fenster wird definiert durch [31]:

$$w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{N-1}\right) \right), \quad n = 0, 1, \dots, N-1, \quad (4.1)$$

wobei N die Länge des Fensters in Samples bedeutet.

Durch die Anwendung der Fenster-Funktion entstehen bei Verschiebung des Fensters in Fensterlänge Lücken im Signal. Dies kann verhindert werden, indem das Fenster nicht in Fensterlänge, sondern mit einem Faktor kleiner als der Fensterlänge verschoben wird. Dadurch werden Lücken bei der Addierung des rücktransformierten Signals (Overlap-Add-Methode) vermieden. Diesen Faktor nennt man Hop-Size und er gibt an, mit welcher Schrittweite das Fenster über die Samples verschoben wird, bevor die FFT angewendet wird. Um das Signal schlussendlich wieder korrekt rücktransformieren zu können, muss bei der Wahl der Hop-Size die *COLA*-Bedingung (Constant Overlap Add) erfüllt werden. Durch die Erfüllung dieser Bedingung wird sichergestellt, dass sich die überlappenden Fensterabschnitte so summieren, dass ihre Gewichtung an jeder Stelle konstant bleibt. So kann das Originalsignal fehlerfrei rekonstruiert werden. Dies wird z. B. mit einer Hop-Size von 0,5 erreicht, ohne nachträglich eine Amplitudenanpassung vornehmen zu müssen [5, Kapitel 20].

Die korrekte Anwendung des Hann-Fensters, die richtige Wahl der Hop-Size unter Anwendung der Overlap-Add-Methode und die Fenstergröße sind für die Implementierung der FFT essenziell.

4.1.2 Architektur

Für den grundlegenden Aufbau der Applikation wurde mit dem von JUCE zur Verfügung gestellten Tutorial *Audio Thumbnail* begonnen [32]. Für die Struktur des Audio Processing Setups wurde sich an dem Programmcode des *Simple Multiband Comp Tutorials* der MatCat Music LLC orientiert [18, 33]. Die Kombination dieser beiden Ansätze für die Umsetzung von Audioapplikationen hat eine gute Basis für einen modularen Grundaufbau der Applikation geboten, der es erlaubt, die späteren Algorithmen in den Signalweg einzuschleifen und somit leichter gegeneinander zu vergleichen. Die daraus resultierende Klassenstruktur mit Trennung von UI und DSP (Separation of Concerns) ist in Abbildung 4.1 zu erkennen.

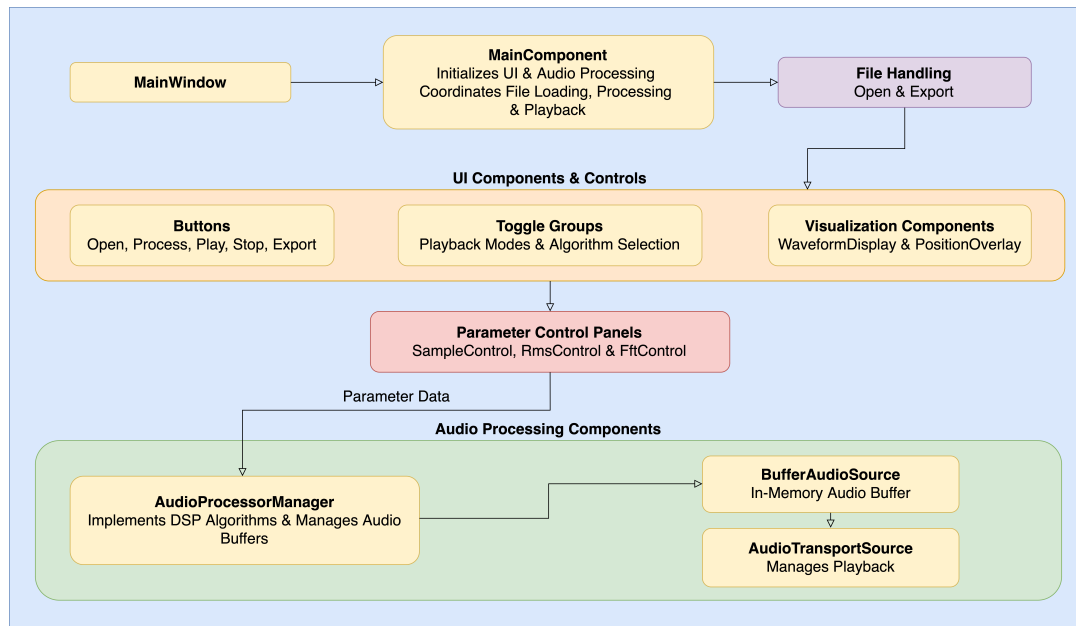


Abbildung 4.1: Überblick der Architektur der Applikation. Quelle: Eigene Auswertung.

Das grundlegende Vorgehen der Applikation ist es, eine Audiodatei einzulesen, die in einem Buffer gehaltenen Daten durch einen in den Signalweg einschleifbaren Algorithmus zu bearbeiten und in verschiedene Buffer aufzuteilen, um die Sibilanten zu separieren. Danach wird der ausgewählte bearbeitete Buffer wieder als Audiodatei exportiert, um die Ergebnisse richtig analysieren zu können.

Der grundsätzliche Aufbau des *Audio Thumbnail Tutorial*-Codes bietet eine Applikation, in der Audiodateien geladen und über die `AudioTransportSource`-Klasse mittels einer State Machine für Start- und Stopp-Controls abgespielt werden können [34]. Des Weiteren wird eine Waveform durch die JUCE-Klasse `AudioThumbnail` erzeugt, die zur Visualisierung des Audiofiles genutzt wird, sowie ein `PositionOverlay`, welches erlaubt, an eine bestimmte Abspielposition zu springen [32]. Das größte Problem dieses Applikations-Aufbaus ist, dass die `AudioTransportSource` nie die kompletten Daten der Datei einliest, sondern beim Abspielen direkt aus der Audiodatei wiedergibt. Dies ist für die geplante Offline-Bearbeitung

nicht geeignet. Abhilfe schafft hier die eigens hinzugefügte `BufferAudioSource`, welche die JUCE-Klasse `PositionableAudioSource` implementiert und somit die Transport Source ersetzt, um nun aus einem Buffer statt direkt aus der Datei abspielen zu können [35].

Der Aufbau der `AudioProcessorManager`-Klasse orientiert sich wie erwähnt an dem Vorbild des *Simple Multiband Compressor Tutorials*. Dort wird ein Audio-Signalfuss für eine Audio-Plugin-Anwendung vorgestellt, in der nach der Philosophie *Separation of Concerns* das Audio Processing vom Rest der Applikation getrennt wird. Außerdem werden hier sowohl die Parameterübergabe und -speicherung mittels eines algorithmusübergreifenden *struct* als auch die Grundlage für die Arbeit mit mehreren Buffern und die Frequenztrennung durch die bereits erklärten Linkwitz-Riley-Filter geschaffen, womit die Grundbausteine für die Implementierung der Filterungsalgorithmen gegeben sind [18, 33].

Der vereinfachte Signalfuss der App sieht demnach folgendermaßen aus:

1. Laden einer Datei (`FileChooser`).
2. Einlesen in einen Buffer (`AudioFormatReader` → `AudioBuffer`).
3. Weitergabe an die De-Essing-Algorithmen (Sample/RMS/FFT).
4. Trennung in Sibilanten- und tonalen Anteil auf verschiedene Buffer.
5. Auslesen des ausgewählten Buffers für die Wiedergabe als `TransportSource`.
6. Darstellung im GUI als Waveform.
7. Export der verarbeiteten Dateien.

Neben dem `MainComponent`, welcher das GUI, den Import und Export der Audiofiles und das Abspielen der Buffer verwaltet, ist für das Verständnis der Implementierung der Algorithmen besonders die Implementierung des `AudioProcessorManager` wichtig. Denn hier wird die zentrale DSP-Logik verwaltet.

Essenziell sind hier die vier *JUCE AudioBuffer* [36]:

- `originalBuffer`: Speicherung des unverarbeiteten Signals.
- `processedBuffer`: Speicherung des rückkombinierten Signals aus tonalem Material und lautstärkeangepassten Sibilanten.
- `sibilantBuffer`: Speicherung der reinen Sibilanten.
- `noSibilantBuffer`: Speicherung des tonalen Signals ohne Sibilanten.

Diese werden in den De-Essing-Algorithmen verwendet, um die verschiedenen Zustände der verarbeiteten Audiodaten zu speichern.

Um nun in der Applikation die Algorithmen implementieren zu können, wurde die `processFileForSibilant` Funktion als das Herzstück des `AudioProcessorManager` implementiert. Diese wird durch

die hinzugefügte `processFile` Funktion des `MainComponents` aufgerufen. Sie liest die ausgewählte Datei ein, speichert sie im `originalBuffer` und ruft den ausgewählten De-Essing-Algorithmus auf.

4.2 GUI und Parameter

Um die Anwendung richtig nutzen und auch umfassend testen zu können, wurden neben den Grundfunktionalitäten auch verschiedene Parameter zur Kontrolle der Algorithmen implementiert. Hierüber kann z. B. die Crossover-Frequenz oder die Empfindlichkeit der Sibilantenerkennung eingestellt werden. Wie in Abbildung 4.2 zu sehen, können diese Parameter über das GUI verändert werden.

Wichtig für die Wahl der Parameter ist das implementierte Waveform-Overlay (in Rot), welches die im Sibilanten-Buffer gehaltenen Daten der erkannten Sibilanten visualisiert und über die Waveform des Ausgangssignals (in Blau) legt.



Abbildung 4.2: Bildschirmaufnahme der Testapp nach Verarbeitung einer Gesangsspur durch den FFT-based Algorithmus. Quelle: Eigene Auswertung.

4.3 Sample-based Algorithmus

Der *Sample-based Algorithmus* ist der einfachste der drei Algorithmen. Er trennt das Signal in drei Buffer auf. Wie in der Abbildung 4.3 zu erkennen, wird der `highPassBuffer` mit einem als Hochpass konfigurierten Linkwitz-Riley-Filter bearbeitet, sodass nur noch die hohen Frequenzen ab der eingestellten Cutoff-Frequenz in diesem Buffer vorhanden sind. Dieses Signal dient in diesem Algorithmus nur als Trigger für die Erkennung der Sibilantenanteile. Die anderen beiden Buffer sind zu Beginn des Algorithmus identisch und beinhalten das mit einem Linkwitz-Riley-Allpass-Filter bearbeitete volle Ausgangssignal.

Ablauf (siehe Abbildung 4.3):

- Es wird *Sample für Sample* über das Signal iteriert.
- Im `highPassBuffer` wird kontrolliert, ob das aktuelle Sample den eingestellten Threshold für die Detektion eines Sibilanten überschreitet.
- Falls ja, wurde ein Sibilant entdeckt und die Bedingung greift. Der Hysteresis-Counter, der verhindern soll, dass nur einzelne Samples als Sibilant betrachtet werden, wird auf die eingestellte Größe gesetzt und zählt herunter.
- Solange diese Bedingung gilt (das Sample gehört zu einem Sibilanten), werden die betroffenen Samples im `mainBuffer` auf Null gesetzt. Für alle Samples außerhalb dieser Bedingung wird der `sibilantBuffer` auf Null gesetzt.
- So verbleiben im `mainBuffer` nur das Signal ohne Sibilanten und im `sibilantBuffer` nur die Sibilanten.
- Im letzten Schritt wird dann die Information aus `mainBuffer` und `sibilantBuffer` in den `finalBuffer` geschrieben, wobei auf die Samples im `sibilantBuffer` die Lautstärkereduktion als Faktor angewandt wird.

Die drei Zustände zur Ausgabe und dem Datelexport werden dann in die jeweiligen Klassen-Buffer kopiert.

Probleme: Bei der Implementierung des *Sample-Based Algorithmus* sind mehrere Probleme aufgetreten. Erstens erwies sich die Erkennung einzelner Samples als nicht besonders praxistauglich für die Detektion von Sibilanten. Einzelne den Threshold überschreitende Samples sind nicht sehr repräsentativ für die Erkennung des Ausschlags der gesuchten Frequenzen. Zwar löst die vorhergehende Hochpassfilterung einen Großteil des Problems der Erkennung, aber es werden trotzdem nur die lautesten Samples herausgelöscht, was bei einer Waveform mit Werten, die regelmäßig durch Null gehen, nicht zielführend ist und durch das ständige An- und Ausschalten des Signals zu extremen Verzerrungen führt. Die Hinzunahme der *Hysteresis*-Funktion ist also essenziell, um dieses Verhalten abzufangen.

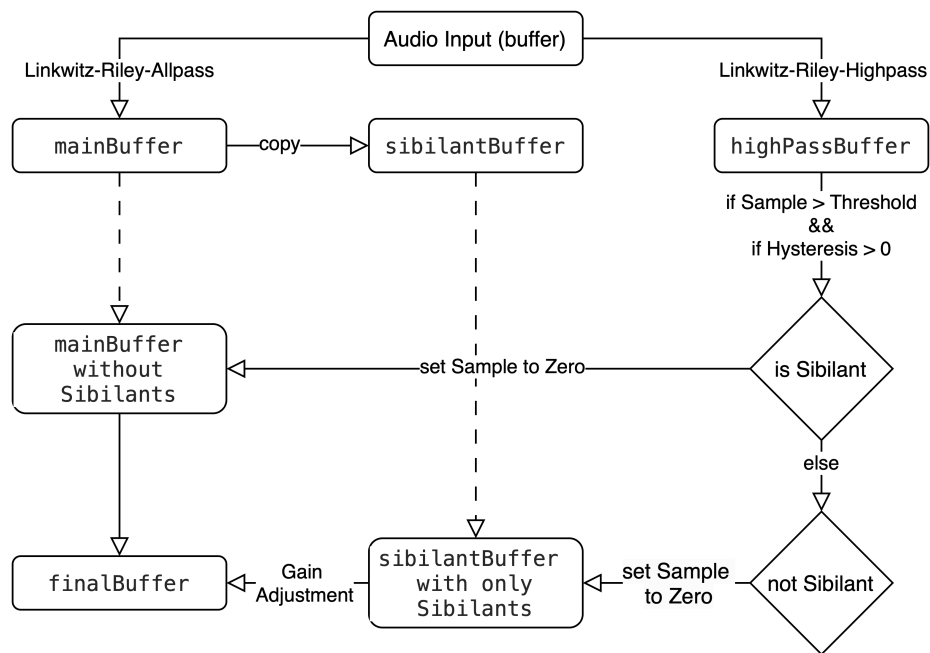


Abbildung 4.3: Datenfluss des Sample-based Algorithmus. Quelle: Eigene Auswertung.

Ein weiteres Problem bestand in der ursprünglichen Idee, die Werte des hochpassgefilterten Signals von den Samplewerten des gesamten Signals zu subtrahieren, um eine Frequenztrennung in den Sibilanten zu erreichen. In der Praxis hat die fehlende Erkennung geringer Samplewerte zu unerwünschten Verzerrungen und Frequenzverschiebungen geführt. Die erkannten Sibilanten klangen sehr dumpf mit zusätzlichem knisterähnlichen Rauschen. Diese Experimente während der Entwicklung haben zu dem Verwerfen dieses Ansatzes geführt. Das daraus resultierende Verzicht auf diese Frequenztrennung hat in dem Algorithmus letztlich zu den klanglich besten Ergebnissen geführt.

4.4 RMS-based Algorithmus

Der *RMS-based Algorithmus* ist im Vergleich zum Sample-based Algorithmus deutlich komplexer, da er neben der Berechnung und Betrachtung des RMS-Werts auch einen Crossfade (*Überblendung*) an den Übergängen der Sibilanten implementiert und sich dabei die Möglichkeit der Offline-Bearbeitung des Gesamtsignals zunutze macht.

Ablauf (siehe Abbildung 4.4):

- Das Ausgangssignal wird im ersten Schritt wieder auf drei Buffer aufgeteilt.

- Statt mit einem Allpass-Filter wie im vorherigen Algorithmus wird der `lowPassBuffer` mit einem Linkwitz-Riley-Lowpass-Filter an der eingestellten Crossover-Frequenz bearbeitet, während die anderen beiden Buffer ein an der selben Frequenz hochpassgefiltertes Signal enthalten.
- Dadurch wird die Sibilantentrennung nur im hochfrequenten Signal durchgeführt, während tieffrequente Anteile gänzlich *unbetrachtet* bleiben.
- Um nun im hochfrequenten Signal Sibilant- und Tonanteile zu trennen, wird wieder *Sample für Sample* über die Bufferlänge iteriert. Diesmal werden jedoch nicht einzelne Samples, sondern ein *RMS*-Fenster betrachtet.
- Die Samples werden in eine FIFO-Queue (Länge z. B. 10 ms) geschrieben. Aus den Werten in dieser Queue wird laufend in jeder Iteration der RMS-Wert berechnet.
- Überschreitet der RMS-Wert den eingestellten Threshold, wird der Index des aktuellen Samples als Start eines Sibilanten in einem Tupel des `sibilantSegments Vector` gespeichert. Um die Sibilant-Bedingung wieder zu verlassen, muss der RMS-Wert unter den um den Hysteresis-Wert reduzierten Threshold fallen. Dann wird das aktuelle Sample als Endpunkt eingetragen.
- Nachdem das gesamte Signal durchlaufen ist und alle Sibilantensegmente mit Start- und Endpunkt im Vektor gespeichert sind, werden diese Daten auf die Samples im Buffer angewendet.
- Ein Bool-Vector (`true/false`) markiert die zugehörigen Samples. Alle Indizes zwischen den Start- und Endpunkten eines Sibilantensegments bekommen `true`, der Rest `false`.
- Entsprechend wird im `sibilantBuffer` jedes Sample auf Null gesetzt, wenn dessen Index im Bool-Vector `false` ist. So enthält der `sibilantBuffer` nur noch Sibilantenteile.
- Ein zusätzlicher Crossfade (z. B. 5 ms) sorgt für weiche Übergänge am Anfang und Ende der Segmente.
- Um die Sibilanten aus dem regulären Signal zu entfernen, werden die Samplewerte aus dem `sibilantBuffer` vom `mainBuffer` subtrahiert. Anschließend kann der `sibilantBuffer` mit einer gewünschten Pegelreduktion wieder hinzugemischt werden.
- Ebenfalls wird der `lowPassBuffer` im `finalBuffer` hinzuaddiert, um die tieffrequenten Anteile beizubehalten.

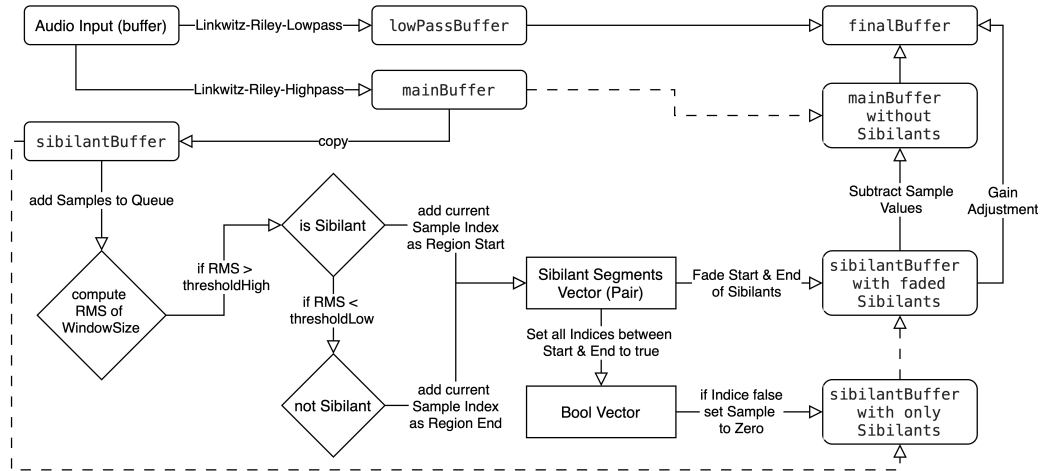


Abbildung 4.4: Datenfluss des RMS-based Algorithmus. Quelle: Eigene Auswertung.

Probleme: Die größte Hürde bei der Implementierung des RMS-based Algorithmus war die Implementierung des *Crossovers*. Zunächst führte das Abschneiden der Sibilanten von einem Sample auf das nächste wie im Sample-based Algorithmus zu hörbaren Klick-Artefakten an den Übergängen zwischen Sibilant und tonalem Material, wenn das Signal getrennt oder lautstärkeangepasst rekombiniert wurde. Um dies zu verhindern, wird eine Überblendung an den Übergängen implementiert. Wichtig ist hierbei, nicht die Grenzen des Sibilanten zu verlassen, um zu verhindern, dass in das tonale Material eingegriffen wird. Da die Applikation *nicht echtzeittauglich* sein muss, konnte dies über die als Tupel gespeicherten Start- und Endpunkte der erkannten Sibilanten gelöst werden. So lässt sich flexibel auf diese Grenzbereiche zugreifen und eine komplexere Crossfade-Logik umsetzen, welche auch vom Endpunkt des Sibilanten zurück in den Sibilanten überblenden kann.

4.5 FFT-based Algorithmus

Im Gegensatz zu den beiden vorhergehenden Algorithmen ist der Ansatz der *Fast Fourier Transformation* ein wenig anders strukturiert. Hier ist keine manuelle Filterung durch Hoch- oder Tiefpassfilter nötig, da dieser Teil in der durch die FFT entstehenden Frequenzdomäne umgesetzt wird.

Ablauf (siehe Abbildung 4.5):

- Bei der bekannten Iteration über den `mainBuffer` wird nicht *Sample für Sample*, sondern in Abschnitten (Fenstern) mit einer definierten `FFTSIZE` gearbeitet, die in Schrittweite der Hop-Size über den Buffer geschoben werden.

- In jedem Iterationsschritt wird das Fenster durch die Anwendung der Hann-Funktion gewichtet und anschließend die `performRealOnlyForwardTransform` (FFT) durchgeführt. Dadurch wird jedes Fenster in Frequenzbins zerlegt.
- Nun wird festgestellt, ob das vorliegende Fenster zu einem Sibilanten gehört oder nicht. Dazu wird anhand der Bins das Verhältnis von hochfrequentem Anteil zu Gesamtanteil berechnet.
- Ist dieses Verhältnis größer als das eingestellte *Detection Ratio*, wird der betroffene Frame als Sibilant markiert.
- In diesem Fall werden die Bins, die oberhalb des, der Cutoff-Frequenz naheliegendsten, Frequenzbins liegen, in einen Sibilanten-Vektor kopiert und im ursprünglichen Vektor auf Null gesetzt.
- Parallel erfolgt eine Überblendung, welche die Werte in einem Bereich von z. B. 1 kHz über der Cutoff-Frequenz sanft von Null auf Eins bzw. umgekehrt interpoliert, um Aliasing in der inversen Transformation zu minimieren.
- Sind die Bins aufgeteilt, werden sie getrennt voneinander mit der inversen FFT (`performRealOnlyInverseTransform`) in die Zeitdomäne zurücktransformiert.
- Schließlich werden die rücktransformierten Frames per Overlap-Add wieder addiert und in `dryBuffer` bzw. `sibilantBuffer` gespeichert.
- Wie schon in den beiden vorherigen Algorithmen werden diese Buffer mit lautstärkeangepassten Sibilanten im `finalBuffer` wieder zusammenaddiert.

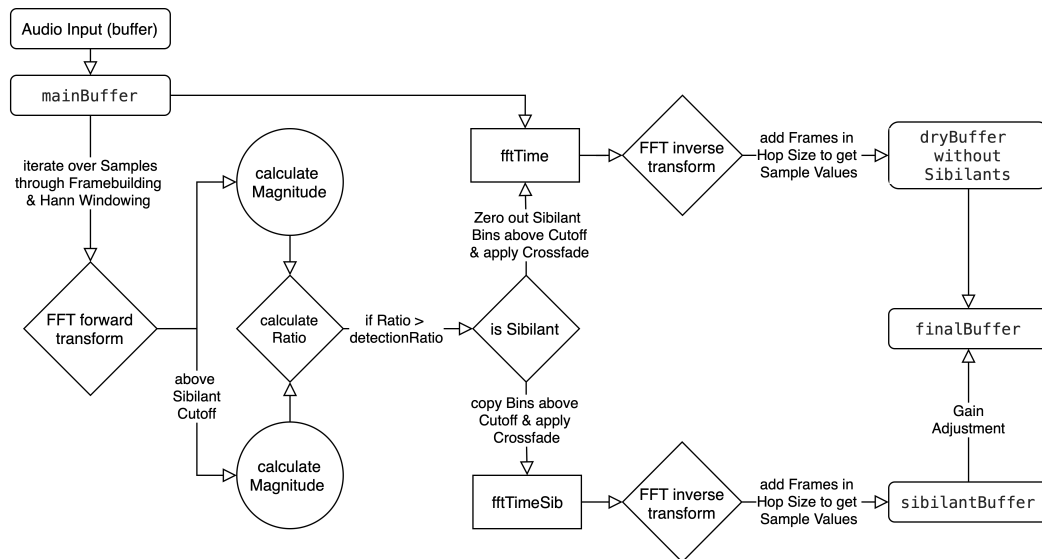


Abbildung 4.5: Datenfluss des FFT-based Algorithmus. Quelle: Eigene Auswertung.

Probleme: Obwohl der *FFT-basierte Algorithmus* logisch nicht so komplex erscheint wie der RMS-basierte Algorithmus, hat er in der Praxis zu den meisten Problemen geführt. Das primäre Problem ist, die FFT so zu implementieren, dass die Rücktransformation ein artefaktfrei klingendes Signal zurückgibt. Anfänglich kommt es zu extremem Comb-Filtering und Phaseneffekten. Gründe dafür sind zum einen eine fälschlicherweise erneute Anwendung des Hann-Windows, was zu Verzerrungen der Frequenzwerte führt, und zum anderen eine fehlerhafte Overlap-Add-Logik, bei der mit den Samples des nächsten Frames die letzte Hälfte der vorherigen Werte überschrieben wird anstatt sie zu addieren. Dies führt zu einem erheblichen Qualitätsverlust im finalen Signal, ähnlich klingend zu einer mp3 Datei mit extrem schlechter Bitrate.

Ein weiterer Punkt der ersten FFT Implementierungsversuche ist deutliches Aliasing an der Crossover-Frequenz, an der der Frequenzbereich der erkannten Sibilanten vom restlichen Signal abgetrennt wird. Dies lässt sich auf das fälschliche Errechnen nicht existierender Frequenzen in der inversen FFT zurückführen. Die Transformation versucht ein vollständiges Signal zu rekonstruieren und füllt die genullten Lücken fälschlicherweise mit Frequenzmaterial auf. Dies entsteht dadurch, dass die FFT mit begrenzten Frequenzbins arbeitet und ein gewisser Anteil an Approximation im Signal unvermeidlich ist. Gelöst wird dies durch eine Überblendung der Randfrequenzen zwischen Sibilanten- und Restsignal. Indem im Bereich von z. B. 1 kHz über der Crossover-Frequenz die Frequenzinformationen im Restsignal linear auf Null gesetzt und im Sibilanten-Signal umgekehrt von Null auf Eins eingeblendet werden, entsteht ein sanfter Übergang. Wie in der Abbildung 4.6 gut zu erkennen, wird so das Aliasing in der inversen Transformation deutlich reduziert.

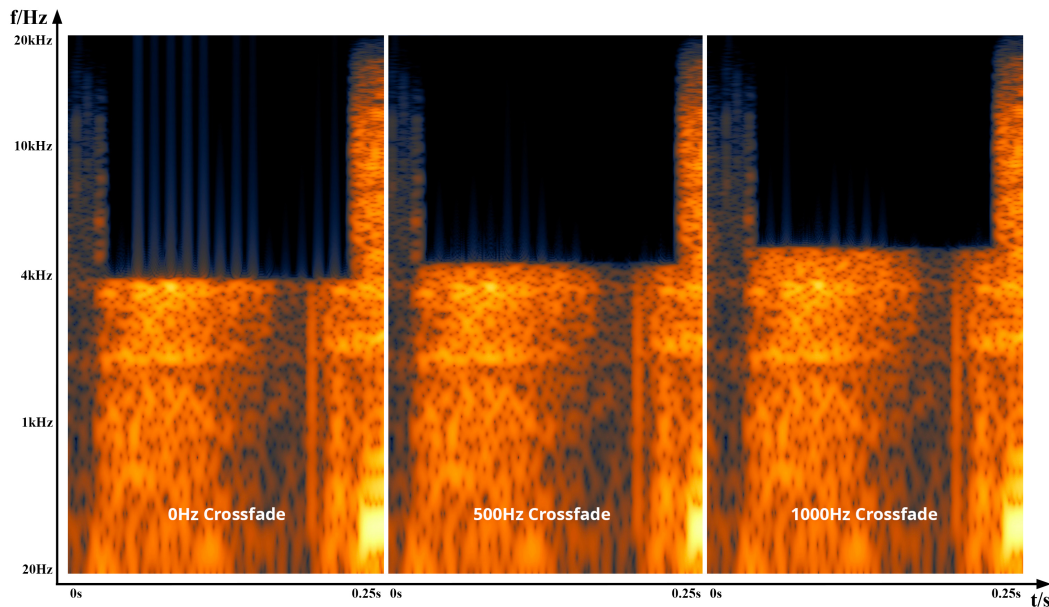


Abbildung 4.6: Auswirkung der Crossfade Implementierung auf das Aliasing im Spektrogramm. Quelle: Eigene Auswertung.

Kapitel 5: Tests und Validierung

In dem folgenden Kapitel werden die zuvor beschriebenen und in der Testapplikation implementierten Algorithmen gegeneinander auf Erfüllung der in Kapitel 3 definierten Anforderungen getestet. Zuerst wird die verwendete Testumgebung und die Wahl der Parameter aufgezeigt, bevor die Beobachtungen der zwei Testphasen beschrieben werden.

5.1 Testumgebung

Um beurteilen zu können, ob die drei implementierten Algorithmen auch die angestrebten Eigenschaften erfüllen und anwendungstauglich sind, bietet sich das im Folgenden beschriebene Testsetup an, um die vier vorab definierten Kriterien zu überprüfen. Die Basis für die Tests bilden zwei unbearbeitete und professionell im Tonstudio aufgenommene Gesangsspuren (jeweils eine männliche und eine weibliche Stimme). Für den Test der Abstraktionsfähigkeit der Algorithmen auf andere Anwendungsbereiche außerhalb von Gesangsaufnahmen wird ebenfalls eine unter denselben Konditionen aufgenommene Akustikgitarrenspur hinzugezogen. Diese Akustikgitarrenspur wird allerdings nur am Ende als Vergleich hinzugezogen und bildet nicht die Bewertungsgrundlage der Algorithmen. Bis auf minimale Kompression und Höhenanhebung auf dem Aufnahmeweg sind diese Spuren nicht weiter nachbearbeitet und demnach reale Beispiele für den Anwendungsbereich in professionellen Produktionen.

Um für die Bewertung der Artefaktfreiheit einen Vergleich zu haben, werden außerdem in einer DAW (Digital Audio Workstation) durch manuelle Trennung der Sibilanten in den beiden Gesangsspuren anhand der Waveform drei Referenzspuren geschaffen:

1. Das tonale Signal ohne Sibilanten
2. Nur die Sibilanten
3. Die Zusammenmischung des tonalen Signals mit um 9 dB abgesenkten Sibilanten

Die Absenkung um 9 dB wurde hier gewählt, um die maximale Grenze an Reduzierung zu testen, bei der die Gesangsaufnahme noch natürlich klingt. Eine Reduzierung um 9 dB bedeutet, dass die Sibilanten nur noch ein Drittel ihres ursprünglichen Ausschlags haben. Solche Werte sind aus eigener Erfahrung mit problematischen Gesangsspuren in der praktischen Anwendung nicht ungewöhnlich.

Zusätzlich werden, unter anderem mit Hilfe der manuell getrennten Signale, einige erweiterte Testszenarien erstellt, um die Grenzen der Algorithmen zu testen:

- **Tonal -6 dB:** Das tonale Material wurde um 6 dB abgesenkt, damit die Sibilanten im Verhältnis lauter sind (entspricht einer Reduktion von 50 %).
- **Sibilanten -6 dB:** Die Sibilanten werden künstlich um 6 dB leiser gemacht, damit diese nicht so herausstechen.
- **Mit Musik:** Unter die Gesangsspuren wird ein Sinuston gelegt, der im Grundton der Tonart und in derselben Oktave mit gleicher Lautstärke wie der Grundton der Gesangsspur konstant über das gesamte Signal mitläuft. Dies verstärkt den Grundtonbereich des Gesangssignals und fügt auch tonales Material an den Sibilantenstellen hinzu, wo vorher keins war.
- **Akustische Gitarre:** Um stichprobenartig auch andere Anwendungsgebiete der Algorithmen zu betrachten, wird eine Akustikgitarrenaufnahme mit prominenten Saitenrutschern als weiteres Szenario mitgetestet.

Mit diesen Szenarien werden nun alle drei Algorithmen in zwei Phasen getestet. In der ersten Phase wird mit den zwei unbearbeiteten Gesangsspuren experimentiert, um alle grundlegenden Beobachtungen und Vergleiche aufzustellen. Anhand dieser Auswertungen werden die in Tabelle 3.1 aufgestellten vier Kriterien bewertet und die technischen Bedingungen wie die Artefaktfreiheit bewertet und die Nulltests durchgeführt. In der zweiten Phase werden die Algorithmen mithilfe der zusätzlichen Szenarien (Tonal -6 dB, Sibilant -6 dB, etc.) weiteren Tests unterzogen und ihr Verhalten in den Randbereichen möglicher Audioinputs beobachtet. Hierbei wird vor allem die Robustheit der Erkennung und Trennung beobachtet, um die Algorithmen auf die Bandbreite möglicher Praxisszenarien zu testen.

Neben den Hörtests kommen zur Auswertung und Analyse der zu untersuchenden Kriterien verschiedene Vorgehensweisen zum Einsatz:

- Der Vergleich der angezeigten Waveform der Dateien in einer DAW.
- Die Analyse der Spuren durch das Spektrogramm in iZotope RX.
- Ein Nulltest, bei dem die verarbeitete Spur mit dem phasengedrehten Original summiert wird, um eventuelle Reste oder Phasendifferenzen zu identifizieren

5.2 Parameter

In allen Tests und Szenarien wurden die Parameter der Algorithmen so gewählt, dass sie bestmöglich arbeiten können. Kriterium hierfür war der sichtbare Einschluss möglichst großer Sibilantenanteile, während möglichst wenig tonales Material als Fehlklassifizierung übrig blieb. Die Einstellungen wurden anhand von AB-Tests und der in der Anwendung

abgebildeten Waveform-Visualisierung mit Sibilanten-Overlay ausgewählt. Manchmal wurden in den Phase-2-Tests bewusst Abstriche zugunsten einer geringeren Fehl Kennzeichnung des tonalen Materials vorgenommen. Wie in den Beobachtungstabellen des Anhangs zu sehen, sind folgende Erkenntnisse bei der Wahl der Parameter entstanden:

- **Cutoff-Frequenz:** Konträr zu der anfänglichen Vermutung hat sowohl in der männlichen als auch in der weiblichen Stimme eine Cutoff-Frequenz von 4kHz zu den besten Ergebnissen geführt. Einzig und allein hat der Randfall der Akustikgitarre mit dieser Cutoff-Frequenz nicht gut funktioniert, wobei 1kHz hier die beste Wahl war. Eine weitere Erkenntnis war ebenfalls, dass dies für alle drei Algorithmen gleichermaßen der Fall war und auch der FFT-basierte Algorithmus mit der gleichen Cutoff-Frequenz am besten funktioniert hat.
- **Threshold:** Wo schon ein höherer Threshold im Sample-based Algorithmus zu vermehrten Fehlerkennungen des tonalen Signals geführt hat, konnte im RMS basierten Algorithmus der Threshold deutlich tiefer (grob -10dB mehr) gesetzt werden.
- **FFT-Size:** Die FFT-Size hat sowohl auf Aliasingartefakte als auch die Rückstände im Nulltest Auswirkungen, sobald diese zu gering gewählt wird. Der Wert von 1024 Samples hat hier wiederholbar robuste Ergebnisse geliefert.
- **FFT-Ratio:** Das Ratio hat in den meisten Fällen keinen großen Unterschied gemacht. Werte von 0,3 (30%) bis hin zu 0,7 (70%) haben verlässlich robuste Ergebnisse geliefert. Eine Reduktion dieses Wertes war nur in den Fällen mit Musik nötig, da hier ein größeres Verhältnis von tonalem Material vorhanden ist. Mit Werten von 0,0 (0%) bis 0,2 (20%) konnten diese Fälle abgedeckt werden.
- **RMS dB Hysteresis:** Die Einstellung von grob 6dB Hysteresis hat in fast allen Tests gute Ergebnisse geliefert. Der einzig kritische Fall war die Gesangsspuren mit -6 dB abgesenkten Sibilanten. In diesem Fall hat auch ein geringer Hysteresis Wert zu erhöhter Fehldetektion tonalen Materials geführt.

5.3 Testergebnisse

5.3.1 Phase 1: Baseline Tests

Für die Grundlagen-Tests der Algorithmen auf die vier Testkriterien werden in dieser Testphase 1 die beiden unbearbeiteten Gesangsspuren (männlich & weiblich) gegeneinander getestet. Die Erkenntnisse aus den Beobachtungen, wie in Tabelle A.1 und Tabelle A.2 im Anhang zu erkennen sind, sind im Folgenden für jedes Kriterium getrennt beschrieben.

Kriterium 1: Sound / Hörbare Verbesserung

Manuelle Trennung

- Subjektiv sehr gute und natürlich klingende Ergebnisse der Sibilanten-Reduktion.
- Einige Übergänge sind durchaus hörbar, da der starke Pegelabfall ohne Crossfade durchaus hörbar ist.

Sample-based

- Hörbar reduzierte Sibilanten, aber sehr unnatürlicher Klang.
- Häufig hörbares Stottern an Stellen, an denen nur Teile der Sibilanten erkannt wurden.
- Hörbare Klicks an manchen Übergängen zwischen als Sibilant erkanntem und als tonal erkanntem Material.

RMS-based

- Gute und natürlich klingende Ergebnisse und Absenkung der Sibilanten.
- Hörbare Sprünge in den Anfangsteilen einiger Sibilanten als Folge von teilweise nicht erkannten Anfängen.
- Keine hörbaren Klick- und Popgeräusche an den Übergängen.

FFT-based

- Klanglich sehr sauber und natürlich mit der von allen Beispielen am besten klingenden Verbesserung.
- Sehr unauffällige Übergänge zwischen tonalem und sibilantem Material ohne Stottern oder hörbarem Pegelabfall.

Kriterium 2: Erkennung

Manuelle Trennung

- Sehr gute Erkennung der stark herausstechenden Sibilanten (ermöglicht durch die Waveformanzeige in der DAW, wie in Abbildung 5.1 zu erkennen).
- Schlechte Erkennung des genauen Übergangs, wann der Sibilant anfängt.
- Schlechte Erkennung der nicht sehr präsenten Level-2-Sibilanten (T, F).

Sample-based

- Tendiert zu Über- oder Untererkennung, je nach eingestelltem Threshold.
- Meist wird nur ein Teilstück des Sibilanten oder sogar mehrere Teilstücke desselben Sibilanten erkannt (siehe Abbildung 5.1).
- Bei zu geringem Threshold wird zwar mehr Anteil der Sibilanten erkannt, allerdings auch viel tonaler Anteil.
- Die erkannten Sibilanten sind fast ausschließlich die, die stark aus dem Material hervorstechen.

RMS-based

- Sehr gute Erkennung von Level-1-Sibilanten (S, Sch).
- Schlechte Erkennung des korrekten Startpunktes der Sibilanten (siehe Abbildung 5.1).
- Gute Erkennung des exakten Endpunktes der Sibilanten, durch den durch Hysteresis erweiterten Threshold.

FFT-based

- Sehr gute und lückenlose Erkennung beider Sibilanten-Level bis hin zum kleinsten als Sibilant klassifizierbaren Anteil.

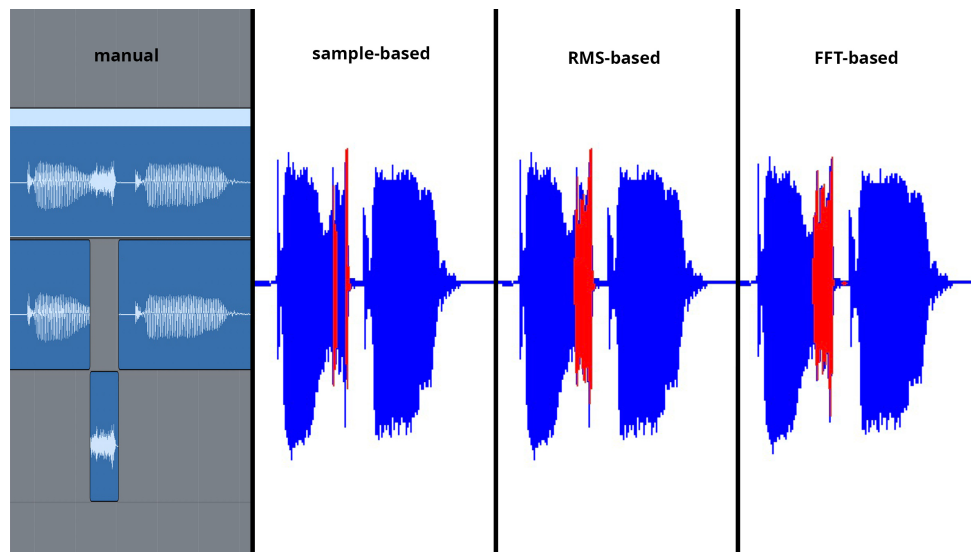


Abbildung 5.1: Vergleich der Erkennung anhand der Waveform der DAW und Beispielapplikation. Quelle: Eigene Auswertung.

Kriterium 3: Trennung

Manuelle Trennung

- Die Trennung bezieht sich auf das gesamte Frequenzspektrum (wie in Abbildung 5.2 zu erkennen).
- Sehr gute und präzise Trennung bis auf einzelne Stellen, an denen die Schnitte nicht perfekt gesetzt wurden.

Sample-based

- Harte zeitliche Ein-/Ausschaltunkte einzelner Samples bzw. Samplegruppen (durch Hysteresis) im gesamten Frequenzbereich, was zu sehr unnatürlichen und abgehackten Ergebnissen führt.
- Abrupte Trennung häufig auch innerhalb von Sibilanten.

RMS-based

- Weiche Übergänge ohne Klick- und Popgeräusche.
- Natürlicher Frequenzübergang am Crossoverpunkt, obwohl im Spektrogramm sichtbar ist, dass erstaunlich viel Höhenanteil im tonalen Signal an den Sibilantenstellen übrig bleibt (siehe Abbildung 5.2).

FFT-based

- Präzise und kurze, aber dennoch klick- und popgeräuschfreie Übergänge an den zeitlichen Grenzen der Sibilanten.
- Deutlicher Schnitt des Frequenzbereichs an der Crossoverfrequenz mit durch den Crossfade minimalem, aber nicht vollständig eliminiertem Aliasing im genullten Bereich (siehe Abbildung 5.2).

Kriterium 4: Artefakte im tonalen Signal

Manuelle Trennung

- Keine Beeinflussung des tonalen Materials.
- Im Nulltest können Klicks an den Übergängen erkannt werden, falls nicht samplegenau editiert wurde (zu erkennen in Abbildung 5.3).

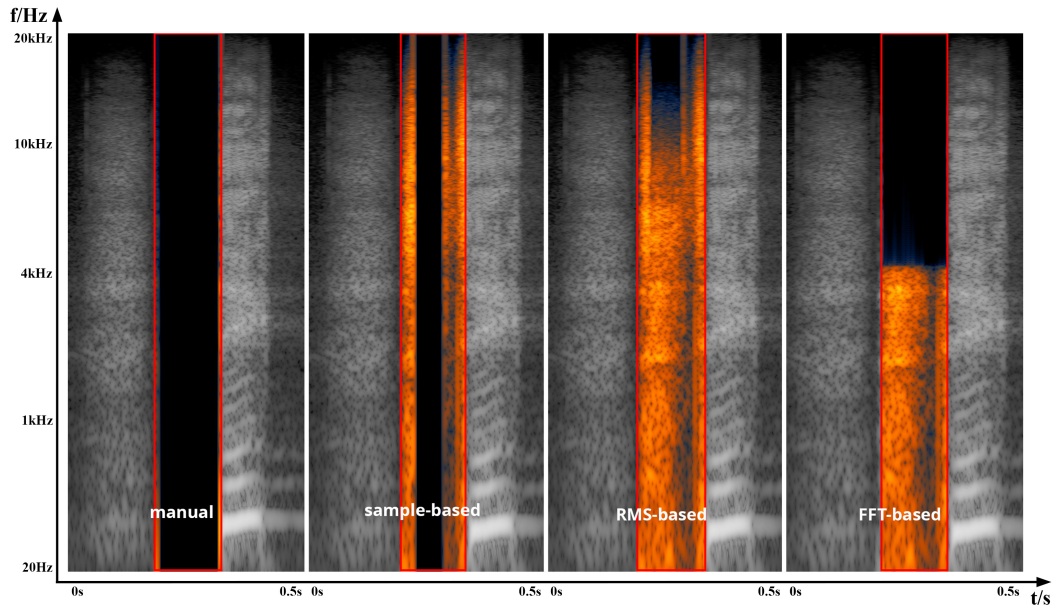


Abbildung 5.2: Vergleich der Trennung anhand des Spektrogramms. Quelle: Eigene Auswertung.

Sample-based

- Keine hörbare oder im Spektrogramm erkennbare Veränderung im Frequenzgang.
- Der Nulltest fällt unzureichend aus. Beim Vergleich mit dem Original verbleibt ein nahezu vollständiges Signal mit geändertem Frequenzgang (siehe Abbildung 5.3). Im Spektrogramm und Höreindruck zeigen sich verschobene Frequenzamplituden. Dies ist auf die Phasendrehung der Linkwitz-Riley Filter zurückzuführen, die auch als Allpass-Filter die Phase des Signals drehen.

RMS-based

- Wie beim Sample-based-Algorithmus keine auffällige Veränderung im Frequenzgang, wenn das Signal allein betrachtet wird.
- Der Nulltest fällt ebenfalls unzureichend aus, mit ähnlichen, durch die Linkwitz-Riley Filterung ausgelösten Effekten. Allerdings entsteht hier eher eine Höhenbetonung.

FFT-based

- Keine hörbare oder im Spektrogramm erkennbare Veränderung im Frequenzgang.
- Der Nulltest zeigt minimale Rückstände über das gesamte Signal, die auf die Ungenauigkeit der inversen FFT zurückzuführen sind (siehe Abbildung 5.3).

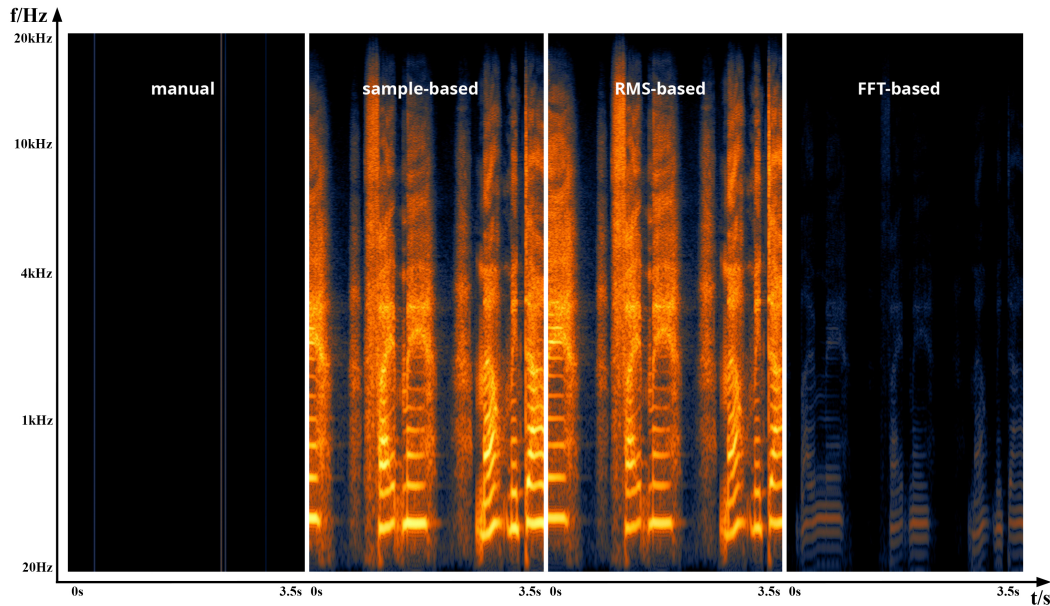


Abbildung 5.3: Vergleich der Nulltests im Spektrogramm. Quelle: Eigene Auswertung.

5.3.2 Phase 2: Test der Randfälle & Akustikgitarre

Fall Tonal -6 dB Bei künstlich überbetonten Sibilanten (im Vergleich zum Originalsignal) profitieren sowohl der Sample- als auch der RMS-basierte Algorithmus stark. Da diese nun stärker hervorstechen, wird mehr von ihnen erkannt, was zu weniger Stottern im sample-basierten Algorithmus führt und zu einer besseren Erkennung der Startpunkte im RMS-basierten Algorithmus. Der FFT-basierte Algorithmus funktioniert weiter wie gehabt, ohne merkliche Veränderung.

Fall Sibilanten -6 dB Bei künstlich unterbetonten Sibilanten fallen sowohl der Sample- als auch der RMS-basierte Algorithmus deutlich ab. Bei beiden wird viel tonales Material mit erkannt, sofern der Threshold so eingestellt wird, dass die leisen Sibilanten überhaupt noch erfasst werden. Der Sample-basierte Algorithmus stottert unkontrollierbar an vielen Stellen und der RMS-basierte Algorithmus erkennt nur noch die Spitzen der lautesten Sibilanten. Die Hysteresis-Einstellung des RMS-Algorithmus ist hier gänzlich unbrauchbar, da schon bei geringsten Werten viel tonales Material eingeschlossen wird. Der FFT-basierte Algorithmus funktioniert weiterhin problemlos ohne hörbare Veränderung.

Fall mit Musik Bei zusätzlichem tonalen Material unter dem gesamten Signal funktionieren der Sample- und der RMS-basierte Algorithmus weitgehend unverändert zum regulären Material. Hier erlebt jedoch der FFT-basierte Algorithmus leichte Einbußen: Die Erkennung funktioniert mit den bisherigen Ratio-Werten (Verhältnissen) nicht mehr

zuverlässig. Dieses Problem lässt sich allerdings durch eine Reduktion des Ratios beheben, nach der der Algorithmus wieder mit geringen Einbußen wie zuvor arbeitet.

Fall Akustikgitarre Der FFT-basierte Algorithmus funktioniert hier erstaunlich gut. Nach Einstellung der Cutoff-Werte auf eine niedrigere Frequenz von 1 kHz statt der vorherigen 4 kHz, dem Optimalwert für die Erkennung von Sibilanten, erkennt dieser die meisten Rutscher. Dies ist nicht verwunderlich nach dem schon in der Abbildung 2.1 erkennbaren weniger konzentriert hochfrequenten Frequenzgang der Rutscher im Akustikgitarrensinal. Negativ zu betrachten ist allerdings, dass neben den Rutschern auch vereinzelt besonders scharf gezupfte Töne erkannt werden. Auch wenn dies in einer reinen Amplitudenabsenkung des artefakthaltigen Signalanteils zu einer insgesamt guten klanglichen Verbesserung führt, ist dies für die reine Trennung der Rutscher negativ zu betrachten. Im Gegensatz zum FFT-basierten Algorithmus sind die Sample- und RMS-basierten Algorithmen hier leider kaum brauchbar, da das Frequenzprofil der Akustikgitarre zu anders als die Gesangssignale ist und der Threshold die Störgeräusche nicht erkennt.

Kapitel 6: Fazit

Basierend auf der Problemstellung wurden drei Algorithmen unter Verwendung von frequenzschwellenbasierten und spektralen Ansätzen implementiert und getestet. Das Ziel war es, ein automatisiertes Verfahren zur Erkennung und Trennung von Sibilanten in Gesangsaufnahmen sowie ggf. anderen Artefakten zu entwickeln, um damit die Schwächen aktueller Tools in Bezug auf Präzision und Kontrolle zu überwinden und ein verbessertes Klangergebnis zu erzielen. Unter Berücksichtigung der Anforderungen an eine fehlerfreie Erkennung und nebenproduktfreie Trennung der Sibilanten, ohne das tonale Material dabei zu beeinträchtigen, wurden die Verfahren konzeptioniert und mit der Hilfe von JUCE in einer Beispielapplikation implementiert. Mit selbst erstellten und kontrollierten Testdaten wurden die Algorithmen in zwei Phasen getestet. Durch die Untersuchung der 4 Kriterien aus Tabelle 3.1 (Klang, Erkennung, Trennung und Artefakte) konnten die Algorithmen in der ersten Phase miteinander verglichen werden, um anhand von Stresstests in der zweiten Phase das Verhalten in Randfällen zu beobachten, wodurch Schlüsse auf die Robustheit erkennbar wurden. Im Folgenden werden die Verfahren anhand der Testergebnisse und Beobachtungen bewertet und ein Ausblick auf ihre Praxistauglichkeit gegeben.

6.1 Bewertung der Algorithmen

Inwiefern ist die Funktionsweise der drei Algorithmen zielführend und welche Vor- und Nachteile haben sie?

Wie schon in der Konzeption angemerkt, ist der Sample-basierte Algorithmus nicht sinnvoll für den Anwendungszweck. Die Verwendung eines großen Hysteresis-Werts macht ihn zwar bedingt brauchbar, allerdings ist auch dann die Erkennung immer noch unsauber und die Trennung entsprechend artefaktbelastet.

Da auch die manuelle Trennung in den Tests nicht komplett artefaktfrei ist und im Zeit- und Arbeitsaufwand enorm hoch liegt, bietet sich der Vergleich der RMS- und FFT-basierten Algorithmen für eine finale Schlussfolgerung an.

Welche Vorteile und Nachteile hat der RMS-basierte Algorithmus? Auch wenn der Nulltest unzureichend ausgefallen ist, weist dies nach technischer Betrachtung des Linkwitz-Riley-Filters nicht auf eine klangliche Beeinträchtigung hin. Der Frequenzgang und das Signal bleiben ohne hörbaren Verlust bestehen, da nur die Phase etwas gedreht wurde. Das ist in der Praxis meist unproblematisch. Da die Trennung in den Tests sehr positiv ausfällt, ist

dieser Algorithmus hauptsächlich hinsichtlich seiner Erkennung und des damit verbundenen Höreindrucks zu kritisieren. Wenn das Ziel ausschließlich die Reduzierung störender Sibilanten ist, ist der RMS-basierte Ansatz eine gute Wahl. Je stärker die Sibilanten hervorstechen, desto besser werden sie erkannt. Auch wenn bei einer Reduktion um 9 dB die nicht erkannten Anfänge hörbar störend sein können, ist eine geringere Absenkung (z. B. nur wenige dB) durchaus praxistauglich. Eine wirkliche Verbesserung gegenüber regulären De-Essern bietet dies jedoch nicht, da diese ähnlich arbeiten. Da die Sibilanten meist nicht vollständig erkannt werden, ist die Trennung auf eine separate Spur nur bedingt nutzbar, da im tonalen Signal noch Rückstände vorhanden sind.

Wie steht dem der FFT-basierte Algorithmus gegenüber? Seine größte Stärke ist die Erkennung: Es werden nicht nur stark hervorstechende Sibilanten erkannt, sondern auch wenig hervorstechende Level-2 Sibilanten. Mit der richtigen Ratio-Einstellung bleibt er frei von Fehlkennungen tonalen Materials oder anderen nicht tonalen Signalanteilen wie z. B. Atmern. Damit ist der FFT-basierte Ansatz den anderen deutlich überlegen. Auch in den Stresstests und bei der Anwendung auf Akustikgitarrensignale liefert der Algorithmus konstante und sehr gute Ergebnisse. Allerdings kommt er nicht ohne Abstriche aus. Das Problem liegt in der Natur der Fast Fourier Transformation und ihrer unvermeidlichen Ungenauigkeit bei der Rückrechnung. Trotz Parameteroptimierung gehen immer Teile der Zeit- und Frequenzinformationen verloren. Dies ist minimierbar, aber nicht eliminierbar und zeigt sich im Nulltest als leichtes Rauschen und Phasenreste. Es entsteht also ein unvermeidlicher Signalverlust gegenüber dem Original. In den Hörtests (auch auf professionellem Equipment bei normalen Lautstärken) ist dieser Rest aber kaum wahrnehmbar.

Ein weiteres Problem ist das Aliasing an der Übergangsfrequenz der Sibilantstellen, sowohl in der Sibilantenspur als auch in der tonalen Spur. Durch den frequenzbasierten Crossfade in der Implementierung wird dies im rückkombinierten Signal zwar stark reduziert, aber nicht vollständig verhindert. Da sich dies ausschließlich auf die Sibilanten auswirkt, ist durch deren rauschähnlichen Frequenzgang der Einfluss auf die Klangqualität für die Praxis durchaus vernachlässigbar. Diese Problematik ist in dem Fall besonders abhängig von der nachträglichen getrennten Bearbeitung beider Signalanteile.

Trotzdem sollte man diesen minimalen Signalverlust nicht außer Acht lassen, auch wenn die gewonnene Kontrolle durch die hervorragende Trennung und die mögliche klangliche Verbesserung in einer praktischen Anwendung deutlich überwiegt.

6.2 Einschätzung der Problemlösung

Die implementierten Algorithmen zur Entwicklung einer Methode zur automatisierten Erkennung und Trennung von Sibilanten und gegebenenfalls anderen Artefakten in Musikaufnahmen haben mit einigen wenigen Einschränkungen das vorangegangene Problem gut gelöst. Besonders der FFT-basierte Ansatz über den Vergleich der Verhältnisse des hochfrequenten Signalanteils zum Gesamtsignal ist in den durchgeführten Tests nicht nur

robust, sondern übertrifft auch die Verlässlichkeit der Artefakttrennung mittels manuellem Ausschneiden. Somit ist das Ziel der verlässlichen Automatisierung gelöst.

Auch die Trennung und Filterung der entwickelten Ansätze hat sich als funktional und praktikabel erwiesen. Die größten Probleme hierbei waren die Auswirkungen der verwendeten Filter oder Trennungsansätze mittels FFT auf das Signal und damit auch auf den Nulltest. Hier liegt der größte Spielraum für Verbesserungen. Die Entkopplung der Filterung von der Erkennung könnte hier ein Ansatz für die Zukunft sein. Mit dieser Entkopplung könnte z. B. die FFT-basierte Erkennung mit einer Trennung per Linkwitz-Riley-Filter kombiniert werden oder für einen absolut perfekten Nulltest mit einer Trennung des Gesamtsignals ohne Phasendrehung durch etwaige Filterung. Auch vorstellbar wäre der Vergleich der Verhältnisse per RMS-Fenster gefilterter Signale statt per FFT, was zu einer deutlich effizienteren Anwendung führen könnte.

6.3 Ausblick

Dem herkömmlichen De-Esser oder den manuellen Methoden ist der hier vorgestellte verhältnisbasierte Ansatz hinsichtlich Automatisierung und Präzision weit überlegen und hat durchaus Relevanz, um kommerziell im Markt eingesetzt zu werden. Spannend wäre für den Einsatz in kommerzieller Software eine eventuelle Echtzeittauglichkeit, aus der sich Möglichkeiten für den Einsatz in regulären Audioplugins ergeben würden, außerhalb der Verwendung als Standalone-Applikation oder ARA-Plugin. Der einzige kritische Punkt hierfür wäre die Crossfade-Logik des RMS-basierten Algorithmus. Alle anderen Ansätze ließen sich in Echtzeit anwenden, da sie nur das aktuelle Fenster an Samples betrachten und nicht im Signal hin und her springen.

Der größte offene Punkt für die Beurteilung der Algorithmen ist die Ausweitung des Anwendungsgebietes auf andere Quellen abseits von Gesangssignalen und den darin enthaltenen Sibilanten. Der Ausblick in Testphase 2 für die Verwendung mit Akustikgitarrensensignalen hat zwar Aufschlüsse für die Abstrahierung der vorhandenen Algorithmen gegeben, allerdings lässt sich dadurch nicht auf die mögliche kategorische Verwendung auf alle Akustikgitarrensensignale oder auf andere mit ähnlichen Artefakten behaftete Signale schließen. Um dies zu gewährleisten, müsste ein deutlich mathematischerer Ansatz der Bewertung und Kategorisierung von Artefakten entwickelt werden, um einen robusten und abstrahierbaren Algorithmus zu entwickeln. Dafür könnten zum Beispiel Datenbanken mit Testsignalen manuell getrennt werden, und diese Signale dann genau auf Frequenzverhältnisse untersucht werden, um exakte mathematische Verhältnisse zur Erkennung dieser quellenspezifischen Artefakte sicherzustellen. Unter Beachtung dieser Daten kann der Algorithmus dann an die Kategorie der Signale angepasst werden. Um hier besonders präzise zu arbeiten, könnten neben Frequenzschwellenwerten und Verhältnissen auch die Betrachtung von Hüllkurven oder anderen Parametern in Kombination genutzt werden, um weitgehend feiner einstellbare Algorithmen zu entwickeln.

Um die Praxistauglichkeit noch verlässlicher zu bewerten, fehlen in manchen Bereichen mathematischere Tests bzw. Vergleiche mit den gängigen Konkurrenzprogrammen zur Beurteilung der Ergebnisse. Um die Aussagekraft über die Relevanz von Rückständen der FFT oder Phasenverschiebungen durch Linkwitz-Riley-Filter zu verstärken, könnten die gängigsten Programme durch Nulltests bewertet und gegenübergestellt und detailliertere Tests zur Bewertung der Klangveränderung genutzt werden.

Ein spannender Ausblick wäre letztlich auch der Vergleich der FFT mit Ansätzen wie der Wavelet-Analyse. Die Implementierung eines Wavelet-basierten Ansatzes wäre besonders im Bezug auf die Nulltest-Rückstände und der daraus resultierenden Klangqualität interessant.

Anhang A: Anhang

A.1 Testbeobachtungen

Test ID	Voice	Scenario	Algorithm	Cutoff (Hz)	Threshold (dB)	Ratio	FFT Size	Overlap/Hyst/Crossfade	Reduction (dB)	Nulltests
0	male	full	manual	N/A	N/A	N/A	N/A	N/A	-9	Hörbare und im Spektrogramm erkennbare Klicks an den Übergängen, wo die Schnitte nicht perfekt gesetzt wurden.
1	female	full	manual	N/A	N/A	N/A	N/A	N/A	-9	Hörbare und im Spektrogramm erkennbare Klicks an den Übergängen, wo die Schnitte nicht perfekt gesetzt wurden.
2	male	full	sample	4000	-11	N/A	N/A	500 samples hyst	-9	Nulltest mit dem Original: Fällt durch. Gibt ein fast vollständiges Signal wieder mit leicht geänderter Frequenzwidrigabe. Nulltest mit eigenem gefilterten aber nicht bearbeiteten Buffer (ohne Sibilanttrennung und Erkennung): Nulltest perfekt.
3	male	full	RMS	4000	-20	N/A	N/A	6 dB hysteresis	-9	Nulltest mit dem Original: Fällt durch. Gibt ein fast vollständiges Signal wieder mit leicht geänderter Frequenzwidrigabe. Nulltest mit eigenem gefilterten aber nicht bearbeiteten Buffer (ohne Sibilanttrennung und Erkennung): Nulltest perfekt.
4	male	full	FFT	4000	N/A	0,4	1024	500 Hz crossfade	-9	Nulltest mit dem Original: Nullt nicht komplett. Bei normalen Lautstärken fast unhörbar, aber auf dem Spektrogramm sichtbar und bei erhöhter Lautstärke klar hörbares zitterige Rückstände über den kompletten Frequenzbereich. Über das komplette Signal vorzufinden.
5	female	full	sample	4000	-10	N/A	N/A	500 samples hyst	-9	Nulltest mit dem Original: Fällt durch. Gibt ein fast vollständiges Signal wieder mit leicht geänderter Frequenzwidrigabe. Nulltest mit eigenem gefilterten aber nicht bearbeiteten Buffer (ohne Sibilanttrennung und Erkennung): Nulltest perfekt.
6	female	full	RMS	4000	-24	N/A	N/A	6 dB hysteresis	-9	Nulltest mit dem Original: Fällt durch. Gibt ein fast vollständiges Signal wieder mit leicht geänderter Frequenzwidrigabe. Nulltest mit eigenem gefilterten aber nicht bearbeiteten Buffer (ohne Sibilanttrennung und Erkennung): Nulltest perfekt.
7	female	full	FFT	4000	N/A	0,5	1024	500 Hz crossfade	-9	Nulltest mit dem Original: Nullt nicht komplett. Bei normalen Lautstärken fast unhörbar, aber auf dem Spektrogramm sichtbar und bei erhöhter Lautstärke klar hörbares zitterige Rückstände über den kompletten Frequenzbereich. Über das komplette Signal vorzufinden.

Abbildung A.1: Beobachtungen Tests: Phase 1 (1)

Test ID	Criterion 1 (Sound)	Criterion 2 (Detection)	Criterion 3 (Separation)	Criterion 4 (Tonal Signal Artifacts)
0	Hörbare & erkennbare Verbesserung. Übergänge nahezu unhörbar.	Erkennung von Level 1 Sibilanten sehr gut. Erkennung von Level 2 Sibilanten schlecht.	Trennung gut, allerdings Frequenz unspezifisch. Übergang teils unklar in der Waveform.	Keine vorhanden.
1	Hörbare & erkennbare Verbesserung. Übergänge nahezu unhörbar.	Erkennung von Level 1 Sibilanten sehr gut. Erkennung von Level 2 Sibilanten schlecht.	Trennung gut, allerdings Frequenz unspezifisch. Übergang teils unklar in der Waveform.	Keine vorhanden.
2	Deutliche Brüche und Stottern um die Sibilanten. Übergänge und Fehlerkennungen / Teilerkennungen hörbar.	Erkennung von Level 1 Sibilanten schlecht (Teilerkennung, Stottern, Fehlerkennungen). Erkennung von Level 2 Sibilanten schlecht.	Trennung schlecht da Rückstände von unerkannten Teilen der Sibilanten. Direkte Brüche führen zu Klicks.	Keine hörbare Verschlechterung. Trotz Linkwitz-Riley-Filter Verschiebung der Phase im tonalen Output.
3	Keine Klicks und natürlich klingende Übergänge an der Absenkung. Teilweise hörbar, dass Sibilantenteile nicht erkannt wurden.	Erkennung von Level 1 Sibilanten gut (vermehrt aber fehlende Erkennung vom Start der Sibilanten). Erkennung von Level 2 Sibilanten schlecht.	Trennung generell gut, da Crossfade und Blockerkennung durch Hysteresis natürliche Ergebnisse hervorbringen.	Keine hörbare Verschlechterung. Trotz Linkwitz-Riley-Filter Verschiebung der Phase im tonalen Output.
4	Natürlicher Klang an den Übergängen und in der Absenkung. Leichte Resonanz (tonales Fiepen) an der Übergangsfrequenz während der Sibilanten.	Erkennung von Level 1 Sibilanten sehr gut. Erkennung von Level 2 Sibilanten sehr gut.	Trennung sehr gut.	Aliasing an den Übergängen. Ungenaue Rücktransformation mit leichten Änderungen und Nulltest Rückständen.
5	Deutliche Brüche und Stottern um die Sibilanten. Übergänge und Fehlerkennungen / Teilerkennungen hörbar.	Erkennung von Level 1 Sibilanten schlecht (Teilerkennung, Stottern, Fehlerkennungen). Erkennung von Level 2 Sibilanten schlecht.	Trennung schlecht da Rückstände von unerkannten Teilen der Sibilanten. Direkte Brüche führen zu Klicks.	Keine hörbare Verschlechterung. Trotz Linkwitz-Riley-Filter Verschiebung der Phase im tonalen Output.
6	Keine Klicks und natürlich klingende Übergänge an der Absenkung. Teilweise hörbar, dass Sibilantenteile nicht erkannt wurden. Rekonstruktion ohne hörbare Arefakte.	Erkennung von Level 1 Sibilanten gut (vermehrt aber fehlende Erkennung vom Start der Sibilanten). Erkennung von Level 2 Sibilanten schlecht.	Trennung generell gut, da Crossfade und Blockerkennung durch Hysteresis natürliche Ergebnisse hervorbringen.	Keine hörbare Verschlechterung. Trotz Linkwitz-Riley-Filter Verschiebung der Phase im tonalen Output.
7	Natürlicher Klang an den Übergängen und in der Absenkung. Leichte Resonanz (tonales Fiepen) an der Übergangsfrequenz während der Sibilanten.	Erkennung von Level 1 Sibilanten sehr gut. Erkennung von Level 2 Sibilanten sehr gut.	Trennung sehr gut.	Aliasing an den Übergängen. Ungenaue Rücktransformation mit leichten Änderungen und Nulltest Rückständen.

Abbildung A.2: Beobachtungen Tests: Phase 1 (2)

A Anhang

Test ID	Voice	Scenario	Algorithm	Cutoff (Hz)	Threshold (dB)	Ratio	FFT Size	Overlap/Hyst/Crossfade	Reduction (dB)	Notes
8	female	full	sample	6500	-17	N/A	N/A	500 samples hyst	-9	Erkennung schlechter, da Teil der Energie der Sibilanten nicht mehr in dem gefilterten Frequenzbereich. Deutlich mehr Stottern und Erkennung von tonalem Material.
9	female	full	RMS	4000	-29	N/A	N/A	2 dB hysteresis	-9	Unter dem Versuch die kompletten Sibilanten hier zu erkennen, wird selbst mit minimaler Hysteresis viel tonales Material erkannt. Höhere Hysteresis verschlechtert dies enorm.
10	female	full	FFT	4000	N/A	0,5	1024	0 Hz crossfade	-9	Ohne Crossfade deutlich mehr Aliasing an der Übergangsfrequenz.
11	female	full	FFT	4000	N/A	0,5	1024	1000 Hz crossfade	-9	Aliasing an der Übergangsfrequenz nicht merklich reduzierter als in der 500Hz FFT (Test 7) Version.
12	female	full	sample	4000	-16	N/A	N/A	1 samples hyst	-9	Ohne Hysteresis ist der Sample-based Algorithmus komplett unbrauchbar. Es gibt super viele "An-Aus" Erkennung in Form von hörbarem Stottern und partialer Erkennung und Filterung.
13	male	ton -6	sample	4000	-13	N/A	N/A	500 samples hyst	-9	Deutlich verbesserte Erkennung, da die Sibilanten jetzt viel mehr raus stechen und auch an den "leiseren" Teilen der Sibilanten über dem tonalen Material stehen. Erkennung trotzdem nicht perfekt an den Rändern der Sibilanten.
14	male	ton -6	RMS	4000	-26	N/A	N/A	6 dB hysteresis	-9	Extrem verbesserte Erkennung. Sogar für Level 2 Sibilanten. Immernoch teilweise unerkannte Startanteile von Sibilanten.
15	male	ton -6	FFT	4000	N/A	0,8	1024	500 Hz crossfade	-9	Keine oder wenn nur absolut minimal hörbare bzw. erkennbare Verbesserung der Erkennung (die Verhältnisse in der FFT Size haben sich ja nicht verändert, nur die Gesamtamplitude).
16	female	ton -6	sample	4000	-16	N/A	N/A	500 samples hyst	-9	Bedingt verbesserte Erkennung. Große Teile immernoch nicht erkannt. Reduziertes Stottern.
17	female	ton -6	RMS	4000	-29	N/A	N/A	6 dB hysteresis	-9	Verbesserte Erkennung sogar für Level 2 Sibilanten. Immernoch teilweise unerkannte Startanteile von Sibilanten.
18	female	ton -6	FFT	4000	N/A	0,5	1024	500 Hz crossfade	-9	Keine oder wenn nur absolut minimal hörbare bzw. erkennbare Verbesserung der Erkennung (die Verhältnisse in der FFT Size haben sich ja nicht verändert, nur die Gesamtamplitude).

Abbildung A.3: Beobachtungen Tests: Phase 2 (1)

A Anhang

Test ID	Voice	Scenario	Algorithm	Cutoff (Hz)	Threshold (dB)	Ratio	FFT Size	Overlap/Hyst/Crossfade	Reduction (dB)	Notes
19	male	sib -6	sample	4000	-17	N/A	N/A	500 samples hyst	-9	Extrem schlechte Erkennung! Große Teile von tonalem Material erkannt und viele stotter Artefakte.
20	male	sib -6	RMS	4000	-26	N/A	N/A	6 dB hysteresis	-9	Keine extreme, aber trotzdem schlechtere Erkennung. Tonale Anteile werden jetzt fast unvermeidlich miterkannt.
21	male	sib -6	FFT	4000	N/A	0,4	1024	500 Hz crossfade	-9	Keine merkliche Verschlechterung im Algorithmus. Keine merklichen hinzugekommenen Artefakte.
22	female	sib -6	sample	4000	-19	N/A	N/A	500 samples hyst	-9	Extrem schlechte Erkennung! Große Teile von tonalem Material erkannt und viele stotter Artefakte.
23	female	sib -6	RMS	4000	-30	N/A	N/A	6 dB hysteresis	-9	Deutlich schlechtere Erkennung. Tonale Anteile werden jetzt unvermeidlich miterkannt.
24	female	sib -6	FFT	4000	N/A	0,5	1024	500 Hz crossfade	-9	Keine merkliche Verschlechterung im Algorithmus. Keine merklichen hinzugekommenen Artefakte.
25	male	w/ music	sample	4000	-11	N/A	N/A	500 samples hyst	-9	Keine Verschlechterung im Vergleich zum nicht-Musik Signal.
26	male	w/ music	RMS	4000	-20	N/A	N/A	6 dB hysteresis	-9	Keine Verschlechterung im Vergleich zum nicht-Musik Signal.
27	male	w/ music	FFT	4000	N/A	0,2	1024	500 Hz crossfade	-9	Mit Anpassung des Ratios funktioniert der Algorithmus weiterhin mit nur minimaler Verschlechterung. Erstaunlicherweise ist das tonal hinzugefügte Musiksinal nicht laut genug um den Algorithmus irreparabel unbrauchbar zu machen.
28	female	w/ music	sample	4000	-10	N/A	N/A	500 samples hyst	-9	Keine Verschlechterung im Vergleich zum nicht-Musik Signal.
29	female	w/ music	RMS	4000	-24	N/A	N/A	6 dB hysteresis	-9	Keine Verschlechterung im Vergleich zum nicht-Musik Signal.
30	female	w/ music	FFT	4000	N/A	0,05	1024	500 Hz crossfade	-9	Mit Anpassung des Ratios funktioniert der Algorithmus weiterhin mit nur minimaler Verschlechterung. Erstaunlicherweise ist das tonal hinzugefügte Musiksinal nicht laut genug um den Algorithmus irreparabel unbrauchbar zu machen.

Abbildung A.4: Beobachtungen Tests: Phase 2 (2)

Test ID	Voice	Scenario	Algorithm	Cutoff (Hz)	Threshold (dB)	Ratio	FFT Size	Overlap/Hyst/Crossfade	Reduction (dB)
31	acgit	full	sample	1000	-15	N/A	N/A	500 samples hyst	-9
32	acgit	full	RMS	1000	-26	N/A	N/A	6 dB hysteresis	-9
33	acgit	full	FFT	1000	N/A	0.3	1024	500 Hz crossfade	-9

Abbildung A.5: Beobachtungen Tests: Phase 2 (3) - Fall Akustikgitarre

Test ID	Criterion 1 (Sound)	Criterion 2 (Detection)	Criterion 3 (Separation)	Criterion 4 (Tonal Signal Artifacts)
31	Keine Verbesserung im Klang. Unnatürliche Absenkung gewollter tonaler Anteile.	Komplett unbrauchbare Detektion. Findet die gesuchten Anteile garnicht.	/	/
32	Keine Verbesserung im Klang. Unnatürliche Absenkung gewollter tonaler Anteile.	Komplett unbrauchbare Detektion. Findet die gesuchten Anteile garnicht.	/	/
33	Deutliche Verbesserung im Klang und deutliche Reduzierung von störenden Rutschern & Quietschern.	Brauchbare Erkennung der größten Rutscher, aber es werden ebenfalls auch scharf angeschlagene Seiten miterkannt (sprich scharf klingende Töne). Für die klangliche Verbesserung gut. Für die genaue Isolation reiner Rutscher nicht perfekt.	Gute Trennung ohne Nebeneffekte	Aliasing an den Übergängen. Ungenaue Rücktransformation mit leichten Änderungen und Nulltest Rückständen.

Abbildung A.6: Beobachtungen Tests: Phase 2 (4) - Fall Akustikgitarre

A.2 Quellcode der Applikation

Die im Code benannten und verwendeten Quellen sind: [32, 37, 33, 18].

A.2.1 Main.cpp

```
1  /*
2      File: Main.cpp
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      The main entry point of the JUCE application.
8      Manages creation and shutdown of the app.
9
10     Sources:
11     This part of the code was copied and modified from the
12     JUCE tutorial "Draw audio waveforms":
13     (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
14     url: https://docs.juce.com/master/tutorial\_audio\_thumbnail.html
15
16     License:
17     Based on open-source examples under the ISC License.
18     http://www.isc.org/downloads/software-support-policy/isc-license.
19 */
20
21 #include "MainComponent.h"
22 #include <JuceHeader.h>
23
24 // -- copied from Source A with minor modifications --
25 class DeEssDoctorApplication : public juce::JUCEApplication {
26 public:
27     DeEssDoctorApplication() = default;
28
29     const juce::String getApplicationName() override { return ←
        "DeEssDoctor"; }
30     const juce::String getApplicationVersion() override { return "1.0.0"; }
31
32     // Creates our MainWindow and attaches MainComponent to it.
33     void initialise(const juce::String &) override {
34         mainWindow.reset(new MainWindow("DeEssDoctorApplication",
35                                         new MainComponent(), *this));
36     }
37
38     // Cleans up (destroys the mainWindow pointer)
39     void shutdown() override { mainWindow = nullptr; }
```

```
40
41 private:
42     // Constructor for the top-level app window, sets size and title bar.
43     class MainWindow : public juce::DocumentWindow {
44     public:
45         MainWindow(const juce::String &name, juce::Component *c,
46                 juce::JUCEApplication &app)
47             : DocumentWindow(
48                 name,
49                 juce::Desktop::getInstance()
50                 .getDefaultLookAndFeel()
51                 .findColour(juce::ResizableWindow::backgroundColourId),
52                 juce::DocumentWindow::allButtons),
53             App(app) {
54             setUsingNativeTitleBar(true);
55             setContentOwned(c, true);
56             setResizable(true, true);
57             centreWithSize(getWidth(), getHeight());
58             setVisible(true);
59         }
60
61         // Called when the user clicks the window's close button.
62         void closeButtonPressed() override { App.systemRequestedQuit(); }
63
64     private:
65         juce::JUCEApplication &App;
66
67         JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainWindow)
68     };
69
70     std::unique_ptr<MainWindow> mainWindow;
71 };
72
73 START_JUCE_APPLICATION(DeEssDoctorApplication)
```

A.2.2 MainComponent.h

```
1  /*
2      File: MainComponent.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      MainComponent is the core interface. It manages audio setup & transport,
8      opening and processing audio files, and the user interface for selecting
9      algorithms and playback modes.
```

```

10
11 Sources:
12 File loading, Constructor Setup and Transport Setup for Playback ↵
    including
13 functions like prepareToPlay(), getNextAudioBlock() & releaseResources()
14 are based on the JUCE tutorial "Draw Audio Waveforms":
15 (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
16 url: https://docs.juce.com/master/tutorial_audio_thumbnail.html
17
18 License:
19 Based on open-source examples under the ISC License.
20 http://www.isc.org/downloads/software-support-policy/isc-license.
21 */
22
23 #pragma once
24
25 #include "AudioProcessorManager.h"
26 #include "BufferAudioSource.h"
27 #include "FftControl.h"
28 #include "PositionOverlay.h"
29 #include "RmsControl.h"
30 #include "SampleControl.h"
31 #include "WaveformDisplay.h"
32 #include <JuceHeader.h>
33
34 // -- derived from Source A --
35 class MainComponent : public juce::AudioAppComponent,
36                       public juce::ChangeListener {
37 public:
38     MainComponent();
39     ~MainComponent() override;
40
41     void prepareToPlay(int samplesPerBlockExpected, double sampleRate) ↵
42         override;
43     void getNextAudioBlock(
44         const juce::AudioSourceChannelInfo &bufferToFill) override;
45     void releaseResources() override;
46     void resized() override;
47     void changeListenerCallback(juce::ChangeBroadcaster *source) override;
48
49     void processFile();
50     void exportCurrentPlaybackBuffer();
51 private:
52     std::mutex audioDataMutex;
53
54     enum class PlaybackMode { Original, SibilantsOnly, DeEssed, ↵

```

```

        NoSibilants };
55     PlaybackMode playbackMode = PlaybackMode::Original;
56     void updatePlaybackSource();
57
58     enum class AlgorithmType { SampleBased, RMSBased, FFTBased };
59     AlgorithmType currentAlgorithm = AlgorithmType::SampleBased;
60     void algorithmChanged(AlgorithmType newAlgorithm);
61
62     juce::File loadedFile;
63
64     juce::TextButton openButton{"Open..."};
65     juce::TextButton processButton{"Process"};
66     juce::TextButton exportButton{"Export"};
67
68     juce::TextButton playButton{"Play"};
69     juce::TextButton stopButton{"Stop"};
70
71     juce::TextButton originalButton{"Original"};
72     juce::TextButton sibilantsButton{"Sibilants"};
73     juce::TextButton deEssedButton{"De-Essed"};
74     juce::TextButton noSibilantsButton{"No Sibilants"};
75
76     juce::TextButton sampleBasedButton{"Sample-based"};
77     juce::TextButton rmsBasedButton{"RMS-based"};
78     juce::TextButton fftBasedButton{"FFT-based"};
79
80     std::unique_ptr<juce::FileChooser> chooser;
81     std::unique_ptr<juce::FileChooser> exportChooser;
82     juce::AudioFormatManager formatManager;
83     std::unique_ptr<juce::AudioFormatReaderSource> readerSource;
84     juce::AudioTransportSource transportSource;
85     std::unique_ptr<BufferAudioSource> bufferAudioSource;
86
87     juce::AudioThumbnailCache waveformCache;
88     WaveformDisplay waveformDisplay;
89     PositionOverlay positionOverlay;
90
91     juce::Label fileLabel;
92
93     SampleControl sampleControl;
94     RmsControl rmsControl;
95     FftControl fftControl;
96
97     AudioProcessorManager processorManager;
98
99     enum TransportState { Stopped, Starting, Playing, Stopping };
100    TransportState state;

```

```
101
102     void changeState(TransportState newState);
103     void transportSourceChanged();
104     void openButtonClicked();
105     void playButtonClicked();
106     void stopButtonClicked();
107
108     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(MainComponent)
109 };
```

A.2.3 MainComponent.cpp

```
1  /*
2      File: MainComponent.cpp
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      MainComponent is the core interface. It manages audio setup & transport,
8      opening and processing audio files, and the user interface for selecting
9      algorithms and playback modes.
10
11     Sources:
12     File loading, Constructor Setup and Transport Setup for Playback ↔
13     including
14     functions like prepareToPlay(), getNextAudioBlock() & releaseResources()
15     are based on the JUCE tutorial "Draw Audio Waveforms":
16     (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
17     url: https://docs.juce.com/master/tutorial\_audio\_thumbnail.html
18
19     License:
20     Based on open-source examples under the ISC License.
21     http://www.isc.org/downloads/software-support-policy/isc-license.
22 */
23 #include "MainComponent.h"
24
25 // Constructor
26 // -- derived from Source A --
27 MainComponent::MainComponent()
28     : waveformCache(5),
29     waveformDisplay(512, formatManager, waveformCache),
30     positionOverlay(transportSource),
31     state(Stopped)
32 {
33     addAndMakeVisible(&openButton);
```

```

34     openButton.onClick = [this] { openButtonClicked(); };
35
36     addAndMakeVisible(&exportButton);
37     exportButton.onClick = [this] { exportCurrentPlaybackBuffer(); };
38
39     addAndMakeVisible(&playButton);
40     playButton.onClick = [this] { playButtonClicked(); };
41     playButton.setColour(juce::TextButton::buttonColourId,
42                          juce::Colours::green);
43     playButton.setEnabled(false);
44
45     addAndMakeVisible(&stopButton);
46     stopButton.onClick = [this] { stopButtonClicked(); };
47     stopButton.setColour(juce::TextButton::buttonColourId, ↵
48                          juce::Colours::red);
49     stopButton.setEnabled(false);
50
51     addAndMakeVisible(processButton);
52     processButton.onClick = [this]() { processFile(); };
53
54     addAndMakeVisible(waveformDisplay);
55     addAndMakeVisible(positionOverlay);
56     addAndMakeVisible(sampleControl);
57     addAndMakeVisible(rmsControl);
58     addAndMakeVisible(fftControl);
59
60     fileLabel.setText("No File Loaded", juce::dontSendNotification);
61     fileLabel.setJustificationType(juce::Justification::centredLeft);
62     addAndMakeVisible(fileLabel);
63
64     // Mode buttons as toggleable TextButtons
65     int modeRadioGroup = 2345;
66     originalButton.setClickingTogglesState(true);
67     originalButton.setRadioGroupId(modeRadioGroup);
68     originalButton.setToggleState(true, juce::dontSendNotification);
69     originalButton.setColour(juce::TextButton::buttonOnColourId,
70                             juce::Colours::darkblue);
71     originalButton.onClick = [this]() {
72         playbackMode = PlaybackMode::Original;
73         updatePlaybackSource();
74     };
75     addAndMakeVisible(originalButton);
76
77     sibilantsButton.setClickingTogglesState(true);
78     sibilantsButton.setRadioGroupId(modeRadioGroup);
79     sibilantsButton.setColour(juce::TextButton::buttonOnColourId,
80                              juce::Colours::darkblue);

```

```

80     sibilantsButton.onClick = [this]() {
81         playbackMode = PlaybackMode::SibilantsOnly;
82         updatePlaybackSource();
83     };
84     addAndMakeVisible(sibilantsButton);
85
86     deEssedButton.setClickingTogglesState(true);
87     deEssedButton.setRadioGroupId(modeRadioGroup);
88     deEssedButton.setColour(juce::TextButton::buttonOnColourId,
89                             juce::Colours::darkblue);
90     deEssedButton.onClick = [this]() {
91         playbackMode = PlaybackMode::DeEssed;
92         updatePlaybackSource();
93     };
94     addAndMakeVisible(deEssedButton);
95
96     noSibilantsButton.setClickingTogglesState(true);
97     noSibilantsButton.setRadioGroupId(modeRadioGroup);
98     noSibilantsButton.setColour(juce::TextButton::buttonOnColourId,
99                                 juce::Colours::darkblue);
100    noSibilantsButton.onClick = [this]() {
101        playbackMode = PlaybackMode::NoSibilants;
102        updatePlaybackSource();
103    };
104    addAndMakeVisible(noSibilantsButton);
105
106    // Algorithm buttons as toggleable TextButtons
107    int algoRadioGroup = 3456;
108    sampleBasedButton.setClickingTogglesState(true);
109    sampleBasedButton.setRadioGroupId(algoRadioGroup);
110    sampleBasedButton.setToggleState(
111        true, juce::dontSendNotification); // default selected
112    sampleBasedButton.setColour(juce::TextButton::buttonOnColourId,
113                                juce::Colours::darkblue);
114    sampleBasedButton.onClick = [this]() {
115        algorithmChanged(AlgorithmType::SampleBased);
116    };
117    addAndMakeVisible(sampleBasedButton);
118
119    rmsBasedButton.setClickingTogglesState(true);
120    rmsBasedButton.setRadioGroupId(algoRadioGroup);
121    rmsBasedButton.setColour(juce::TextButton::buttonOnColourId,
122                                juce::Colours::darkblue);
123    rmsBasedButton.onClick = [this]() {
124        algorithmChanged(AlgorithmType::RMSBased);
125    };
126    addAndMakeVisible(rmsBasedButton);

```

```

127
128     fftBasedButton.setClickingTogglesState(true);
129     fftBasedButton.setRadioGroupId(algoRadioGroup);
130     fftBasedButton.setColour(juce::TextButton::buttonOnColourId,
131                             juce::Colours::darkblue);
132     fftBasedButton.onClick = [this]() {
133         algorithmChanged(AlgorithmType::FFTBased);
134     };
135     addAndMakeVisible(fftBasedButton);
136
137     setSize(1200, 800);
138
139     formatManager.registerBasicFormats();
140     transportSource.addChangeListener(this);
141     setAudioChannels(0, 2);
142 }
143
144 // Destructor
145 // -- copied from Source A --
146 MainComponent::~MainComponent() { shutdownAudio(); }
147
148 // Switches between the sample-based, RMS-based, or FFT-based callback ←
149 //   lambda in
150 // AudioProcessorManager. It re-processes automatically if there is a file
151 // loaded.
152 // -- own implementation --
153 void MainComponent::algorithmChanged(AlgorithmType newAlgorithm) {
154     currentAlgorithm = newAlgorithm;
155
156     switch (currentAlgorithm) {
157     case AlgorithmType::SampleBased:
158         processorManager.setDeEssingAlgorithm(
159             [this](juce::AudioBuffer<float> &buffer,
160                 const DeEssingParameters &p, double sr) {
161                 processorManager.sampleBasedDeEssingAlgorithm(buffer, p, sr);
162             });
163         break;
164     case AlgorithmType::RMSBased:
165         processorManager.setDeEssingAlgorithm(
166             [this](juce::AudioBuffer<float> &buffer,
167                 const DeEssingParameters &p, double sr) {
168                 processorManager.rmsDeEssingAlgorithm(buffer, p, sr);
169             });
170         break;
171     case AlgorithmType::FFTBased:

```

```
173     processorManager.setDeEssingAlgorithm(  
174         [this](juce::AudioBuffer<float> &buffer,  
175             const DeEssingParameters &p, double sr) {  
176                 processorManager.fftBasedDeEssingAlgorithm(buffer, p, sr);  
177             });  
178     break;  
179 }  
180 }  
181  
182 // Runs the chosen de-essing algorithm on the loaded file by gathering slider  
183 // params (threshold, freq, etc.) setting them in DeEssingParameters,  
184 // instructing the processorManager to process the file and then calling  
185 // updatePlaybackSource() to handle post-processing playback.  
186 // -- own implementation --  
187 void MainComponent::processFile() {  
188  
189     std::lock_guard<std::mutex> lock(audioDataMutex);  
190  
191     transportSource.stop();  
192     transportSource.setSource(nullptr);  
193     transportSource.releaseResources();  
194  
195     if (!loadedFile.existsAsFile()) {  
196         DBG("No file loaded to process!");  
197         return;  
198     }  
199  
200     DeEssingParameters newParams;  
201  
202     newParams.sampleThreshold = sampleControl.getThreshold();  
203     newParams.sampleMixLevel = sampleControl.getReduction();  
204     newParams.sampleFrequency = sampleControl.getFrequency();  
205     newParams.sampleHysteresis = (int)sampleControl.getHysteresis();  
206  
207     newParams.rmsThreshold = rmsControl.getThreshold();  
208     newParams.rmsMixLevel = rmsControl.getReduction();  
209     newParams.rmsFrequency = rmsControl.getFrequency();  
210     newParams.rmsHysteresisDb = rmsControl.getHysteresisDb();  
211     newParams.rmsDetectionWindowMs = rmsControl.getDetectionWindow();  
212     newParams.rmsCrossfadeMs = rmsControl.getCrossfade();  
213  
214     newParams.fftWindowSize = fftControl.getWindowSize();  
215     newParams.fftOverlap = fftControl.getOverlap();  
216     newParams.fftDetectionRatio = fftControl.getRatioThreshold();  
217     newParams.fftLowFreqCutoff = fftControl.getLowFreq();  
218     newParams.fftCrossfadeFreq = fftControl.getCrossfadeFreq();  
219     newParams.fftMixLevel = fftControl.getSibilantGainDb();
```

```

220
221     processorManager.setParameters(newParams);
222     processorManager.processFileForSibilants(loadedFile);
223     updatePlaybackSource();
224
225     DBG("Processing complete.");
226 }
227
228 // After processing is done, we pick which buffer to play back (Original,
229 // SibilantsOnly, or DeEssed) and set the transport source to that.
230 // -- own implementation --
231 void MainComponent::updatePlaybackSource() {
232     transportSource.setSource(nullptr);
233     transportSource.releaseResources();
234
235     const juce::AudioBuffer<float> *chosenBuffer = nullptr;
236     switch (playbackMode) {
237     case PlaybackMode::Original:
238         chosenBuffer = &processorManager.getOriginalBuffer();
239         break;
240     case PlaybackMode::SibilantsOnly:
241         chosenBuffer = &processorManager.getSibilantBuffer();
242         break;
243     case PlaybackMode::DeEssed:
244         chosenBuffer = &processorManager.getProcessedBuffer();
245         break;
246     case PlaybackMode::NoSibilants:
247         chosenBuffer = &processorManager.getNoSibilantBuffer();
248         break;
249     }
250
251     if (chosenBuffer->getNumSamples() == 0) {
252         DBG("No processed data available.");
253         return;
254     }
255
256     double sampleRate = processorManager.getSampleRate();
257     bufferAudioSource = std::make_unique<BufferAudioSource>(*chosenBuffer, ←
        sampleRate);
258     transportSource.setSource(bufferAudioSource.get(), 0, nullptr);
259     transportSource.setPosition(0.0);
260     transportSource.stop();
261     changeState(Stopped);
262
263     waveformDisplay.setSibilantBuffer(processorManager.getSibilantBuffer());
264     repaint();
265 }

```

```

266
267 // Called by JUCE before audio playback starts.
268 // We configure the transport source and any DSP to be ready for streaming
269 // audio.
270 // -- copied from Source A with minor modifications --
271 void MainComponent::prepareToPlay(int samplesPerBlockExpected,
272                                   double sampleRate) {
273     transportSource.prepareToPlay(samplesPerBlockExpected, sampleRate);
274     int numChannels = deviceManager.getCurrentAudioDevice()
275                     ->getActiveOutputChannels()
276                     .countNumberOfSetBits();
277
278     processorManager.prepare(sampleRate, samplesPerBlockExpected, ←
279                             numChannels);
279     DBG("ProcessorManager prepared with sample rate: "
280         << sampleRate << ", block size: " << samplesPerBlockExpected
281         << ", num channels: " << numChannels);
282 }
283
284 // Continuously called by JUCE to fill the audio buffer with data.
285 // If no file is loaded, it clears the buffer. Otherwise, it pulls data from
286 // transportSource.
287 // -- copied from Source A with minor modifications --
288 void MainComponent::getNextAudioBlock(
289     const juce::AudioSourceChannelInfo &bufferToFill) {
290     std::lock_guard<std::mutex> lock(audioDataMutex);
291
292     transportSource.getNextAudioBlock(bufferToFill);
293 }
294
295 // Called when audio playback stops or devices are changed. Frees any audio
296 // resources in use.
297 // -- copied from Source A --
298 void MainComponent::releaseResources() { ←
299     transportSource.releaseResources(); }
300
301 // Called whenever the component's size changes.
302 // We manually carve out sub-rectangles for each UI section (top controls,
303 // waveform, button rows, bottom slider columns) and lay out child components
304 // using either direct setBounds() or FlexBoxes.
305 // -- derived from Source A --
306 void MainComponent::resized() {
307     auto bounds = getLocalBounds();
308
309     // Sectioning the Layout
310     const int topSectionHeight = bounds.getHeight() / 16;
311     const int waveformSectionHeight = bounds.getHeight() / 2;

```

```

311     const int buttonRowHeight = bounds.getHeight() / 16;
312
313     // File name, Open button, Process button
314     {
315         juce::FlexBox topSection;
316         topSection.flexDirection = juce::FlexBox::Direction::row;
317         topSection.items.add(juce::FlexItem(fileLabel).withFlex(1.0f));
318         topSection.items.add(juce::FlexItem(openButton).withFlex(0.25f));
319         topSection.items.add(juce::FlexItem(processButton).withFlex(0.25f));
320         topSection.items.add(juce::FlexItem(exportButton).withFlex(0.25f));
321         topSection.performLayout(bounds.removeFromTop(topSectionHeight));
322     }
323
324     // Waveform display
325     auto waveformArea = bounds.removeFromTop(waveformSectionHeight);
326     waveformDisplay.setBounds(waveformArea);
327     positionOverlay.setBounds(waveformArea);
328
329     // Original / Sibilants / De-essed buttons
330     {
331         juce::FlexBox modeSection;
332         modeSection.flexDirection = juce::FlexBox::Direction::row;
333         modeSection.justifyContent = ←
334             juce::FlexBox::JustifyContent::spaceAround;
335         modeSection.items.add(juce::FlexItem(originalButton).withFlex(1.0f));
336         modeSection.items.add(juce::FlexItem(sibilantsButton).withFlex(1.0f));
337         modeSection.items.add(juce::FlexItem(deEssedButton).withFlex(1.0f));
338         modeSection.items.add(juce::FlexItem(noSibilantsButton).withFlex(1.0f));
339         modeSection.performLayout(bounds.removeFromTop(buttonRowHeight));
340     }
341
342     // Play / Stop buttons
343     {
344         juce::FlexBox transportSection;
345         transportSection.flexDirection = juce::FlexBox::Direction::row;
346         transportSection.justifyContent =
347             juce::FlexBox::JustifyContent::spaceAround;
348         transportSection.items.add(juce::FlexItem(playButton).withFlex(1.0f));
349         transportSection.items.add(juce::FlexItem(stopButton).withFlex(1.0f));
350         transportSection.performLayout(bounds.removeFromTop(buttonRowHeight));
351     }
352
353     // Buttons for algorithms
354     {
355         juce::FlexBox algoSection;
356         algoSection.flexDirection = juce::FlexBox::Direction::row;
357         algoSection.justifyContent = ←

```

```

        juice::FlexBox::JustifyContent::spaceAround;
357     algoSection.items.add(juce::FlexItem(sampleBasedButton).withFlex(1.0f));
358     algoSection.items.add(juce::FlexItem(rmsBasedButton).withFlex(1.0f));
359     algoSection.items.add(juce::FlexItem(fftBasedButton).withFlex(1.0f));
360     algoSection.performLayout(bounds.removeFromTop(buttonRowHeight));
361 }
362
363 // Three columns for the slider sets at the bottom
364 int columnWidth = bounds.getWidth() / 3;
365 auto bottomArea = bounds;
366
367 auto sampleCol = bottomArea.removeFromLeft(columnWidth);
368 sampleControl.setBounds(sampleCol.reduced(10));
369
370 auto rmsCol = bottomArea.removeFromLeft(columnWidth);
371 rmsControl.setBounds(rmsCol.reduced(10));
372
373 auto fftCol = bottomArea;
374 fftControl.setBounds(fftCol.reduced(10));
375 }
376
377 // We listen to the transport source changes. If it starts or stops playing,
378 // we update the GUI state (enable/disable buttons, etc.).
379 // -- copied from Source A --
380 void MainComponent::changeListenerCallback(juce::ChangeBroadcaster <-
    *source) {
381     if (source == &transportSource)
382         transportSourceChanged();
383 }
384
385 // Called when the transport source's play state changes.
386 // We update our own state to reflect whether it's playing or stopped.
387 // -- copied from Source A --
388 void MainComponent::transportSourceChanged() {
389     if (transportSource.isPlaying())
390         changeState(Playing);
391     else
392         changeState(Stopped);
393 }
394
395 // Internal function to change the transport play state (Stopped, Starting,
396 // Playing, Stopping) and update the UI accordingly (e.g., enable or disable
397 // Play/Stop buttons).
398 // -- copied from Source A --
399 void MainComponent::changeState(TransportState newState) {
400     if (state != newState) {
401         state = newState;

```

```
402
403     switch (state) {
404     case Stopped:
405         stopButton.setEnabled(false);
406         playButton.setEnabled(true);
407         transportSource.setPosition(0.0);
408         break;
409
410     case Starting:
411         playButton.setEnabled(false);
412         transportSource.start();
413         break;
414
415     case Playing:
416         stopButton.setEnabled(true);
417         break;
418
419     case Stopping:
420         transportSource.stop();
421         break;
422
423     default:
424         jassertfalse;
425         break;
426     }
427 }
428 }
429
430 // Opens a FileChooser to let the user pick a .wav file.
431 // If valid, sets up the reader and updates the UI so we can play/process the
432 // file.
433 // -- copied from Source A --
434 void MainComponent::openButtonClicked() {
435     chooser = std::make_unique<juce::FileChooser>(
436         "Select a Wave file to play...", juce::File{}, "*.wav");
437     auto chooserFlags = juce::FileBrowserComponent::openMode |
438         juce::FileBrowserComponent::canSelectFiles;
439
440     chooser->launchAsync(chooserFlags, [this](const juce::FileChooser &fc) {
441         auto file = fc.getResult();
442
443         if (file != juce::File{}) {
444             loadedFile = file;
445             auto *reader = formatManager.createReaderFor(file);
446             fileLabel.setText(file.getFileName(), juce::dontSendNotification);
447
448             if (reader != nullptr) {
```

```

449         std::lock_guard<std::mutex> lock(audioDataMutex);
450
451         auto newSource =
452             std::make_unique<juce::AudioFormatReaderSource>(reader,
453                                                         true);
454         transportSource.setSource(newSource.get(), 0, nullptr,
455                                 reader->sampleRate);
456         playButton.setEnabled(true);
457         waveformDisplay.setFile(file);
458         readerSource.reset(newSource.release());
459     }
460 }
461 });
462 }
463
464 // Callback that tell the transport to start playback.
465 // -- copied from Source A --
466 void MainComponent::playButtonClicked() { changeState(Starting); }
467
468 // Callback that tell the transport to stop playback.
469 // -- copied from Source A --
470 void MainComponent::stopButtonClicked() { changeState(Stopping); }
471
472 // Opens a FileChooser to let the user pick a location to save the current
473 // playback buffer as a .wav file.
474 // -- derived from Source A --
475 void MainComponent::exportCurrentPlaybackBuffer() {
476
477     const juce::AudioBuffer<float> *chosenBuffer = nullptr;
478     switch (playbackMode) {
479     case PlaybackMode::Original:
480         chosenBuffer = &processorManager.getOriginalBuffer();
481         break;
482     case PlaybackMode::SibilantsOnly:
483         chosenBuffer = &processorManager.getSibilantBuffer();
484         break;
485     case PlaybackMode::DeEssed:
486         chosenBuffer = &processorManager.getProcessedBuffer();
487         break;
488     case PlaybackMode::NoSibilants:
489         chosenBuffer = &processorManager.getNoSibilantBuffer();
490         break;
491     }
492
493     if (chosenBuffer == nullptr || chosenBuffer->getNumSamples() == 0) {
494         juce::AlertWindow::showMessageBoxAsync(
495             juce::AlertWindow::WarningIcon, "Export Failed",

```

```

496         "No audio data is available to export!");
497     return;
498 }
499
500 exportChooser = std::make_unique<juce::FileChooser>(
501     "Select a location to save the WAV file...", juce::File{}, "*.wav");
502
503 auto fileBrowserFlags = (juce::FileBrowserComponent::saveMode |
504     juce::FileBrowserComponent::canSelectFiles);
505
506 exportChooser->launchAsync(
507     fileBrowserFlags, [this, chosenBuffer](const juce::FileChooser &
508         &fc) {
509         auto file = fc.getResult();
510         if (file == juce::File()) {
511             DBG("[Export] user canceled the export FileChooser");
512             return;
513         }
514
515         std::unique_ptr<juce::FileOutputStream> outStream(
516             file.createOutputStream());
517         if (!outStream) {
518             juce::AlertWindow::showMessageBoxAsync(
519                 juce::AlertWindow::WarningIcon, "Export Failed",
520                 "Couldn't create output stream for file!");
521             return;
522         }
523
524         juce::WavAudioFormat wavFormat;
525         double sampleRate = processorManager.getSampleRate();
526         int numChannels = chosenBuffer->getNumChannels();
527         int bitsPerSample = 24;
528
529         std::unique_ptr<juce::AudioFormatWriter> writer(
530             wavFormat.createWriterFor(outStream.get(), sampleRate,
531                 (unsigned int)numChannels,
532                 bitsPerSample, {}, // metadata
533                 0 // compression
534             ));
535
536         if (!writer) {
537             juce::AlertWindow::showMessageBoxAsync(
538                 juce::AlertWindow::WarningIcon, "Export Failed",
539                 "Couldn't create WAV writer!");
540             return;
541         }

```

```
542         outputStream.release();
543
544         writer->writeFromAudioSampleBuffer(*chosenBuffer, 0,
545                                           chosenBuffer->getNumSamples());
546         writer->flush();
547
548         juce::AlertWindow::showMessageBoxAsync(
549             juce::AlertWindow::InfoIcon, "Export Successful",
550             "File saved to:\n" + file.getFullPathName());
551
552         DBG("[Export] WAV Export complete");
553     });
554 }
```

A.2.4 BufferAudioSource.h

```
1  /*
2   File: BufferAudioSource.h
3   Author: Leif Benedikt Rehtanz
4   Date: 2024 / 2025
5
6   Description:
7   This class is a basic implementation of PositionableAudioSource. It ↵
8   allows for
9   playing back audio from a memory buffer instead of a file. It is used ↵
10  as the
11  transport source since we're holding our processed audio in buffers ↵
12  before exporting
13  them to files.
14  Raw Material Software Limited. "PositionableAudioSource". 2024.
15  url: https://docs.juce.com/master/classPositionableAudioSource.html
16  */
17
18 #pragma once
19
20 #include <JuceHeader.h>
21
22 // Implements PositionableAudioSource. It plays audio from a memory buffer
23 // instead of a file on disk.
24 // -- own implementation --
25 class BufferAudioSource : public juce::PositionableAudioSource {
26 public:
27     BufferAudioSource(const juce::AudioBuffer<float> &bufferToUse,
28                     double sampleRate);
29
30     void setNextReadPosition(juce::int64 newPosition) override;
```

```

28     juce::int64 getNextReadPosition() const override;
29     juce::int64 getTotalLength() const override;
30     bool isLooping() const override { return false; }
31
32     void prepareToPlay(int samplesPerBlockExpected, double sampleRate) ↵
        override;
33     void releaseResources() override {}
34     void getNextAudioBlock(
35         const juce::AudioSourceChannelInfo &bufferToFill) override;
36
37 private:
38     mutable std::mutex positionMutex;
39     const juce::AudioBuffer<float> &buffer;
40     double sampleRate;
41     juce::int64 readPosition{0};
42
43     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(BufferAudioSource)
44 };

```

A.2.5 BufferAudioSource.cpp

```

1  /*
2   File: BufferAudioSource.cpp
3   Author: Leif Benedikt Rehtanz
4   Date: 2024 / 2025
5
6   Description:
7   This class is a basic implementation of PositionableAudioSource. It ↵
       allows for
8   playing back audio from a memory buffer instead of a file. It is used ↵
       as the
9   transport source since we're holding our processed audio in buffers ↵
       before exporting
10  them to files.
11  Raw Material Software Limited. "PositionableAudioSource". 2024.
12  url: https://docs.juce.com/master/classPositionableAudioSource.html
13  */
14
15  #include "BufferAudioSource.h"
16
17  // Constructor
18  // -- own implementation --
19  BufferAudioSource::BufferAudioSource(
20      const juce::AudioBuffer<float> &bufferToUse, double sampleRateToUse)
21      : buffer(bufferToUse), sampleRate(sampleRateToUse) {}
22

```

```

23 // Sets the sample index from which playback should continue.
24 // -- own implementation --
25 void BufferAudioSource::setNextReadPosition(juce::int64 newPosition) {
26     std::lock_guard<std::mutex> lock(positionMutex);
27     readPosition = juce::jlimit(juce::int64(0), getTotalLength(), ↵
        newPosition);
28 }
29
30 // Returns the current read index.
31 // -- own implementation --
32 juce::int64 BufferAudioSource::getNextReadPosition() const {
33     std::lock_guard<std::mutex> lock(positionMutex);
34     return readPosition;
35 }
36
37 // The total number of samples in the buffer.
38 // -- own implementation --
39 juce::int64 BufferAudioSource::getTotalLength() const {
40     return buffer.getNumSamples();
41 }
42
43 // Unused but implemented to satisfy the PositionableAudioSource interface
44 // requirements.
45 // -- own implementation --
46 void BufferAudioSource::prepareToPlay(int /*samplesPerBlockExpected*/,
47     double /*sampleRate*/) {}
48
49 // Copies data from our internal buffer to JUCEs output buffer, advancing the
50 // read position. Clears remainder if we hit the end.
51 // -- own implementation --
52 void BufferAudioSource::getNextAudioBlock(
53     const juce::AudioSourceChannelInfo &bufferToFill) {
54     std::lock_guard<std::mutex> lock(positionMutex);
55     auto numSamplesToCopy =
56         juce::jmin(bufferToFill.numSamples,
57             buffer.getNumSamples() - static_cast<int>(readPosition));
58
59     for (int channel = 0; channel < buffer.getNumChannels(); ++channel) {
60         if (channel < bufferToFill.buffer->getNumChannels()) {
61             bufferToFill.buffer->copyFrom(channel, bufferToFill.startSample,
62                 buffer, channel, (int)readPosition,
63                 (int)numSamplesToCopy);
64         }
65     }
66
67     readPosition += numSamplesToCopy;
68

```

```

69     if (numSamplesToCopy < bufferToFill.numSamples) {
70         bufferToFill.clearActiveBufferRegion();
71     }
72 }

```

A.2.6 AudioProcessorManager.h

```

1  /*
2      File: AudioProcessorManager.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Header file defining the AudioProcessorManager class, which manages the
8      parameters and processing logic for de-essing algorithms.
9
10     Sources:
11     Parameter handling concepts, struct setup, function setup for ↔
12     processing audio
13     with multiple functions, filtering principles (Linkwitz-Riley) and DSP ↔
14     logic were
15     informed by the SimpleMultiBandComp tutorial provided by Free Code ↔
16     Camp & Mat Kat Music LLC:
17     (Source C) Mat Kat Music LLC. SimpleMultiBandComp Tutorial (GitHub). ↔
18     Nov. 2021.
19     url: https://github.com/matkatmusic/SimpleMultiBandComp
20     (Source C) Mat Kat Music LLC. C++ Programming Tutorial - Build a ↔
21     3-Band Compressor Audio Plugin
22     (w/ JUCE Framework). Nov. 2021.
23     url: https://www.youtube.com/watch?v=Mo00co3Vimo&t=7276s
24
25     The basic FFT setup was informed by the JUCE tutorial "The fast ↔
26     Fourier transform":
27     (Source B) Raw Material Software Limited. The fast Fourier transform. ↔
28     2024.
29     url: https://docs.juce.com/master/tutorial_simple_fft.html
30
31     License:
32     JUCE's code are open-source examples under the ISC License.
33     http://www.isc.org/downloads/software-support-policy/isc-license
34
35     Mat Kat Music's code is licensed under the GNU General Public License.
36     https://www.gnu.org/licenses/gpl-3.0.en.html
37 */
38 #pragma once

```

```
33
34 #include <JuceHeader.h>
35 #include <functional>
36
37 // Holds parameters for the de-essing algorithms.
38 // -- derived from Source C --
39 struct DeEssingParameters {
40     float sampleThreshold = -20.0f;
41     float sampleMixLevel = 0.0f;
42     float sampleFrequency = 6500.0f;
43     int sampleHysteresis = 100;
44
45     float rmsThreshold = -20.0f;
46     float rmsMixLevel = 0.0f;
47     float rmsFrequency = 6500.0f;
48     float rmsHysteresisDb = 10.0f;
49     float rmsDetectionWindowMs = 10.0f;
50     float rmsCrossfadeMs = 5.0f;
51
52     int fftWindowSize = 1024;
53     float fftOverlap = 0.5f;
54     float fftDetectionRatio = 0.2f;
55     float fftLowFreqCutoff = 4000.0f;
56     float fftCrossfadeFreq = 10000.0f;
57     float fftMixLevel = 0.0f;
58 };
59
60 // Manages audio processing for de-essing algorithms.
61 // -- derived from Source C --
62 class AudioProcessorManager {
63 public:
64     AudioProcessorManager();
65     ~AudioProcessorManager() = default;
66
67     void prepare(double sampleRate, int samplesPerBlock, int numChannels);
68
69     void processFileForSibilants(const juce::File &file);
70
71     const juce::AudioBuffer<float> &getOriginalBuffer() const {
72         return originalBuffer;
73     }
74     const juce::AudioBuffer<float> &getProcessedBuffer() const {
75         return processedBuffer;
76     }
77     const juce::AudioBuffer<float> &getSibilantBuffer() const {
78         return sibilantBuffer;
79     }
79 }
```

```

80     const juce::AudioBuffer<float> &getNoSibilantBuffer() const {
81         return noSibilantBuffer;
82     }
83
84     void
85     setDeEssingAlgorithm(std::function<void(juce::AudioBuffer<float> &,
86                                     const DeEssingParameters &, double)>
87                         algorithm);
88
89     void setParameters(const DeEssingParameters &newParams) {
90         params = newParams;
91     }
92
93     double getSampleRate() const { return processedSampleRate; }
94
95     void sampleBasedDeEssingAlgorithm(juce::AudioBuffer<float> &buffer,
96                                     const DeEssingParameters &p,
97                                     double sampleRate);
98     void rmsDeEssingAlgorithm(juce::AudioBuffer<float> &buffer,
99                             const DeEssingParameters &p, double sampleRate);
100    void fftBasedDeEssingAlgorithm(juce::AudioBuffer<float> &buffer,
101                                  const DeEssingParameters &p,
102                                  double sampleRate);
103
104    private:
105        double processedSampleRate = 44100.0;
106
107        juce::AudioBuffer<float> originalBuffer;
108        juce::AudioBuffer<float> processedBuffer;
109        juce::AudioBuffer<float> sibilantBuffer;
110        juce::AudioBuffer<float> noSibilantBuffer;
111
112        juce::dsp::LinkwitzRileyFilter<float> highPassFilter;
113        juce::dsp::LinkwitzRileyFilter<float> allPassFilter;
114        juce::dsp::LinkwitzRileyFilter<float> lowPassFilter;
115
116        DeEssingParameters params;
117
118        std::function<void(juce::AudioBuffer<float> &, const ↵
119                            DeEssingParameters &,
120                            double)>
121        deEssingAlgorithm;
122    };

```

A.2.7 AudioProcessorManager.cpp

```

1  /*
2      File: AudioProcessorManager.cpp
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Implements de-essing algorithms, including sample-based, RMS-based, and
8      FFT-based processing. Manages audio buffers and DSP preparation.
9
10     Sources:
11     Parameter handling concepts, struct setup, function setup for ↵
12         processing audio
13     with multiple functions, filtering principles (Linkwitz-Riley) and DSP ↵
14         logic were
15     informed by the SimpleMultiBandComp tutorial provided by Free Code ↵
16         Camp & Mat Kat Music LLC:
17     (Source C) Mat Kat Music LLC. SimpleMultiBandComp Tutorial (GitHub). ↵
18         Nov. 2021.
19     url: https://github.com/matkatmusic/SimpleMultiBandComp
20     (Source C) Mat Kat Music LLC. C++ Programming Tutorial - Build a ↵
21         3-Band Compressor Audio Plugin
22     (w/ JUCE Framework). Nov. 2021.
23     url: https://www.youtube.com/watch?v=Mo0Oco3Vimo&t=7276s
24
25     The basic FFT setup was informed by the JUCE tutorial "The fast ↵
26         Fourier transform":
27     (Source B) Raw Material Software Limited. The fast Fourier transform. ↵
28         2024.
29     url: https://docs.juce.com/master/tutorial_simple_fft.html
30
31     License:
32     JUCE's code are open-source examples under the ISC License.
33     http://www.isc.org/downloads/software-support-policy/isc-license
34
35     Mat Kat Music's code is licensed under the GNU General Public License.
36     https://www.gnu.org/licenses/gpl-3.0.en.html
37 */
38
39 #include "AudioProcessorManager.h"
40
41 // Initializes the Linkwitz-Riley high-pass and all-pass filters,
42 // and sets default to sampleBasedDeEssingAlgorithm.
43 // -- derived from Source C --
44 AudioProcessorManager::AudioProcessorManager() {
45     highPassFilter.setType(juce::dsp::LinkwitzRileyFilterType::highpass);
46     highPassFilter.setCutoffFrequency(params.sampleFrequency);
47     allPassFilter.setType(juce::dsp::LinkwitzRileyFilterType::allpass);

```

```

41     lowPassFilter.setType(juce::dsp::LinkwitzRileyFilterType::lowpass);
42     lowPassFilter.setCutoffFrequency(params.sampleFrequency);
43
44     // setting default
45     deEssingAlgorithm = [this](juce::AudioBuffer<float> &buffer,
46                               const DeEssingParameters &p, double sr) {
47         sampleBasedDeEssingAlgorithm(buffer, p, sr);
48     };
49 }
50
51 // Prepares our DSP filters by setting their sample rate, block size, etc.
52 // Called before we do any actual audio processing.
53 // -- copied from Source C with minor modifications --
54 void AudioProcessorManager::prepare(double sampleRate, int samplesPerBlock,
55                                    int numChannels) {
56     juce::dsp::ProcessSpec spec{sampleRate,
57                                static_cast<juce::uint32>(samplesPerBlock),
58                                static_cast<juce::uint32>(numChannels)};
59     highPassFilter.prepare(spec);
60     highPassFilter.reset();
61
62     allPassFilter.prepare(spec);
63     allPassFilter.reset();
64
65     lowPassFilter.prepare(spec);
66     lowPassFilter.reset();
67
68     DBG("AudioProcessorManager prepared with sample rate: "
69         << sampleRate << ", block size: " << samplesPerBlock
70         << ", num channels: " << numChannels);
71 }
72 // Sets the de-essing algorithm.
73 // -- own implementation --
74 void AudioProcessorManager::setDeEssingAlgorithm(
75     std::function<void(juce::AudioBuffer<float> &, const ↵
76         DeEssingParameters &,
77         double)>
78     algorithm) {
79     deEssingAlgorithm = std::move(algorithm);
80 }
81 // Reads the entire audio file into memory (originalBuffer), copies it to
82 // processedBuffer, then applies the currently selected de-essing ↵
83 // algorithm. The
84 // result is stored in processedBuffer, and the sibilantBuffer is updated ↵
85 // with
86 // whatever we identified as sibilants.

```

```

85 // -- own implementation --
86 void AudioProcessorManager::processFileForSibilants(const juce::File &file) {
87     juce::AudioFormatManager formatManager;
88     formatManager.registerBasicFormats();
89
90     auto reader = std::unique_ptr<juce::AudioFormatReader>(
91         formatManager.createReaderFor(file));
92
93     processedSampleRate = reader->sampleRate;
94     int fileNumChannels = (int)reader->numChannels;
95     prepare(processedSampleRate, 512, 2);
96
97     originalBuffer.setSize(2, (int)reader->lengthInSamples);
98     processedBuffer.setSize(2, (int)reader->lengthInSamples);
99     sibilantBuffer.setSize(2, (int)reader->lengthInSamples);
100    noSibilantBuffer.setSize(2, (int)reader->lengthInSamples);
101
102    // Read the audio data into originalBuffer
103    reader->read(&originalBuffer, 0, (int)reader->lengthInSamples, 0, true,
104        true);
105
106    // Apply the selected algorithm
107    if (deEssingAlgorithm)
108        deEssingAlgorithm(originalBuffer, params, processedSampleRate);
109    DBG("processFileForSibilants with channels: "
110        << fileNumChannels << ", sample rate: " << processedSampleRate);
111 }
112
113 // Uses a high-pass filter and a simple hysteresis approach (based on sample
114 // amplitude) to detect sibilants. Subtracts them from the original, then
115 // mixes
116 // them back in at a reduced level.
117 // -- own implementation including concepts from Source C in regards to
118 // filtering --
119 void AudioProcessorManager::sampleBasedDeEssingAlgorithm(
120     juce::AudioBuffer<float> &buffer, const DeEssingParameters &p,
121     double sampleRate) {
122     float threshold = p.sampleThreshold;
123     float mixLevel = p.sampleMixLevel;
124     float frequency = p.sampleFrequency;
125     int hysteresisSamples = p.sampleHysteresis;
126
127     juce::AudioBuffer<float> mainBuffer;
128     mainBuffer.makeCopyOf(buffer);
129
130     juce::AudioBuffer<float> sibilantBufferTemp;

```

```

129     sibilantBufferTemp.makeCopyOf(buffer);
130
131     juce::AudioBuffer<float> highPassBuffer;
132     highPassBuffer.makeCopyOf(buffer);
133
134     juce::AudioBuffer<float> finalBuffer;
135     finalBuffer.makeCopyOf(buffer);
136
137     // High-pass filtered buffer
138     {
139         juce::dsp::AudioBlock<float> highpassBlock(highPassBuffer);
140         juce::dsp::ProcessContextReplacing<float> ←
            highpassContext(highpassBlock);
141         highPassFilter.setCutoffFrequency(frequency);
142         highPassFilter.process(highpassContext);
143     }
144
145     // All-pass filtered buffer to compensate for high-passed buffers latency
146     {
147         juce::dsp::AudioBlock<float> mainBlock(mainBuffer);
148         juce::dsp::ProcessContextReplacing<float> mainContext(mainBlock);
149         allPassFilter.process(mainContext);
150     }
151     sibilantBufferTemp.makeCopyOf(mainBuffer);
152     finalBuffer.makeCopyOf(mainBuffer);
153
154     int numChannels = buffer.getNumChannels();
155     int numSamples = buffer.getNumSamples();
156
157     for (int channel = 0; channel < numChannels; ++channel) {
158         auto *mainData = mainBuffer.getWritePointer(channel);
159         auto *sibilantData = sibilantBufferTemp.getWritePointer(channel);
160         auto *highpassedData = highPassBuffer.getWritePointer(channel);
161
162         int counter = 0; // hysteresis counter
163
164         for (int sample = 0; sample < numSamples; ++sample) {
165             // Check threshold
166             if (std::abs(highpassedData[sample]) >
167                 juce::Decibels::decibelsToGain(threshold)) {
168                 counter = hysteresisSamples;
169             }
170
171             if (counter > 0) {
172                 // In sibilant region
173                 --counter;
174                 mainData[sample] = 0.0f;

```

```

175         } else {
176             // Outside sibilant region
177             sibilantData[sample] = 0.0f;
178         }
179     }
180 }
181
182 // Mix adjusted sibilants back into the original signal
183 float gainFactor = juce::Decibels::decibelsToGain(mixLevel);
184 for (int channel = 0; channel < numChannels; ++channel) {
185     auto *mainData = mainBuffer.getWritePointer(channel);
186     auto *sibilantData = sibilantBufferTemp.getWritePointer(channel);
187     auto *finalData = finalBuffer.getWritePointer(channel);
188
189     for (int sample = 0; sample < numSamples; ++sample) {
190         finalData[sample] =
191             mainData[sample] + gainFactor * sibilantData[sample];
192     }
193 }
194
195 noSibilantBuffer.makeCopyOf(mainBuffer);
196 sibilantBuffer.makeCopyOf(sibilantBufferTemp);
197 processedBuffer.makeCopyOf(finalBuffer);
198 }
199
200 // Similar approach to sample based but uses a rolling RMS window to detect
201 // sibilants, plus hysteresis in dB. Also crossfades transitions into and ↵
202 // out of
203 // sibilant segments.
204 // -- own implementation including concepts from Source C in regards to ↵
205 // filtering --
206 void AudioProcessorManager::rmsDeEssingAlgorithm(
207     juce::AudioBuffer<float> &buffer, const DeEssingParameters &p,
208     double sampleRate) {
209     float thresholdHigh = p.rmsThreshold;
210     float thresholdLow = p.rmsThreshold - p.rmsHysteresisDb;
211     float frequency = p.rmsFrequency;
212     float mixLevel = p.rmsMixLevel;
213
214     int detectionWindowSamples = (int)(p.rmsDetectionWindowMs * sampleRate ↵
215         / 1000.0);
216     int crossfadeSamples = (int)(p.rmsCrossfadeMs * sampleRate / 1000.0);
217
218     juce::AudioBuffer<float> mainBuffer;
219     mainBuffer.makeCopyOf(buffer);
220
221     juce::AudioBuffer<float> sibilantBufferTemp;

```

```

219     sibilantBufferTemp.makeCopyOf(buffer);
220
221     juce::AudioBuffer<float> lowPassBuffer;
222     lowPassBuffer.makeCopyOf(buffer);
223
224     juce::AudioBuffer<float> finalBuffer;
225     finalBuffer.makeCopyOf(buffer);
226
227     // two high passed buffers for splitting the top end into sibilant and ↵
        non-sibilant signals
228     {
229         juce::dsp::AudioBlock<float> mainBlock(mainBuffer);
230         juce::dsp::ProcessContextReplacing<float> mainContext(mainBlock);
231         highPassFilter.setCutoffFrequency(frequency);
232         highPassFilter.process(mainContext);
233     }
234
235     sibilantBufferTemp.makeCopyOf(mainBuffer);
236
237     // one low passed buffer for the lower frequency band to be unaffected
238     // since those aren't in the sibilant range
239     {
240         juce::dsp::AudioBlock<float> lowpassBlock(lowPassBuffer);
241         juce::dsp::ProcessContextReplacing<float> ↵
            lowpassContext(lowpassBlock);
242         lowPassFilter.setCutoffFrequency(frequency);
243         lowPassFilter.process(lowpassContext);
244     }
245
246     int numChannels = buffer.getNumChannels();
247     int numSamples = buffer.getNumSamples();
248
249     float gainFactor = juce::Decibels::decibelsToGain(mixLevel);
250
251     // Process each channel
252     for (int channel = 0; channel < numChannels; ++channel) {
253         auto *mainData = mainBuffer.getWritePointer(channel);
254         auto *sibilantData = sibilantBufferTemp.getWritePointer(channel);
255         auto *lowpassedData = lowPassBuffer.getWritePointer(channel);
256         auto *finalData = finalBuffer.getWritePointer(channel);
257
258         std::deque<float> windowSamples;
259         double sumOfSquares = 0.0;
260
261         bool isSibilant = false;
262         int segmentStart = -1;
263         std::vector<std::pair<int, int>> sibilantSegments;

```

```

264
265 // find the sibilant segments through calculating rms
266 for (int sample = 0; sample < numSamples; ++sample) {
267     float s = sibilantData[sample];
268     float sq = s * s;
269
270     windowSamples.push_back(sq);
271     sumOfSquares += sq;
272
273     // check if rms window length exceeded
274     if ((int>windowSamples.size() > detectionWindowSamples) {
275         sumOfSquares -= windowSamples.front();
276         windowSamples.pop_front();
277     }
278
279     float rmsDb = -100.0f;
280     if (!windowSamples.empty()) {
281         double meanSq = sumOfSquares / windowSamples.size();
282         float rms = (float)std::sqrt(meanSq);
283         rmsDb = juce::Decibels::gainToDecibels(rms);
284     }
285
286     // hysteresis based sibilant detection
287     if (!isSibilant && rmsDb > thresholdHigh) {
288         isSibilant = true;
289         segmentStart = sample;
290     } else if (isSibilant && rmsDb < thresholdLow) {
291         isSibilant = false;
292         sibilantSegments.push_back({segmentStart, sample - 1});
293     }
294 }
295
296 // go from segments to sample level
297 std::vector<bool> isSibilantSample(numSamples, false);
298 for (auto &seg : sibilantSegments) {
299     for (int i = seg.first; i <= seg.second; ++i)
300         isSibilantSample[i] = true;
301 }
302
303 // zero out non sibilants so only sibilants are left
304 for (int i = 0; i < numSamples; ++i) {
305     if (!isSibilantSample[i])
306         sibilantData[i] = 0.0f;
307 }
308
309 // crossfade segment boundaries
310 for (auto &seg : sibilantSegments) {

```

```

311         int segLen = seg.second - seg.first + 1;
312         int fadeLength = juce::jmin(crossfadeSamples, segLen / 2);
313
314         // Fade in
315         for (int i = 0; i < fadeLength; ++i) {
316             float factor = (float)i / fadeLength;
317             sibilantData[seg.first + i] *= factor;
318         }
319
320         // Fade out
321         for (int i = 0; i < fadeLength; ++i) {
322             float factor = 1.0f - (float)i / fadeLength;
323             sibilantData[seg.second - i] *= factor;
324         }
325     }
326
327     // subtract sibilants from original
328     for (int sample = 0; sample < numSamples; ++sample) {
329         if (sibilantData[sample] != 0.0f) {
330             mainData[sample] -= sibilantData[sample];
331         }
332         mainData[sample] += lowpassedData[sample];
333     }
334
335     // add back sibilants with gain factor
336     for (int sample = 0; sample < numSamples; ++sample) {
337         finalData[sample] =
338             mainData[sample] + gainFactor * sibilantData[sample];
339     }
340 }
341
342 noSibilantBuffer.makeCopyOf(mainBuffer);
343 sibilantBuffer.makeCopyOf(sibilantBufferTemp);
344 processedBuffer.makeCopyOf(finalBuffer);
345 }
346
347 // Splits the audio into FFT frames, identifies sibilant frequency bins
348 // above 4kHz by comparing high frequency to total frequency ratios,
349 // and separates them from the main signal with a frequency crossfade.
350 // Then inversely transforms both signals back, allowing for
351 // control over reducing sibilants in the time domain.
352 // -- own implementation including concepts from source B in regards to ↵
353 // FFT setup --
354 void AudioProcessorManager::fftBasedDeEssingAlgorithm(
355     juce::AudioBuffer<float> &buffer, const DeEssingParameters &p,
356     double sampleRate) {
357     const int fftSize = (int)p.fftWindowSize;

```

```

357     const float overlapFrac = p.fftOverlap;
358     const float detectionRatio = p.fftDetectionRatio;
359     float sibilantRangeStart = p.fftLowFreqCutoff;
360     float sibilantGain = juce::Decibels::decibelsToGain(p.fftMixLevel);
361     float fadeWidthHz = p.fftCrossfadeFreq;
362
363     DBG("[fftDeEss] sampleRate=" << sampleRate << " fftSize=" << fftSize
364         << " overlap=" << overlapFrac
365         << " freqThreshold=" << detectionRatio);
366
367     juce::AudioBuffer<float> mainBuffer;
368     mainBuffer.makeCopyOf(buffer);
369
370     juce::AudioBuffer<float> sibilantBufferTemp;
371     sibilantBufferTemp.makeCopyOf(buffer);
372
373     juce::AudioBuffer<float> finalBuffer;
374     finalBuffer.makeCopyOf(buffer);
375
376     const int numChannels = buffer.getNumChannels();
377     const int totalSamples = buffer.getNumSamples();
378
379     juce::AudioBuffer<float> dryBuffer;
380     dryBuffer.setSize(numChannels, totalSamples);
381     dryBuffer.clear();
382
383     sibilantBufferTemp.clear();
384
385     juce::dsp::FFT fft((int)std::log2(fftSize));
386     const int hopSize = (int)std::floor(fftSize * (1.0f - overlapFrac));
387
388     // Hann window
389     std::vector<float> window(fftSize);
390     for (int i = 0; i < fftSize; ++i)
391         window[i] = 0.5f * (1.0f - std::cos((2.0f * i) ←
392             juce::MathConstants<float>::pi * i) / (fftSize - 1)));
393
394     std::vector<float> fftTime(fftSize);
395     std::vector<float> fftTimeSib(fftSize, 0.0f);
396
397     int rangeStartBin =
398         (int)std::floor(sibilantRangeStart * (fftSize / sampleRate));
399     int fadeBins = (int)std::round(fadeWidthHz * (fftSize / sampleRate));
400     int fadeEndBin = rangeStartBin + fadeBins;
401
402     // process each channel
403     for (int channel = 0; channel < numChannels; ++channel) {

```

```

403     const float *audioData = mainBuffer.getReadPointer(channel);
404
405     std::vector<float> dryOverlap(totalSamples + fftSize, 0.0f);
406     std::vector<float> sibOverlap(totalSamples + fftSize, 0.0f);
407
408     // Iterate over frames
409     for (int start = 0; start <= totalSamples - fftSize; start += ↵
        hopSize) {
410         // Window the input
411         for (int i = 0; i < fftSize; ++i) {
412             jassert(start + i < (int)totalSamples);
413             fftTime[i] = audioData[start + i] * window[i];
414         }
415
416         // Forward FFT
417         fft.performRealOnlyForwardTransform(fftTime.data());
418
419         // calculate ratio
420         double totalEnergy = 0.0;
421         double hfEnergy = 0.0;
422         for (int bin = 0; bin < fftSize / 2; ++bin) {
423             float re = fftTime[2 * bin];
424             float im = fftTime[2 * bin + 1];
425             double mag2 = (double)re * re + (double)im * im;
426             totalEnergy += mag2;
427
428             float freqHz = bin * (sampleRate / fftSize);
429             if (freqHz >= sibilantRangeStart)
430                 hfEnergy += mag2;
431         }
432
433         double hfRatio = 0.0;
434         if (totalEnergy > 0.0) {
435             hfRatio = hfEnergy / totalEnergy;
436         }
437
438         std::fill(fftTimeSib.begin(), fftTimeSib.end(), 0.0f);
439
440         // entering sibilant mode
441         if (hfRatio > detectionRatio) {
442             int minBin = (int)std::floor(sibilantRangeStart * fftSize / ↵
                sampleRate);
443
444             int maxBin = fftSize / 2 - 1;
445             if (minBin < 1)
446                 minBin = 1;
447

```

```

448         for (int bin = minBin; bin <= maxBin; ++bin) {
449             float re = fftTime[2 * bin];
450             float im = fftTime[2 * bin + 1];
451
452             // crossfade until fadeEndBin
453             if (bin < fadeEndBin) {
454                 float fraction = (float)(bin - minBin) / ←
                     (float)(fadeBins);
455                 float fadeFactorSibilant = fraction;
456                 float fadeFactorRest = 1.0f - fraction;
457
458                 fftTimeSib[2 * bin] = re * fadeFactorSibilant;
459                 fftTimeSib[2 * bin + 1] = im * fadeFactorSibilant;
460
461                 fftTime[2 * bin] = re * fadeFactorRest;
462                 fftTime[2 * bin + 1] = im * fadeFactorRest;
463             }
464             // if not in crossfade region add full sibilant values ←
                     and zero out rest
465             else {
466                 fftTimeSib[2 * bin] = fftTime[2 * bin];
467                 fftTimeSib[2 * bin + 1] = fftTime[2 * bin + 1];
468
469                 fftTime[2 * bin] = 0.0f;
470                 fftTime[2 * bin + 1] = 0.0f;
471             }
472         }
473     }
474
475     // Inverse FFT
476     fft.performRealOnlyInverseTransform(fftTime.data());
477     fft.performRealOnlyInverseTransform(fftTimeSib.data());
478
479     // Overlap-add
480     for (int i = 0; i < fftSize; ++i) {
481         dryOverlap[start + i] += fftTime[i];
482         sibOverlap[start + i] += fftTimeSib[i];
483     }
484 }
485
486 // copy from overlap arrays to dryBuffer and sibBuffer
487 auto *dryData = dryBuffer.getWritePointer(channel);
488 auto *sibilantData = sibilantBufferTemp.getWritePointer(channel);
489 for (int i = 0; i < totalSamples; ++i) {
490     dryData[i] = dryOverlap[i];
491     sibilantData[i] = sibOverlap[i];
492 }

```

```

493     }
494
495     // mix sibilants with dry signal
496     for (int channel = 0; channel < numChannels; ++channel) {
497         auto *dryData = dryBuffer.getReadPointer(channel);
498         auto *sibilantData = sibilantBufferTemp.getReadPointer(channel);
499         auto *finalData = finalBuffer.getWritePointer(channel);
500
501         for (int i = 0; i < totalSamples; ++i) {
502             finalData[i] = dryData[i] + sibilantGain * sibilantData[i];
503         }
504     }
505
506     noSibilantBuffer.makeCopyOf(dryBuffer);
507     sibilantBuffer.makeCopyOf(sibilantBufferTemp);
508     processedBuffer.makeCopyOf(finalBuffer);
509
510     DBG("[fftDeEss] Done.");
511 }

```

A.2.8 SampleControl.h

```

1  /*
2      File: SampleControl.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Provides the interface for setting parameters for to the
8      Sample-based de-essing algorithm.
9
10     Sources:
11     Parameter handling concepts and setup were informed by the
12     SimpleMultiBandComp tutorial provided by Free Code Camp & Mat Kat ↵
13         Music LLC:
14     (Source C) Mat Kat Music LLC. SimpleMultiBandComp Tutorial (GitHub). ↵
15         Nov. 2021.
16     url: https://github.com/matkatmusic/SimpleMultiBandComp
17     (Source C) Mat Kat Music LLC. C++ Programming Tutorial - Build a ↵
18         3-Band Compressor Audio Plugin
19     (w/ JUCE Framework). Nov. 2021.
20     url: https://www.youtube.com/watch?v=Mo0Uco3Vimo&t=7276s
21
22     Mat Kat Music's code is licensed under the GNU General Public License.
23     https://www.gnu.org/licenses/gpl-3.0.en.html
24 */

```

```

22
23 #pragma once
24
25 #include <JuceHeader.h>
26
27 // -- derived from Source C --
28 class SampleControl : public juce::Component {
29     public:
30     SampleControl() {
31         addAndMakeVisible(frequencySlider);
32         frequencySlider.setRange(1000.0, 20000.0, 10.0);
33         frequencySlider.setValue(4000.0);
34         frequencySlider.setTextValueSuffix(" Hz highpass");
35         frequencySlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
36                                         20);
37
38         addAndMakeVisible(thresholdSlider);
39         thresholdSlider.setRange(-60.0, 0.0, 1.0);
40         thresholdSlider.setValue(-20.0);
41         thresholdSlider.setTextValueSuffix(" dB threshold");
42         thresholdSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
43                                         20);
44
45         addAndMakeVisible(reductionSlider);
46         reductionSlider.setRange(-60.0, 0.0, 1.0);
47         reductionSlider.setValue(-9.0);
48         reductionSlider.setTextValueSuffix(" dB reduction");
49         reductionSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
50                                         20);
51
52         addAndMakeVisible(hysteresisSlider);
53         hysteresisSlider.setRange(1.0, 500.0, 1.0);
54         hysteresisSlider.setValue(300.0);
55         hysteresisSlider.setTextValueSuffix(" smpIs hysteresis");
56         hysteresisSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, ↵
57                                         100,
58                                         20);
59     }
60
61     float getFrequency() const { return frequencySlider.getValue(); }
62     float getThreshold() const { return thresholdSlider.getValue(); }
63     float getReduction() const { return reductionSlider.getValue(); }
64     float getHysteresis() const { return hysteresisSlider.getValue(); }
65
66     void resized() override {
67         auto area = getLocalBounds();

```

```

68     frequencySlider.setBounds(area.removeFromTop(25));
69     thresholdSlider.setBounds(area.removeFromTop(25));
70     reductionSlider.setBounds(area.removeFromTop(25));
71     hysteresisSlider.setBounds(area.removeFromTop(25));
72 }
73
74 private:
75     juce::Slider frequencySlider;
76     juce::Slider thresholdSlider;
77     juce::Slider reductionSlider;
78     juce::Slider hysteresisSlider;
79
80     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(SampleControl)
81 };

```

A.2.9 RmsControl.h

```

1  /*
2      File: RmsControl.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Provides the interface for setting parameters for to the
8      RMS-based de-essing algorithm.
9
10     Sources:
11     Parameter handling concepts and setup were informed by the
12     SimpleMultiBandComp tutorial provided by Free Code Camp & Mat Kat ↵
13     Music LLC:
14     (Source C) Mat Kat Music LLC. SimpleMultiBandComp Tutorial (GitHub). ↵
15     Nov. 2021.
16     url: https://github.com/matkatmusic/SimpleMultiBandComp
17     (Source C) Mat Kat Music LLC. C++ Programming Tutorial - Build a ↵
18     3-Band Compressor Audio Plugin
19     (w/ JUCE Framework). Nov. 2021.
20     url: https://www.youtube.com/watch?v=Mo0Oco3Vimo&t=7276s
21
22     Mat Kat Music's code is licensed under the GNU General Public License.
23     https://www.gnu.org/licenses/gpl-3.0.en.html
24 */
25
26 #pragma once
27
28 #include <JuceHeader.h>

```

```

27 // -- derived from Source C --
28 class RmsControl : public juce::Component {
29     public:
30         RmsControl() {
31             // High-pass freq slider
32             addAndMakeVisible(frequencySlider);
33             frequencySlider.setRange(1000.0, 20000.0, 100.0);
34             frequencySlider.setValue(4000.0);
35             frequencySlider.setTextValueSuffix(" Hz highpass");
36             frequencySlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
37                                           20);
38
39             // Threshold slider
40             addAndMakeVisible(thresholdSlider);
41             thresholdSlider.setRange(-60.0, 0.0, 1.0);
42             thresholdSlider.setValue(-20.0);
43             thresholdSlider.setTextValueSuffix(" dB threshold");
44             thresholdSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
45                                           20);
46
47             // Reduction slider
48             addAndMakeVisible(reductionSlider);
49             reductionSlider.setRange(-60.0, 0.0, 1.0);
50             reductionSlider.setValue(-9.0);
51             reductionSlider.setTextValueSuffix(" dB reduction");
52             reductionSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
53                                           20);
54
55             // Hysteresis slider
56             addAndMakeVisible(hysteresisSlider);
57             hysteresisSlider.setRange(1.0, 30.0, 1.0);
58             hysteresisSlider.setValue(6.0);
59             hysteresisSlider.setTextValueSuffix(" dB hysteresis");
60             hysteresisSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, ↵
61                                           100,
62                                           20);
63
64             // Detection Window slider (ms)
65             addAndMakeVisible(detectionWindowSlider);
66             detectionWindowSlider.setRange(1.0, 50.0, 1.0);
67             detectionWindowSlider.setValue(10.0);
68             detectionWindowSlider.setTextValueSuffix(" ms window");
69             detectionWindowSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, ↵
70                                           true,
71                                           100, 20);
72
73             // Crossfade slider (ms)

```

```

72     addAndMakeVisible(crossfadeSlider);
73     crossfadeSlider.setRange(0.0, 20.0, 1.0);
74     crossfadeSlider.setValue(5.0);
75     crossfadeSlider.setTextValueSuffix(" ms crossfade");
76     crossfadeSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100,
77                                     20);
78 }
79
80 float getFrequency() const { return frequencySlider.getValue(); }
81 float getThreshold() const { return thresholdSlider.getValue(); }
82 float getReduction() const { return reductionSlider.getValue(); }
83 float getHysteresisDb() const { return hysteresisSlider.getValue(); }
84 float getDetectionWindow() const {
85     return detectionWindowSlider.getValue();
86 }
87 float getCrossfade() const { return crossfadeSlider.getValue(); }
88
89 void resized() override {
90     auto area = getLocalBounds();
91
92     frequencySlider.setBounds(area.removeFromTop(25));
93     thresholdSlider.setBounds(area.removeFromTop(25));
94     reductionSlider.setBounds(area.removeFromTop(25));
95     hysteresisSlider.setBounds(area.removeFromTop(25));
96     detectionWindowSlider.setBounds(area.removeFromTop(25));
97     crossfadeSlider.setBounds(area.removeFromTop(25));
98 }
99
100 private:
101     juce::Slider frequencySlider;
102     juce::Slider thresholdSlider;
103     juce::Slider reductionSlider;
104     juce::Slider hysteresisSlider;
105     juce::Slider detectionWindowSlider;
106     juce::Slider crossfadeSlider;
107
108     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(RmsControl)
109 };

```

A.2.10 FftControl.h

```

1  /*
2  File: FftControl.h
3  Author: Leif Benedikt Rehtanz
4  Date: 2024 / 2025
5

```

```

6      Description:
7      Provides the interface for setting parameters for to the
8      FFT-based de-essing algorithm.
9
10     Sources:
11     Parameter handling concepts and setup were informed by the
12     SimpleMultiBandComp tutorial provided by Free Code Camp & Mat Kat ↵
13         Music LLC:
14     (Source C) Mat Kat Music LLC. SimpleMultiBandComp Tutorial (GitHub). ↵
15         Nov. 2021.
16     url: https://github.com/matkatmusic/SimpleMultiBandComp
17     (Source C) Mat Kat Music LLC. C++ Programming Tutorial - Build a ↵
18         3-Band Compressor Audio Plugin
19     (w/ JUCE Framework). Nov. 2021.
20     url: https://www.youtube.com/watch?v=Mo00co3Vimo&t=7276s
21
22     Mat Kat Music's code is licensed under the GNU General Public License.
23     https://www.gnu.org/licenses/gpl-3.0.en.html
24 */
25
26 #pragma once
27
28 #include <JuceHeader.h>
29
30 // -- derived from Source C --
31 class FftControl : public juce::Component {
32 public:
33     FftControl() {
34         addAndMakeVisible(fftSizeCombo);
35         fftSizeCombo.addItem("512", 512);
36         fftSizeCombo.addItem("1024", 1024);
37         fftSizeCombo.addItem("2048", 2048);
38
39         // Default
40         fftSizeCombo.setSelectedId(1024, juce::dontSendNotification);
41
42         fftSizeCombo.onChange = [this]() {
43             int selectedID = fftSizeCombo.getSelectedId();
44             currentFftSize = selectedID;
45         };
46
47         // overlap slider
48         addAndMakeVisible(overlapSlider);
49         overlapSlider.setRange(0.0, 1.0, 0.1);
50         overlapSlider.setValue(0.5);
51         overlapSlider.setTextValueSuffix(" overlap");
52         overlapSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, ↵

```

```

100, 20);
50
51 // ratio slider
52 addAndMakeVisible(ratioSlider);
53 ratioSlider.setRange(0.0, 1.0, 0.01);
54 ratioSlider.setValue(0.2);
55 ratioSlider.setTextValueSuffix(" ratio");
56 ratioSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, 100, ↵
    20);
57
58 // sibilant start
59 addAndMakeVisible(lowFreqSlider);
60 lowFreqSlider.setRange(1000.0, 6000.0, 100.0);
61 lowFreqSlider.setValue(4000.0);
62 lowFreqSlider.setTextValueSuffix(" Hz start");
63 lowFreqSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true, ↵
    100, 20);
64
65 // crossfade width
66 addAndMakeVisible(crossfadeFreqSlider);
67 crossfadeFreqSlider.setRange(0.0, 1000.0, 100.0);
68 crossfadeFreqSlider.setValue(500.0);
69 crossfadeFreqSlider.setTextValueSuffix(" Hz crossfade");
70 crossfadeFreqSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, true,
71                                     100, 20);
72
73 // sibilant gain reduction
74 addAndMakeVisible(sibilantGainSlider);
75 sibilantGainSlider.setRange(-60.0, 0.0, 1.0);
76 sibilantGainSlider.setValue(-9.0);
77 sibilantGainSlider.setTextValueSuffix(" dB reduction");
78 sibilantGainSlider.setTextBoxStyle(juce::Slider::TextBoxLeft, ↵
    true, 100,
79                                     20);
80
81 currentFftSize = fftSizeCombo.getSelectedId();
82 }
83
84 float getWindowSize() const { return (float)currentFftSize; }
85 float getOverlap() const { return (float)overlapSlider.getValue(); }
86 float getSibilantGainDb() const {
87     return (float)sibilantGainSlider.getValue();
88 }
89 float getRatioThreshold() const { return ratioSlider.getValue(); }
90 float getLowFreq() const { return lowFreqSlider.getValue(); }
91 float getCrossfadeFreq() const { return ↵
    crossfadeFreqSlider.getValue(); }

```

```

92
93     void resized() override {
94         auto area = getLocalBounds().reduced(10);
95
96         fftSizeCombo.setBounds(area.removeFromTop(25));
97         overlapSlider.setBounds(area.removeFromTop(25));
98         ratioSlider.setBounds(area.removeFromTop(25));
99         lowFreqSlider.setBounds(area.removeFromTop(25));
100        crossfadeFreqSlider.setBounds(area.removeFromTop(25));
101        sibilantGainSlider.setBounds(area.removeFromTop(25));
102    }
103
104 private:
105     juce::ComboBox fftSizeCombo;
106     int currentFftSize = 1024;
107
108     juce::Slider overlapSlider;
109     juce::Slider sibilantGainSlider;
110
111     juce::Slider ratioSlider;
112     juce::Slider lowFreqSlider;
113     juce::Slider crossfadeFreqSlider;
114
115     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(FftControl)
116 };

```

A.2.11 PositionOverlay.h

```

1  /*
2      File: PositionOverlay.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Draws a vertical line at the current playback position. Allows the ↔
        user to
8      click and drag to seek. Uses Timer to paint the position line ↔
        periodically.
9
10     Sources:
11     This part of the code was copied from the JUCE tutorial "Draw Audio ↔
        Waveforms":
12     (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
13     url: https://docs.juce.com/master/tutorial\_audio\_thumbnail.html
14
15     License:

```

```

16     Based on open-source examples under the ISC License.
17     http://www.isc.org/downloads/software-support-policy/isc-license.
18 */
19
20 #pragma once
21
22 #include <JuceHeader.h>
23
24 // -- copied from Source A --
25 class PositionOverlay : public juce::Component, private juce::Timer {
26 public:
27     PositionOverlay(juce::AudioTransportSource &transportSourceToUse);
28
29     void paint(juce::Graphics &g) override;
30     void mouseDown(const juce::MouseEvent &event) override;
31
32 private:
33     void timerCallback() override;
34
35     juce::AudioTransportSource &transportSource;
36
37     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(PositionOverlay)
38 };

```

A.2.12 PositionOverlay.cpp

```

1  /*
2     File: PositionOverlay.cpp
3     Author: Leif Benedikt Rehtanz
4     Date: 2024 / 2025
5
6     Description:
7     Draws a vertical line at the current playback position. Allows the ↔
8     user to
9
10    click and drag to seek. Uses Timer to paint the position line ↔
11    periodically.
12
13    Sources:
14    This part of the code was copied from the JUCE tutorial "Draw Audio ↔
15    Waveforms":
16    (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
17    url: https://docs.juce.com/master/tutorial_audio_thumbnail.html
18
19    License:
20    Based on open-source examples under the ISC License.
21    http://www.isc.org/downloads/software-support-policy/isc-license.

```

```
18  */
19
20  #include "PositionOverlay.h"
21
22  // Constructor - takes a reference to the transport source.
23  // -- copied from Source A --
24  PositionOverlay::PositionOverlay(
25      juce::AudioTransportSource &transportSourceToUse)
26      : transportSource(transportSourceToUse) {
27      startTimer(40);
28  }
29
30  // Draws a line at (currentPosition / totalDuration) * width.
31  // -- copied from Source A --
32  void PositionOverlay::paint(juce::Graphics &g) {
33      auto duration = (float)transportSource.getLengthInSeconds();
34
35      if (duration > 0.0) {
36          auto audioPosition = (float)transportSource.getCurrentPosition();
37          auto drawPosition = (audioPosition / duration) * (float)getWidth();
38
39          g.setColour(juce::Colours::green);
40          g.drawLine(drawPosition, 0.0f, drawPosition, (float)getHeight(), ←
41                  2.0f);
42      }
43
44  // Lets the user click on the overlay to jump the playhead to that point.
45  // -- copied from Source A --
46  void PositionOverlay::mouseDown(const juce::MouseEvent &event) {
47      auto duration = transportSource.getLengthInSeconds();
48
49      if (duration > 0.0) {
50          auto clickPosition = event.position.x;
51          auto audioPosition = (clickPosition / (float)getWidth()) * duration;
52
53          transportSource.setPosition(audioPosition);
54      }
55  }
56
57  // Calls repaint() every 40 ms, so the overlay keeps updating in real-time.
58  // -- copied from Source A --
59  void PositionOverlay::timerCallback() { repaint(); }
```

A.2.13 WaveformDisplay

```
1  /*
2      File: WaveformDisplay.h
3      Author: Leif Benedikt Rehtanz
4      Date: 2024 / 2025
5
6      Description:
7      Header file defining the WaveformDisplay class for waveform
8      visualization and sibilant overlays. It uses juce::AudioThumbnail
9      to draw waveforms.
10
11     Sources:
12     The visualization logic was adapted from the JUCE tutorial "Draw Audio ↵
13         Waveforms":
14     (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
15     url: https://docs.juce.com/master/tutorial\_audio\_thumbnail.html
16
17     License:
18     Based on open-source examples under the ISC License.
19     http://www.isc.org/downloads/software-support-policy/isc-license.
20 */
21 #pragma once
22
23 #include <JuceHeader.h>
24
25 // A component that uses juce::AudioThumbnail to draw a waveform of the ↵
26 // loaded
27 // file.
28 // -- copied from Source A with minor modifications --
29 class WaveformDisplay : public juce::Component, private juce::ChangeListener ↵
30 {
31     public:
32     WaveformDisplay(int sourceSamplesPerWaveformSample,
33                     juce::AudioFormatManager &formatManager,
34                     juce::AudioThumbnailCache &cache);
35
36     void setFile(const juce::File &file);
37     void paint(juce::Graphics &g) override;
38
39     void setSibilantBuffer(const juce::AudioBuffer<float> &buffer);
40
41     private:
42     juce::AudioBuffer<float> sibilantBuffer;
43     bool hasSibilantData = false;
44
45     void paintIfNoFileLoaded(juce::Graphics &g);
46     void paintIfFileLoaded(juce::Graphics &g);
```

```

45     void changeListenerCallback(juce::ChangeBroadcaster *source) override;
46     void waveformChanged();
47
48     juce::AudioThumbnail waveform;
49
50     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(WaveformDisplay)
51 };

```

A.2.14 WaveformDisplay.cpp

```

1  /*
2     File: WaveformDisplay.cpp
3     Author: Leif Benedikt Rehtanz
4     Date: 2024 / 2025
5
6     Description:
7     Implements waveform visualization and sibilant overlay functionality.
8     It uses juce::AudioThumbnail to draw waveforms.
9
10    Sources:
11    The visualization logic was adapted from the JUCE tutorial "Draw Audio ↵
12        Waveforms":
13    (Source A) Raw Material Software Limited. "Draw Audio Waveforms". 2024.
14    url: https://docs.juce.com/master/tutorial\_audio\_thumbnail.html
15
16    License:
17    Based on open-source examples under the ISC License.
18    http://www.isc.org/downloads/software-support-policy/isc-license.
19 */
20 #include "WaveformDisplay.h"
21
22 // Constructor
23 // -- copied from Source A --
24 WaveformDisplay::WaveformDisplay(int sourceSamplesPerWaveformSample,
25                                   juce::AudioFormatManager &formatManager,
26                                   juce::AudioThumbnailCache &cache)
27     : waveform(sourceSamplesPerWaveformSample, formatManager, cache) {
28     waveform.addChangeListener(this);
29 }
30
31 // Sets the thumbnail source to a .wav file.
32 // -- copied from Source A --
33 void WaveformDisplay::setFile(const juce::File &file) {
34     waveform.setSource(new juce::FileInputSource(file));
35 }

```

```

36
37 // Accepts an in-memory buffer of detected sibilants for the red overlay.
38 void WaveformDisplay::setSibilantBuffer(
39     const juce::AudioBuffer<float> &buffer) {
40     sibilantBuffer.makeCopyOf(buffer);
41     hasSibilantData = true;
42
43     juce::MessageManager::callAsync([this]() { repaint(); });
44 }
45
46 // Handles paint events for the waveform display if a file is loaded or not.
47 // -- copied from Source A --
48 void WaveformDisplay::paint(juce::Graphics &g) {
49     if (waveform.getNumChannels() == 0)
50         paintIfNoFileLoaded(g);
51     else
52         paintIfFileLoaded(g);
53 }
54
55 // Shows a message if no file is loaded.
56 // -- copied from Source A --
57 void WaveformDisplay::paintIfNoFileLoaded(juce::Graphics &g) {
58     g.fillAll(juce::Colours::white);
59     g.setColour(juce::Colours::darkgrey);
60     g.drawFittedText("No File Loaded", getLocalBounds(),
61                     juce::Justification::centred, 1);
62 }
63
64 // Draws the main waveform in blue, then overlays any sibilant data in red.
65 // -- derived from Source A --
66 void WaveformDisplay::paintIfFileLoaded(juce::Graphics &g) {
67     g.fillAll(juce::Colours::white);
68     g.setColour(juce::Colours::blue);
69
70     // only draw one channel
71     if (waveform.getNumChannels() > 0) {
72         waveform.drawChannel(g, getLocalBounds(), 0.0,
73                             waveform.getTotalLength(), 0, 1.0f);
74     }
75
76     // Draw sibilants as overlay
77     if (hasSibilantData && sibilantBuffer.getNumChannels() > 0) {
78         g.setColour(juce::Colours::red);
79         auto bounds = getLocalBounds();
80         auto numSamples = sibilantBuffer.getNumSamples();
81         auto *data = sibilantBuffer.getReadPointer(0);
82

```

```

83     int totalWidth = bounds.getWidth();
84     if (totalWidth <= 0 || numSamples == 0)
85         return;
86
87     // Determine how many samples correspond to one vertical line
88     int step = std::max(1, numSamples / totalWidth);
89
90     juce::Path sibilantPath;
91     // We'll draw a vertical line per pixel column representing the ↔
92     // min/max
93     // in that segment
94     for (int x = 0; x < totalWidth; ++x) {
95         int start = x * step;
96         int end = std::min(start + step, numSamples);
97
98         float minVal = std::numeric_limits<float>::max();
99         float maxVal = std::numeric_limits<float>::lowest();
100
101         // Find min and max in this chunk
102         for (int i = start; i < end; ++i) {
103             float sampleVal = data[i];
104             if (sampleVal < minVal)
105                 minVal = sampleVal;
106             if (sampleVal > maxVal)
107                 maxVal = sampleVal;
108         }
109
110         // Convert min/max to vertical coordinates
111         float yMin = bounds.getHeight() * (0.5f - 0.5f * maxVal);
112         float yMax = bounds.getHeight() * (0.5f - 0.5f * minVal);
113
114         // Draw a vertical line from min to max
115         sibilantPath.startNewSubPath((float)x, yMin);
116         sibilantPath.lineTo((float)x, yMax);
117     }
118
119     g.strokePath(sibilantPath, juce::PathStrokeType(1.0f));
120 }
121
122 // Callback function that checks if the waveform data changes.
123 // -- copied from Source A --
124 void WaveformDisplay::changeListenerCallback(juce::ChangeBroadcaster ↔
125     *source) {
126     if (source == &waveform)
127         waveformChanged();
128 }

```

```
128  
129 // Repaints the waveform display.  
130 // -- copied from Source A --  
131 void WaveformDisplay::waveformChanged() { repaint(); }
```

Abkürzungsverzeichnis

ARA	Audio Random Access
Buffer	Pufferspeicher
Buffer Size	Puffer-Größe
COLA	Constand Overlap Add
DAW	Digital Audio Workstation
dB	Dezibel
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing
EQ	Equalizer
FFT	Fast Fourier Transform
JUCE	Jules' Utility Class Extensions
RMS	Root Mean Square
Waveform	Wellenform

Abbildungsverzeichnis

2.1	Spektrogramm in iZotope RX - links rot markiert Saitenquietschen, rechts rot markiert Sibilanten. Quelle: Eigene Auswertung.	7
2.2	Links Spektrogramm mit Sibilant in rot, rechts Waveform des selben Sibilanten hochfrequent in der Mitte. Quelle: Eigene Auswertung.	8
2.3	Waveform Abbildung von einem regulären und einem phasengedrehten Signal. Quelle: Eigene Auswertung.	10
3.1	Spektrogramm-Darstellung Überlappung (in rot) von Sibilant und tonalem Material. Quelle: Eigene Auswertung.	16
3.2	Komplementäre Hoch- und Tiefpassfilter in einem Equalizer. Quelle: Eigene Auswertung.	17
3.3	Sibilant (mittig) und tonales Material (links und rechts) im Spektrogramm. Quelle: Eigene Auswertung.	18
4.1	Überblick der Architektur der Applikation. Quelle: Eigene Auswertung. . .	24
4.2	Bildschirmaufnahme der Testapp nach Verarbeitung einer Gesangsspur durch den FFT-based Algorithmus. Quelle: Eigene Auswertung.	26
4.3	Datenfluss des Sample-based Algorithmus. Quelle: Eigene Auswertung. . .	28
4.4	Datenfluss des RMS-based Algorithmus. Quelle: Eigene Auswertung.	30
4.5	Datenfluss des FFT-based Algorithmus. Quelle: Eigene Auswertung.	31
4.6	Auswirkung der Crossfade Implementierung auf das Aliasing im Spektrogramm. Quelle: Eigene Auswertung.	32
5.1	Vergleich der Erkennung anhand der Waveform der DAW und Beispielapplikation. Quelle: Eigene Auswertung.	37
5.2	Vergleich der Trennung anhand des Spektrogramms. Quelle: Eigene Auswertung.	39
5.3	Vergleich der Nulltests im Spektrogramm. Quelle: Eigene Auswertung. . . .	40
A.1	Beobachtungen Tests: Phase 1 (1)	46
A.2	Beobachtungen Tests: Phase 1 (2)	47
A.3	Beobachtungen Tests: Phase 2 (1)	48
A.4	Beobachtungen Tests: Phase 2 (2)	49
A.5	Beobachtungen Tests: Phase 2 (3) - Fall Akustikgitarre	50
A.6	Beobachtungen Tests: Phase 2 (4) - Fall Akustikgitarre	50

Tabellenverzeichnis

3.1 Zusammenfassung der Testkriterien 1–4 für die Bewertung.	21
--	----

Literaturverzeichnis

- [1] Matt Houghton. *Managing Sibilance - How to Stop It Affecting the Quality of Your Mixes*. Online-Artikel. letzter Zugriff: 01. Februar 2025. 2020.
URL: <https://www.soundonsound.com/techniques/managing-sibilance>.
- [2] Classical Guitar Corner. *How to Stop Finger Noise "Squeaks" on Guitar*. Online-Artikel. letzter Zugriff: 03. Februar 2025. 2021.
URL: <https://www.classicalguitarcorner.com/1305-finger-noise-on-the-classical-guitar/>.
- [3] Nick Messitte. *What Is De-Essing? The Dos and Don'ts of Using a De-Esser*. Online-Artikel. letzter Zugriff: 01. Februar 2025. 2024.
URL: <https://www.izotope.com/en/learn/the-dos-and-donts-of-de-essing.html>.
- [4] iZotope Inc. *RX 11 - De-Ess*. Website. letzter Zugriff: 02. Februar 2025. 2024.
URL: <https://www.izotope.com/en/products/rx/features/de-ess.html>.
- [5] William C. Pirkle. *Designing Audio Effect Plugins in C++: For AAX, AU, and VST3 with DSP Theory*. 2. Auflage. New York, NY: Routledge, 2019. ISBN: 9781138591899.
- [6] Applied Math. *Digital Signal Processing (DSP)*. Online-Artikel. letzter Zugriff: 03. Februar 2025.
URL: https://www.applied-math.com/index.php?option=com_content&view=article&id=21&Itemid=141.
- [7] Mixing Monster. *Nyquist Frequency Explained / Proper Signal Sampling in 2025*. Online-Artikel. letzter Zugriff: 01. Februar 2025. 2025.
URL: <https://mixingmonster.com/nyquist-frequency-explained/>.
- [8] Tom Poole. *JUCE on GitHub*. Online-Dokumentation. letzter Zugriff: 30. Januar 2025. 2025.
URL: <https://github.com/juce-framework/JUCE>.
- [9] Raw Material Software Limited. *JUCE Documentation*. letzter Zugriff: 09. Januar 2025. 2025.
URL: <https://juce.com/learn/documentation>.
- [10] Sonible. *Was ist ARA?* Online-Artikel. letzter Zugriff: 27. Januar 2025.
URL: <https://www.sonible.com/de/blog/was-ist-ara/>.
- [11] Nick Messitte. *What Is ARA? How to Make Audio Production Easier in Your DAW*. Online-Artikel. letzter Zugriff: 02. Februar 2025. 2024.
URL: <https://www.izotope.com/en/learn/what-is-ara.html>.

- [12] Paul Heckbert. *Fourier Transforms and the Fast Fourier Transform (FFT) Algorithm*. Vorlesungsskript, Carnegie Mellon University (Computer Graphics 15-463). Überarbeitet 27 Januar 1998. 1995.
URL: <https://www.cs.cmu.edu/afs/andrew/scs/cs/15-463/99/pub/www/notes/fourier/fourier.pdf>.
- [13] Leo McCormack und Vesa Välimäki. „FFT-based dynamic range compression“. In: *Proceedings of the 14th Sound and Music Computing Conference*. Aalto University. 2017, S. 42–49.
URL: http://smc2017.aalto.fi/media/materials/proceedings/SMC17_p42.pdf.
- [14] Ethan Laptew und Yufeng Lu. „Work in Progress: A Simple Plug-in Implementation of Extreme Audio Stretching“. In: *2023 ASEE Illinois-Indiana Section Conference Proceedings*. ECE Department, Bradley University. 2023.
URL: <https://peer.asee.org/45108.pdf>.
- [15] Vicki R. Melchior. „High Resolution Audio: A History and Perspective“. In: *Journal of the Audio Engineering Society* 67.5 (2019), S. 246–257.
URL: <https://doi.org/10.17743/jaes.2018.0056>.
- [16] Karl Heinz Wagner. *Phonetik und Phonologie: Kapitel 5 – Oro-nasaler Prozess. Der Artikulationsprozeß – Artikulationsart*. Online-Artikel. letzter Zugriff: 12. Februar 2025.
URL: <http://www.fb10.uni-bremen.de/khwagner/phonetik/kapitel6.aspx>.
- [17] Carlos San Segundo. *FAQ: Filter-Typen – Filter-Typ und was er macht*. Online-Artikel. letzter Zugriff: 25. Februar 2025. 2025.
URL: <https://www.delamar.de/faq/faq-filter-typen-filter-typ-und-was-er-macht-9486/>.
- [18] Mat Kat Music LLC. *C++ Programming Tutorial - Build a 3-Band Compressor Audio Plugin (w/ JUCE Framework)*. YouTube-Video. letzter Zugriff: 30. Januar 2025. 2021.
URL: <https://www.youtube.com/watch?v=Mo00co3Vimo&t=7276s>.
- [19] Felix Baarß. *Kammfiltereffekt – Was ist das? Definition & Beispiele*. Online-Artikel. letzter Zugriff: 25. Februar 2025. 2025.
URL: <https://www.delamar.de/faq/kammfiltereffekt-31609/>.
- [20] Steinberg. *Null-Test-Spuren in der Audio-Montage – WaveLab Pro 12.0*. Online-Dokumentation. letzter Zugriff: 25. Februar 2025. 2025.
URL: https://www.steinberg.help/r/wavelab-pro/12.0/de/wavelab/topics/audio_montage/audio_montage_null_test_tracks_c.html.
- [21] FabFilter. *Pro-DS De-Esser Plug-in*. Website. letzter Zugriff: 30. Januar 2025.
URL: <https://www.fabfilter.com/products/pro-ds-de-esser-plug-in>.
- [22] Waves Audio. *DeEsser Plug-in*. Website. letzter Zugriff: 30. Januar 2025.
URL: <https://www.waves.com/plugins/deesser>.

- [23] Sonible. *smart:deess - Der intelligente De-Esser*. Website. letzter Zugriff: 20. Februar 2025.
URL: <https://www.sonible.com/de/smartdeess/>.
- [24] Alexander Eberz. *iZotope RX 11 Test*. Online-Artikel. letzter Zugriff: 20. Februar 2025. 2024.
URL: <https://www.bonedo.de/artikel/izotope-rx-11-test/>.
- [25] Mix and Master My Song. *Understanding RMS in Audio – What It Is and Why It Matters*. Online-Artikel. letzter Zugriff: 25. Februar 2025. 2024.
URL: <https://mixandmastermysong.com/understanding-rms-in-audio/>.
- [26] Nick Messitte. *What Is a Noise Gate and How to Use It*. Online-Artikel. letzter Zugriff: 20. Februar 2025. 2021.
URL: <https://www.izotope.com/en/learn/noise-gates.html>.
- [27] Sören Rosenbaum. „Entwurf elektromagnetischer Aktoren unter Berücksichtigung von Hysterese“. Diss. Ilmenau: Technische Universität Ilmenau, 2011.
URL: <https://nbn-resolving.org/urn:nbn:de:gbv:ilm1-2011000261>.
- [28] Raw Material Software Limited. *Tutorial: Projucer Part 1: Getting Started with the Projucer*. Online-Tutorial. letzter Zugriff: 27. Januar 2025.
URL: https://docs.juce.com/master/tutorial_new_projucer_project.html.
- [29] Markus Huber. *Multiband Compression for Ambisonics*. Abschlussarbeit. Institut für Elektronische Musik und Akustik, Graz, 2020.
URL: <https://phaidra.kug.ac.at/detail/o:104573.pdf>.
- [30] Raw Material Software Limited. *dsp::FFT Class*. letzter Zugriff: 27. Januar 2025.
URL: https://docs.juce.com/master/classdsp_1_1FFT.html.
- [31] Prajoy Podder u. a. „Comparative Performance Analysis of Hamming, Hanning and Blackman Window“. In: *International Journal of Computer Applications* 96.18 (2014).
- [32] Raw Material Software Limited. *Tutorial: Draw Audio Waveforms*. Online-Tutorial. letzter Zugriff: 27. Januar 2025.
URL: https://docs.juce.com/master/tutorial_audio_thumbnail.html.
- [33] Mat Kat Music LLC. *SimpleMultiBandComp Tutorial (GitHub)*. GitHub-Repository. letzter Zugriff: 30. Januar 2025. 2021.
URL: <https://github.com/matkatmusic/SimpleMultiBandComp>.
- [34] Raw Material Software Limited. *AudioTransportSource Class*. Online-Dokumentation. letzter Zugriff: 27. Januar 2025.
URL: <https://docs.juce.com/master/classAudioTransportSource.html>.
- [35] Raw Material Software Limited. *PositionableAudioSource Class*. Online-Dokumentation. letzter Zugriff: 27. Januar 2025.
URL: <https://docs.juce.com/master/classPositionableAudioSource.html>.
- [36] Raw Material Software Limited. *AudioBuffer<Type> Class*. Online-Dokumentation. letzter Zugriff: 27. Januar 2025.
URL: <https://docs.juce.com/master/classAudioBuffer.html>.

- [37] Raw Material Software Limited. *Tutorial: The fast Fourier transform*. Online-Tutorial. letzter Zugriff: 27. Januar 2025.
URL: https://docs.juce.com/master/tutorial_simple_fft.html.

Eidesstattliche Versicherung

Hiermit versichere ich an Eides statt durch meine Unterschrift, dass ich die vorstehende Arbeit selbstständig und ohne fremde Hilfe angefertigt und alle Stellen, die ich wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen habe, als solche kenntlich gemacht habe, mich auch keiner anderen als der angegebenen Literatur oder sonstiger Hilfsmittel bedient habe. Die Arbeit hat in dieser oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

Berlin, den 04.03.2025

A handwritten signature in black ink, appearing to read 'L. Rehtanz', with a stylized, flowing script.

Leif Benedikt Rehtanz